



Reykjavik University

DEPARTMENT OF
COMPUTER SCIENCE



University of Camerino

SCHOOL OF SCIENCE AND
TECHNOLOGY

Master's degree in Computer Science (LM-18)

Support for the design and deployment of IoT scenarios through Agentic AI

Candidate

Damiano Buzzo

Supervisor

Prof. Massimo Callisto De Donato

Student ID

131989

Co-Supervisor

Prof. Stefán Ólafsson

A.Y. 2025/2026

Contents

1	Introduction	1
1.1	Problem statement	1
1.2	Research contribution	2
1.3	Objectives	2
1.4	Thesis structure	3
2	Background	5
2.1	Internet of Things	5
2.1.1	Sensing layer and physical metrics	5
2.1.2	Networking layer	6
2.1.3	Data analytics and processing	6
2.1.4	IoT platforms	7
2.2	Building modeling	8
2.3	SAIBA framework	9
2.3.1	Stages of the SAIBA architecture	9
2.3.2	From conversational agents to IoT configuration	9
2.4	Agentic AI	10
2.4.1	Conceptual taxonomy and differences	10
2.4.2	Core components	12
2.4.3	Types of architectures	12
2.4.4	Keeping agents within bounds	15
2.5	Agent interoperability protocols	16
2.5.1	Model Context Protocol (MCP)	17
2.5.2	Agent To Agent(A2A)	17
2.5.3	Agent Communication Protocol (ACP)	19
2.5.4	Agent Network Protocol (ANP)	20
2.5.5	Structural patterns and combinations	20
2.6	Agentic AI frameworks	21
2.6.1	LangGraph	21
2.6.2	LangChain	23
2.6.3	LlamaIndex	23
2.6.4	Microsoft Agent Framework	24
2.6.5	CrewAI	24
2.6.6	OpenAI Agents SDK	24

2.6.7	Comparative evaluation	25
3	Literature Review	27
3.1	Data collection methodologies	28
3.1.1	Inclusion and exclusion criteria	28
3.2	Discussion	29
3.2.1	Natural Language Intent and Architectural Hierarchy	29
3.2.2	Deployment Automation and Workflow Resilience	30
3.2.3	Context Tracking and Autonomous Reasoning	31
3.3	Key findings and gaps	32
3.3.1	Key findings	32
3.3.2	Identified gaps	32
4	Research Methodology	35
4.1	Design science as the research frame	35
4.2	Process model and structure of the work	36
4.3	Design, development and evaluation	37
4.4	Ethical considerations and limitations	37
5	Problem Awareness	39
5.1	Reference scenario	41
5.2	Practical configuration needs of an IoT architect	43
5.3	Derived requirements	44
6	Artifact Design	47
6.1	Conceptual model of an IoT installation	47
6.2	Device catalogue	49
6.3	From natural language to a configuration	50
6.4	Orchestration architecture	51
6.5	Final design	53
6.6	Example workflows	54
6.6.1	Nominal configuration of a wellness installation	54
6.6.2	Incomplete description and failed validation	56
6.6.3	Suspension, resumption and partial failure	57
7	Artifact Implementation	59
7.1	Technologies	59
7.1.1	Orchestration runtime	60
7.1.2	Language model access	61
7.1.3	Data modelling and the knowledge base	62
7.1.4	Platform access	62
7.1.5	Interface and persistence	62
7.2	Structure of the code and its implementation	63
7.2.1	Layout and requirements	63
7.2.2	The shared state and the graph	64

7.2.3	The conceptual model	66
7.2.4	Routing	66
7.2.5	Conceptualisation: constrained extraction and grounding	68
7.2.6	Configuration: intents bounded by the catalogue	71
7.2.7	The validation engine	73
7.2.8	Behaviour planning	73
7.2.9	The platform adapters	74
7.2.10	Deployment: interruption, confirmation and resumption	76
7.2.11	One reply per turn	77
7.2.12	The interface	78
7.3	Results	80
7.3.1	Describing a building and completing the description	80
7.3.2	Reviewing the configuration	82
7.3.3	Consulting the catalogue and equipping the spaces	82
7.3.4	Checking before deploying	84
7.3.5	Planning, confirming and deploying	84
7.3.6	The same model on two platforms	85
7.3.7	Interruption and resumption	86
7.3.8	Traceability	87
7.3.9	Summary of the realised capabilities	87
8	Evaluation	89
8.1	Evaluation protocol	89
8.1.1	Research question	89
8.1.2	Participants	89
8.1.3	Tasks	89
8.1.4	Session	90
8.1.5	Procedure	90
8.1.6	Analysis and limitations	92
8.2	Conduct of the study	92
8.2.1	Participants	92
8.2.2	Time on task	93
8.3	Usability	93
8.3.1	System Usability Scale	93
8.3.2	Task difficulty and additional items	95
8.4	Overview by participant	95
8.5	Summary	96
9	Discussion	97
9.1	Revisiting the identified gaps	97
9.2	Answers to the research questions	98
9.3	Limitations	99
9.3.1	Scalability	99
9.3.2	Coverage and dependence on the model	100

9.3.3 Evaluation and operation	100
9.4 Implications	100
10 Conclusion	103
10.1 Contributions	103
10.2 Future work	104
Acknowledgements	105
Declaration of Originality and Use of AI	107

List of Figures

2.1	Pavatar architecture[10]	5
2.2	The same installation as held by ThingsBoard and by OpenRemote.	8
2.3	SAIBA architecture[9]	9
2.4	An AI agent and an agentic system in a smart home. Reproduced from [6], licensed under CC BY 4.0.	11
2.5	ReAct Single-Agent architecture[5]	13
2.6	Supervisor/hierarchical architecture[5]	14
2.7	MCP architecture[28]	18
2.8	A2A architecture[28]	19
2.9	MCP+A2A integration[3]	20
2.10	How LangGraph executes an application	23
4.1	The three cycles of design science [47] as instantiated in this work.	35
4.2	The design cycle followed by this work [49], with what each phase produced and where it is reported.	36
5.1	The configuration of an IoT installation today and with the proposed system	40
5.2	The Computer Science Building of the University of Camerino, the reference scenario of this work.	41
5.3	The two scenarios used in the evaluation, in the terms of the conceptual model.	42
6.1	An abstract view of the architecture designed in this chapter	47
6.2	Proposed architecture of the artifact.	54
6.3	Nominal configuration of a wellness installation.	56
6.4	Incomplete description and failed validation.	57
6.5	Suspension, resumption and partial failure. Solid arrows leaving the supervisor are delegations; dashed arrows are returns, to the supervisor within a turn or to the architect at its end.	58
7.1	Linear execution in LangChain and cyclic execution in LangGraph. Reproduced from [56].	60
7.2	Openrouter functioning [57]	61
7.3	A building described in natural language, and the model derived from it.	80
7.4	Clarification requested from the rules, and the model after the answer.	81

7.5	The configuration rendered as a table and as a topology diagram.	82
7.6	The catalogue, answered out of the declarative knowledge base	83
7.7	Assignment across every space of a type, with the defaults applied stated explicitly.	83
7.8	The pre-deployment check, with blocking errors distinguished from warnings.	84
7.9	The deployment plan, suspended on the architect's confirmation, abridged.	85
7.10	The plan after confirmation, with the outcome of each operation, abridged	85
7.11	The same installation as realised on ThingsBoard (a) and on OpenRemote (b).	86
7.12	A configuration can be retrieved and continued in a later session through the sidebar selection	86
7.13	The activity trace of a session.	87
8.1	SUS score of each participant, with the mean and its 95% confidence interval. The dashed line marks the benchmark of 68.	94
8.2	Difficulty of each task and responses to items 11 and 12, one dot per participant; the vertical bar marks the median. Higher is better on every row.	95

List of Tables

2.1	Generative AI, AI agents and Agentic AI, adapted from [6].	11
2.2	Comparison of agentic AI frameworks.	25
3.1	Comparative evaluation of literature approaches based on system characteristics.	29
4.1	Correspondence between research questions and the parts of the work that address them.	37
5.1	Characteristics of the reference scenario.	41
5.2	Use-case profiles considered in the reference scenario.	42
5.3	Mapping between the derived requirements and the evaluation characteristics introduced in Chapter 3	45
6.1	Entities of the conceptual model of an IoT installation.	48
6.2	Structure of a device type in the catalogue.	49
6.3	Mapping of the proposed components onto the stages of the SAIBA framework.	52
7.1	Technologies adopted, and what each answers for.	59
7.2	Where each requirement of Section 5.3 is realised.	64
7.3	Capabilities of the implemented artifact.	88
8.1	Task parameters, stated so that the two tasks can be compared and reproduced.	90
8.2	Session structure.	91
8.3	Background of the participants ($n = 7$).	93
8.4	Time on task, in minutes. <i>At the limit</i> counts the participants stopped when the time allowed ran out.	93
8.5	Usability measures ($n = 7$). Higher is better on every scale.	94
8.6	Mean response to each SUS item ($n = 7$). For the negatively worded items, marked †, a lower mean is better.	94
8.7	Results by participant. All participants completed both tasks. Difficulty on a scale from 1, very difficult, to 5, very easy; time in minutes, with limits of 12 for S1 and 6 for S2.	95

Abstract

Configuring an IoT installation inside a real building is still a largely manual activity. The architect reasons about what the installation should achieve, while the platform that will operate it must be told, entity by entity, how that result is obtained. Bridging the two is slow, repetitive and error-prone, and the difficulty grows with the heterogeneity of devices and platforms.

Following a design science approach, this thesis investigates how Agentic Artificial Intelligence can act as an intermediation layer between these two levels. The proposed artifact accepts a description of a building in natural language, builds a model of it by asking for what is missing rather than assuming it, equips its spaces with devices drawn from a declarative catalogue, checks the result against the rules of the selected use case and, once the architect has confirmed the plan, creates the corresponding entities on the target platform. The work is divided among specialised agents, coordinated by a supervisor over a shared and persistent state, so that a configuration can be reviewed, corrected and resumed at any point. Inspired by the SAIBA framework, the reasoning about the installation is kept separate from its realisation, and the same model is deployed on ThingsBoard, through its Model Context Protocol server, and on OpenRemote, through its REST API.

The prototype was evaluated with seven participants on two tasks derived from a real building. All of them reached a deployed installation, one only at the time limit, and five of the seven obtained exactly the expected one. The mean System Usability Scale score was 60.7 (median 72.5, 95% CI 40.4–81.0), with four of the seven participants above the benchmark of 68. The results suggest that a language model can make configuration accessible through natural language, provided that what it produces is bounded by a catalogue, by explicit rules and by the confirmation of the architect.

Keywords: Internet of Things; Agentic AI; Large Language Models; Multi-Agent Systems; SAIBA; IoT deployment automation; Intent-based configuration; Model Context Protocol.

1. Introduction

The Internet of Things has turned buildings, plants and infrastructures into sources of continuous measurement, bridging the physical world and cyberspace through sensing, communication and data processing [1]. A modern installation gathers data in real time, feeds it back into control decisions and makes the state of physical assets observable at a fine granularity across large networks [2].

Before any of this can happen, however, the installation has to be brought into existence: the building has to be described, the devices chosen and associated with the spaces they serve, the relations between devices, gateways and rooms declared, and all of it reproduced on the platform that will operate the installation. This step receives far less attention than the technologies that follow it, and it is where the effort of this thesis concentrates.

1.1 Problem statement

Configuring a distributed IoT installation inside a real building is a manual, time-consuming and error-prone activity. The effort grows with the size of the environment, but the real fragility lies elsewhere. The conceptual representation of the building, the physical deployment of the devices and the configuration present on the platform are three descriptions of the same reality, produced separately and kept aligned only by the diligence of the operator. When they drift apart the installation keeps producing data, but that data is no longer reliably interpretable.

Heterogeneity amplifies the difficulty [1]. Devices differ in capabilities, provisioning procedures and required metadata; platforms differ in the abstractions they expose and in the interfaces that manipulate them. Lacking universal standardisation, integration is habitually achieved through custom and brittle glue code [3], so that configuration knowledge stays bound to individual practitioners and individual platforms.

Underneath these obstacles lies a semantic problem. The architect reasons about *what* the installation should achieve: which building, how organised, which scenario, which spaces covered. The platform requires *how* this is realised: entities to be created, device profiles to be instantiated, relations, credentials and telemetry keys to be declared. Conventional programmatic approaches cannot completely bridge the two, because they lack both the reasoning needed to turn an abstract description into a coherent sequence of configuration actions and the ability to ask for what is missing when that description is incomplete, as descriptions in natural language almost always are.

1.2 Research contribution

Agentic Artificial Intelligence has become one of the most prominent ways of applying large language models [4]. Where a language model used on its own answers a single request, an agentic system is given a goal and works towards it: it decides the steps the goal requires, carries them out through the tools and interfaces available to it, and observes the results before choosing what to do next [4, 5]. Systems of this kind are increasingly used to let people interact in natural language with software that would otherwise require specialised knowledge, and to turn what they ask into concrete actions on that software [6]. The IoT domain is no exception: agents based on language models already create smart-home automations through conversation [7] and answer operators' questions about the state of a deployed system [8].

These capabilities match the difficulties of configuring an installation. An agentic system can interpret a description given in everyday terms and turn it into a structured representation, recognise when that description is incomplete and ask for what is missing, and carry out the resulting sequence of actions through the interfaces a platform exposes. Its main risk is just as relevant: a language model produces a plausible answer whether or not it is correct, so in a task whose mistakes end up on a platform, the value of Agentic AI depends on how it is bounded as much as on what it can do [6].

The contribution of this thesis is an agentic system that applies these capabilities to the configuration of IoT installations. It accepts a description of a building in natural language, builds a coherent model of it while asking for the information that is missing, associates devices drawn from a declarative catalogue with the spaces of that model, checks the result against the rules of the selected use case, and creates the corresponding entities on the target platform. A supervising agent delegates each stage to a specialised agent, the output of the language model is constrained to explicit schemas and grounded in the catalogue, and nothing is written to a platform without an explicit confirmation from the architect.

Following the SAIBA framework [9], deciding what to do is kept separate from realising it, so that the same model can be deployed on two different platforms through interchangeable adapters.

1.3 Objectives

The aim of this work is to make the deployment of a complex IoT infrastructure possible without manual, entity-by-entity configuration. Three research questions articulate it:

- **RQ1 (Architectural design).** What hierarchical orchestration architecture can empower IoT architects to translate abstract, natural language requirements into actionable, platform-agnostic deployment strategies?
- **RQ2 (Implementation strategy).** How can an LLM-based reasoning engine be structured within a multi-agent ecosystem to reliably coordinate complex, multi-step deployment workflows?
- **RQ3 (Empirical validation).** To what extent does the proposed agentic loop allow a natural-language description to be translated into an installation that is correctly configured and realised on the target platform, and how do its intended users judge its usability?

1.4 Thesis structure

- **Chapter 2 (Background)** reviews the foundational technologies: the layers of an IoT architecture, the SAIBA framework, the conceptual taxonomy of Agentic AI, and the protocols and frameworks used to build multi-agent systems.
- **Chapter 3 (Literature review)** compares existing approaches against the six characteristics required by this project and identifies the gaps that motivate the proposed solution.
- **Chapter 4 (Research methodology)** describes the design science approach adopted and the process through which the artifact is conceived, built and evaluated.
- **Chapter 5 (Problem awareness)** analyses the configuration activity in a concrete reference scenario and consolidates the practical needs of an IoT architect into the requirements R1–R7.
- **Chapter 6 (Artifact design)** derives the architecture from those requirements, defining the conceptual model of an installation, the device catalogue and the orchestration of the agents.
- **Chapter 7 (Artifact implementation)** presents the technical realisation on a multi-agent framework, including the adapters for the two target platforms, and what the resulting system does.
- **Chapter 8 (Evaluation)** reports the study conducted with representative users, measuring the correctness of what reaches the platform and the usability of the interaction.
- **Chapter 9 (Discussion)** interprets the results with respect to the research questions and examines the limitations of the approach.
- **Chapter 10 (Conclusion)** summarises the findings and outlines directions for future research.

2. Background

2.1 Internet of Things

The Internet of Things (IoT) has evolved from providing purely surveillance-centric functions to acting as a comprehensive information framework for industrial processes[10]. It serves as a bridge between the physical world and cyberspace, enabling fine-grained sensing of objects, continuous data gathering, and real-time feedback or control. Today, the significance of IoT is emphasized by its role in building "Industry 4.0" and the "Industrial Internet" where billions of connected things provide smart services. The following subsections examine the layers of a typical IoT architecture, shown in Figure 2.1.

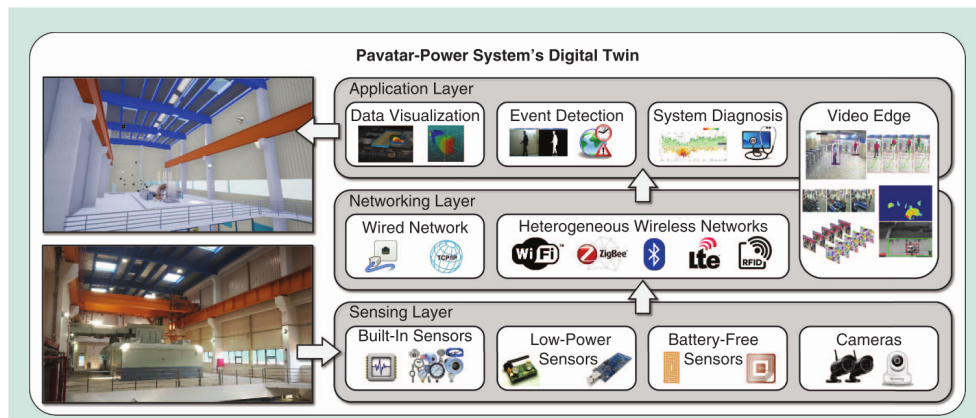


Figure 2.1: Pavatar architecture[10]

2.1.1 Sensing layer and physical metrics

The foundation of modern IoT scenarios is the ability to monitor diverse physical metrics through non-intrusive sensing[10]. In industrial environments, key metrics include:

- **Operational Health:** Metrics such as temperature, pressure, vibration, noise, and rotation speed are critical for monitoring rotating machines
- **Advanced Metrics:** Specialized sensing includes eccentricity (displacement of a rotating center), which is vital for synchronizing compensators in power station management
- **Material and Target Identification:** Techniques like TagScan utilize the differences in Radio Frequency (RF) signals when traversing different liquids or solids to classify materials and image targets.

- **Battery-Free Sensing:** The development of wireless and battery-free sensing, particularly through RFID, allows for low-cost and robust monitoring in complex environments

Furthermore, the sensing layer has expanded to include visual sensing from intensive networked videos. Due to the high bandwidth and low-latency requirements of video data, modern IoT systems increasingly utilize collaborative edge computing to process these intensive streams near the source rather than relying on the cloud. This necessitates the implementation of embedded deep learning models, such as Convolutional Neural Networks (CNNs), directly onto resource-constrained edge devices to provide real-time visual perception and target identification[11].

2.1.2 Networking layer

The networking layer is responsible for the efficient transmission of gathered data between heterogeneous devices, often operating within complex and congested frequency environments[10].

A primary challenge in modern IoT connectivity is the coexistence of incompatible protocols, such as Wi-Fi, ZigBee, and Bluetooth, which frequently occupy the same spectral bands. To address this, current research focuses on Cross-Technology Communication (**CTC**), which enables direct interaction between these disparate devices by utilizing side-channel information like transmission timing, beacon shifting, and RSSI amplitude without requiring hardware modifications.

Furthermore, the standardization of technologies such as Narrow-Band IoT (NB-IoT) has become a central strategy for establishing low-power, wide-area networks (LPWAN) capable of supporting massive device density. These advancements are complemented by resource-saving innovations, including parallel backscatter transmissions and software-defined wireless protocols, which significantly reduce energy consumption and facilitate the deployment of battery-free sensing nodes in large-scale scenarios.[12]

2.1.3 Data analytics and processing

The analytics layer transforms **raw signals into actionable knowledge** through sophisticated signal processing and machine learning techniques.[10] This stage is critical because the massive volume of data generated by billions of devices is often redundant, noisy, or incomplete, requiring advanced “truth-finding” algorithms to resolve conflicts among multisource data and ensure the reliability of the system’s state estimation.

Beyond basic aggregation, modern IoT analytics focuses on knowledge discovery and generation, where the system identifies patterns and predicts future behaviors rather than merely reporting historical data. This involves complex procedures such as **time-series data cleaning**, which transitions from simple anomaly detection to sophisticated anomaly repairing to maintain the continuity of the digital thread.

Furthermore, optimized sensor configuration and feature selection are employed for **advanced fault detection and diagnosis**, ensuring that the most relevant metrics are prioritized for processing. These analytical capabilities allow the system to handle the emergent behaviors of complex environments, effectively bridging the gap between raw physical sensing and the high-level intelligence required for autonomous control and decision-making.

2.1.4 IoT platforms

The layers described so far produce data; an IoT platform is the software that holds the installation that produces it. It keeps the registry of the devices and their credentials, represents the structure of the environment they are deployed in, ingests and stores the telemetry they send, and exposes both to dashboards, rule engines and external applications through an API. Two of its functions matter for this work: **provisioning**, the creation of the entities that represent the installation, and the **entity model**, the abstractions in terms of which those entities must be expressed.

Provisioning is rarely the part a platform is chosen for, and it is where the manual effort accumulates. Every space, every device and every containment relation has to be created, typed and named, and the metadata each device needs in order to be commissioned has to be attached to it. Platforms differ substantially in how they ask for this, which is what makes the activity difficult to automate once and for all.

ThingsBoard

ThingsBoard organises an installation into typed entities, of which devices and assets are the two that describe physical reality: a device is a source of telemetry and carries its own access credentials, an asset is anything else worth representing, such as a building, a floor or a room. Structure is expressed through *relations*, which are first-class objects connecting two entities with a direction and a type, so that a room containing a sensor is stated as a relation of type `Contains` between the two. Additional information is attached as attributes, which are further separated into scopes according to whether they are set by the server or shared with the device. Provisioning is available through a REST API and, more recently, through an official Model Context Protocol server that exposes the same operations as tools for agentic clients [13].

OpenRemote

OpenRemote makes the opposite choice: there is a single kind of entity, the asset, and what a thing is depends on its asset type, drawn from a hierarchy that includes buildings, rooms and generic things. Attributes belong to the asset without any notion of scope. Structure is not a separate entity at all: containment is a property of the child, which stores the identifier of its parent, so a hierarchy is expressed by setting each element's parent rather than by creating links between elements. Installations are partitioned into realms, and programmatic access is granted to service users authenticated through OAuth client credentials against the identity provider bundled with the platform [14].

What differs, and why it matters

The same installation therefore takes two different shapes. What is a device in one platform is an asset of a device type in the other; what is a relation object in one is a field of the child in the other; what is a scoped attribute in one is a plain attribute in the other; and the vocabularies of type names have nothing in common. None of these differences is conceptual: the building, its rooms and its sensors are the same in both cases. They are differences in how a platform wants to be told. Figure 2.2 shows the same installation as held by each platform. ThingsBoard distinguishes devices from assets and records containment as relation entities directed from parent to child;

OpenRemote holds everything as a typed asset and records containment as a reference from each child to its parent.

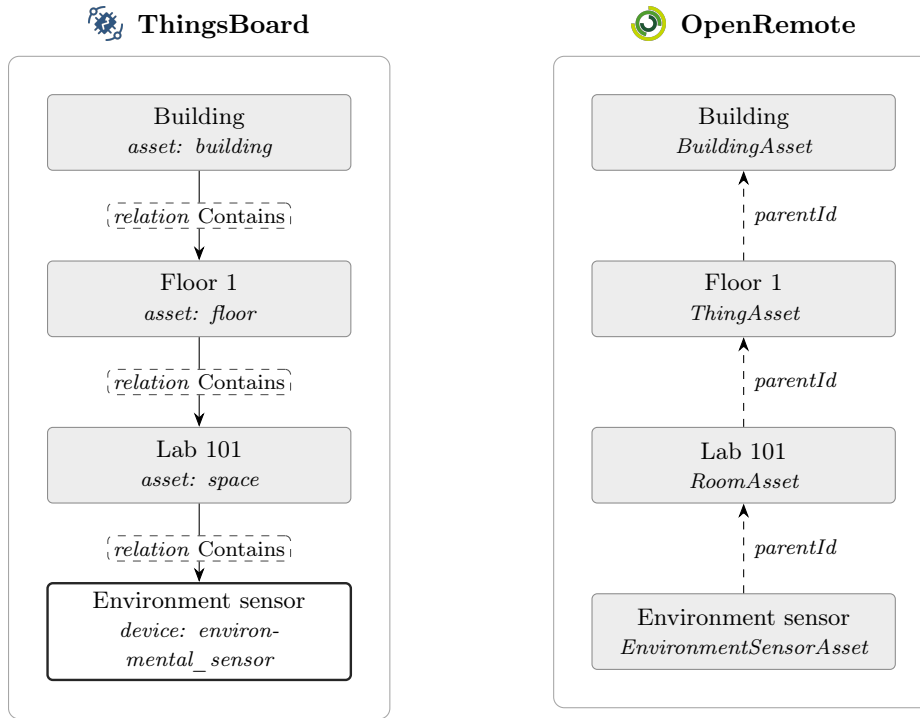


Figure 2.2: The same installation as held by ThingsBoard and by OpenRemote.

This is precisely the kind of divergence that custom integration code absorbs badly, because the knowledge of one platform ends up spread across whatever produced the configuration. A system meant to deploy the same description onto more than one platform must therefore decide where that knowledge is allowed to live, which is the design problem addressed in Chapter 6 and the reason a second platform is used in Chapter 7 to test the boundary.

2.2 Building modeling

Before an installation can be created on a platform it has to be described somewhere, and the question of which vocabulary to describe it in has been addressed by a body of work on building and device semantics. Three efforts are representative, and they divide the problem differently.

The Building Topology Ontology (BOT) is a deliberately minimal vocabulary for the spatial structure of a construction: sites, buildings, storeys, spaces, the elements they contain and the containment and adjacency relations among them [15]. It describes where things are, and nothing else, leaving what they do to other vocabularies extending it. SAREF, the Smart Applications REference ontology, covers the complementary half: devices, the functions they offer, the commands they accept and the properties they measure, with extensions specialised by application domain [16]. Brick is more ambitious than either, providing a richer schema for the equipment, subsystems and data points of a smart building, designed so that applications written against one building can be reused on another [17].

What the three share is the goal of semantic interoperability: a description expressed in them is machine-interpretable by parties that never agreed with one another, and can be queried and reasoned over as a graph. What they also share is a cost. They presuppose an RDF representation and a query language, and a description is valid only if it respects the vocabulary in full, which is a demanding target for a component that generates the description automatically.

2.3 SAIBA framework

The SAIBA framework (representing Situation, Agent, Intent, Behavior, and Animation) is a reference framework proposed by the embodied conversational agent community to unify the generation of behaviours[9]. It proposes a modular, three-stage model that abstracts the "intent" of an agent from its physical "realization," providing a structured approach to building sophisticated autonomous systems [18].

2.3.1 Stages of the SAIBA architecture

The SAIBA framework is defined by three distinct processing stages[9] that manage the transition from high-level reasoning to physical action:

- **Intent Planning:** This first stage determines the goals, beliefs, and intentions of the agent based on its current situation. The output is a functional description of the desired state, expressed through the Function Markup Language (FML), which describes what the system should achieve without specifying how it should physically behave.
- **Behavior Planning:** This stage takes the functional intent (FML) and translates it into a detailed plan of physical actions. This plan is represented using the Behavior Markup Language (BML), a standard XML-based language that describes the multimodal physical realizations required to fulfill the intent.
- **Behavior Realization:** The final stage involves the actual execution of the planned behaviors through a realization engine, which drives the physical or virtual components of the system.

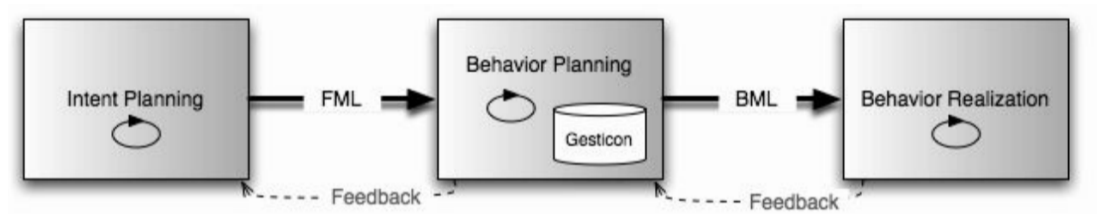


Figure 2.3: SAIBA architecture[9]

2.3.2 From conversational agents to IoT configuration

SAIBA was designed for a different domain, and this work does not apply it: it borrows its central principle, the separation of what is meant from how it is realised, mediated by explicit representations. The configuration of an IoT installation has the same structure,

and the correspondence holds at the level of the representations rather than of the stages alone:

- the role of FML is played by a *configuration model* of the installation, which states which building, spaces and devices must exist without saying how any platform represents them;
- the role of BML is played by an *ordered set of provisioning operations* derived from that model, which is already specific to a target platform but has not yet been executed;
- the role of the realiser is played by *platform adapters*, which carry those operations out against the interfaces of a particular platform.

The same property that motivated SAIBA follows. Reasoning about an installation stays independent of the platform that will host it, and supporting a further platform means providing another realiser rather than revising the planning above it. The feedback between stages also has a direct counterpart: an operation the platform refuses is reported back with its reason, so that what already succeeded is kept and only what failed needs to be planned again. Chapter 6 places the components of the artifact onto these stages (Table 6.3).

The analogy has limits, and they are worth stating. SAIBA is built around real-time generation, in which behaviours must be synchronised to the syllable; nothing in the configuration of an installation is time-critical in that sense. Nor does the configuration model use FML, or the operations BML: the borrowing concerns the architecture of the separation and the discipline of the interfaces, not the languages themselves.

2.4 Agentic AI

A language model on its own produces a response to a request. An agentic system uses such a model to pursue a goal: it decomposes the goal into steps, chooses and invokes the tools each step requires, observes the results and decides what to do next, over as many iterations as the goal demands [4]. The literature has grown quickly around this idea, and with it a vocabulary that is not always used consistently, so it is worth fixing the distinctions this thesis relies on before describing how such systems are built.

2.4.1 Conceptual taxonomy and differences

Three terms are commonly distinguished [6]. *Generative AI* denotes a model that produces content in response to a prompt, in a single pass and without state beyond the conversation it is given. An *AI agent* augments such a model with tools and a reasoning loop, so that it can carry out a well-defined task on its own once started, such as querying a database or following a fixed procedure. *Agentic AI* denotes systems in which several agents, each specialised, are coordinated towards a broader goal, with a shared and persistent state that keeps the work coherent across steps [5]. Table 2.1 summarises the differences. Figure 2.4 illustrates the difference with an example from home automation. A smart thermostat that holds the temperature an occupant has set acts as an AI agent: it carries out one task, on its own, within fixed bounds. A system in which separate agents track weather forecasts, energy prices, the occupants' schedule and the state of the appliances, and are coordinated to manage the home as a whole,

is agentic. The example concerns the operation of an installation, whereas this thesis addresses its configuration; what carries over is the distinction between carrying out a single task and coordinating specialised agents towards a goal.

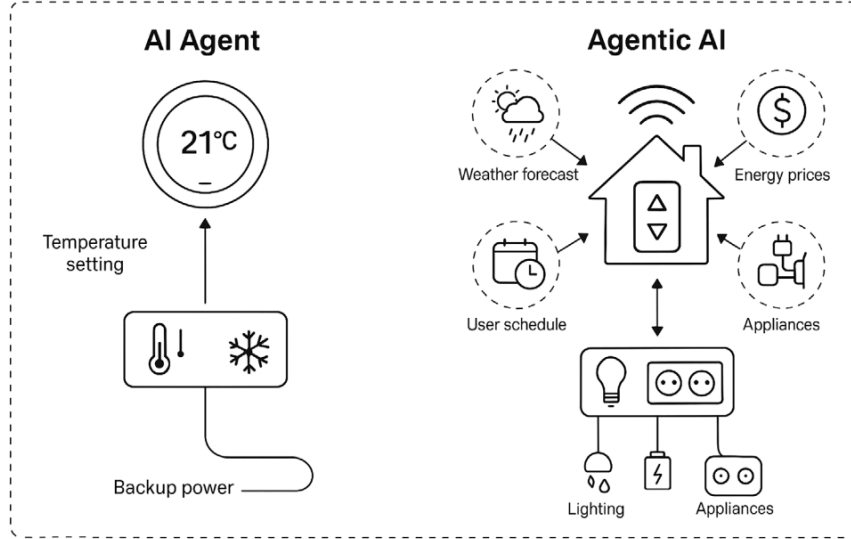


Figure 2.4: An AI agent and an agentic system in a smart home. Reproduced from [6], licensed under CC BY 4.0.

Feature	Generative AI	AI agent	Agentic AI
Scope	A single response	A well-defined task	A goal spanning several tasks
Architecture	One model, stateless	Model with tools and a reasoning loop	Coordinated, specialised agents
Control	Single-pass generation	Iterative choice of the next action	Decomposition of the goal and delegation of its parts
Memory	The context window	The context of the current task	Persistent state shared across steps and sessions

Table 2.1: Generative AI, AI agents and Agentic AI, adapted from [6].

The distinction concerns structure, and it is easy to conflate with a second one that concerns autonomy. Definitions of Agentic AI often emphasise operation with minimal human intervention [4], but how much a system is allowed to decide on its own is a design parameter rather than a defining property. It is usefully described as a spectrum. At one end a person approves every consequential action before it takes place, which is usually called *human-in-the-loop*; in the middle the system acts on its own while a person monitors it and can intervene, *human-on-the-loop*; at the other end the system acts without oversight. Where a system should sit depends on what its actions cost when they are wrong, and in particular on whether they can be undone.

The artifact developed in this thesis is agentic in the structural sense: a supervisor

decomposes the activity and delegates it to specialised agents operating on a shared, persistent state. Its autonomy, however, is deliberately bounded. It asks for missing information rather than inferring it, and it writes nothing to a platform without an explicit confirmation, because provisioning operations are not reversible in general.

2.4.2 Core components

Agentic systems are usually described through a cycle of perception, reasoning and action, in which the agent gathers information about its situation, decides what to do and acts on its environment, repeating the cycle until the goal is reached [4]. Around this cycle, the systems that coordinate several agents add functions that let the work persist and remain coherent over time [6]:

- **Perception Layer:** Utilizes sensors and digital interfaces to collect data (text, images, or real-world signals) from the environment [5]. In modern systems, Large Image Models (LIMs) and multi-modal LLMs serve as the primary perceptual engines [6]
- **Cognitive Reasoning and Planning:** The "brain" of the agent, typically powered by an LLM, which performs task decomposition, evaluates potential strategies, and selects the most optimal course of action [6] [5]
- **Action Execution and Actuation:** Translates cognitive decisions into real-world interactions through an action library, often involving external tool use via APIs [6]
- **Memory Architectures**[6]:
 - **Episodic Memory:** Records past interactions and short-term task history
 - **Semantic Memory:** Stores long-term facts and structured domain knowledge
 - **Vector Memory:** Enables similarity-based retrieval (RAG) to ground responses in factual data
- **Reflection and Evaluation:** Verify immediate or final outputs, critique candidates plans, and trigger selective replanning.
- **Orchestration:** Interaction modules supports human-agent and agent-agent dialogue (clarification, negotiation ...) and surface trace information (actions, tools, sources...) for transparency. Autonomy emerges when perception, planning execution, memory and reflection are orchestrated over time; in many systems, an LLM-based supervisor coordinates sub-agents, invokes memory or tools, and maintains coherence across steps.

2.4.3 Types of architectures

Agentic architectures can be told apart by two choices: how many agents share the work, and whether the plan is formed before acting or while acting. The patterns below are presented along these two axes, from a single agent that decides one step at a time to several agents coordinated by a supervisor. In this section explain the principal types with the relative structure based on the components already explained in the last section.

ReAct Single-Agent

The ReAct (Reasoning and Acting) architecture is an iterative framework designed to overcome the limitations of single-pass inference. It synergizes reasoning traces with task-specific actions to allow an agent to process information more deliberately.

The agent operates in a loop where it first generates a "thought" (reasoning trace), executes an "action" (calling a tool or API), and then "observes" the result from the environment before updating its next thought.

This loop allows the agent to verify retrieved data before summarizing it, which significantly reduces hallucinations and improves logical consistency in multi-step workflows. It effectively bridges the gap between internal reasoning (Chain-of-Thought) and external environment interaction.

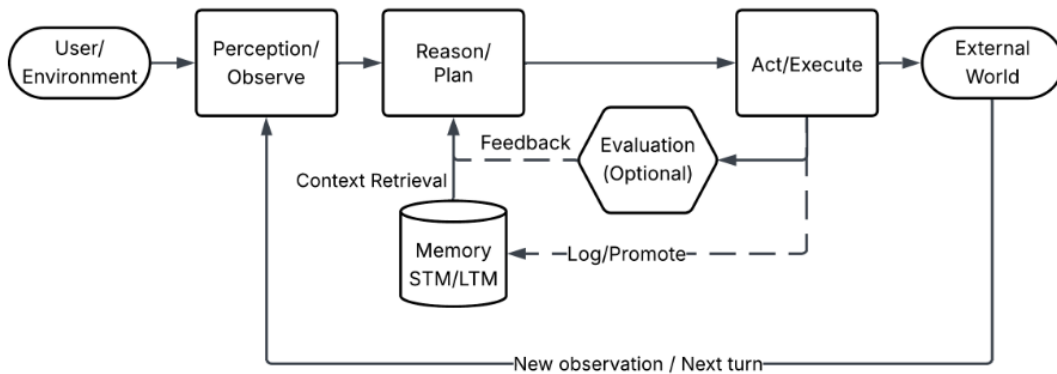


Figure 2.5: ReAct Single-Agent architecture[5]

Plan-and-execute

A second pattern separates planning from execution. A planner first produces the complete sequence of steps, together with the tools each requires, and an executor then carries them out, returning to the planner only when a step fails or the plan has to change [19]. Deciding the plan once avoids invoking the model after every observation, which reduces cost and latency, and it makes the intended course of action visible before any of it takes place. The price is adaptability: a plan fixed in advance cannot react to an unexpected result unless replanning is explicitly triggered.

Supervisor/Hierarchical

The supervisor, or hierarchical, architecture moves beyond the isolated execution of a single agent to a managed multi-agent ecosystem. It introduces a meta-agent that coordinates the behaviour of subordinate specialised agents, each with its own instructions, its own tools and its own context (Figure 2.6). The supervisor parses a high-level goal into smaller, manageable sub-tasks and distributes them across these agents: it manages the dependencies between sub-tasks, assigns each of them to the agent whose specialisation fits it, such as a *Coder* agent for writing a program and a *Reviewer* agent for checking it, and resolves conflicts between the outputs it receives before reconciling them into a single result. HuggingGPT is an early example, in which a language model

acting as controller decomposes a request and delegates each sub-task to a specialised model [20]. Supervisors can in turn be nested, so that an agent coordinated by one supervisor is itself the supervisor of a team, giving hierarchies of arbitrary depth.

The benefits follow from the division itself. Each agent reasons over a smaller context and chooses among fewer tools, its instructions can be specialised to its competence, and it can be developed and tested on its own; delegation also makes the course of the work observable, since every hand-off is an explicit event. The costs are equally structural. Every delegation adds model invocations, and so latency and cost; the supervisor can route a sub-task to the wrong agent; information can be lost when work passes from one agent to another; and a faulty output from one agent can propagate through the others, compounding errors in the decisions that follow and producing coordination breakdowns that no single agent would exhibit [6].

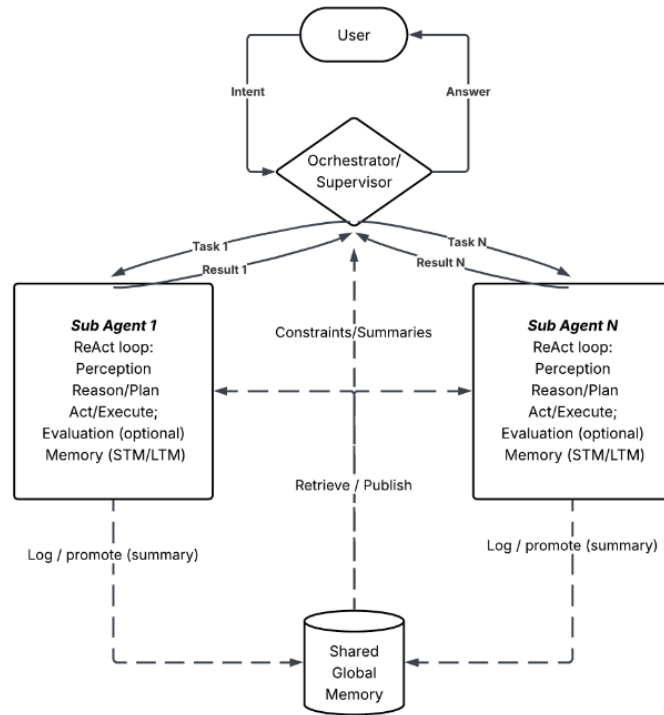


Figure 2.6: Supervisor/hierarchical architecture[5]

Parallelisation and aggregation

Where a supervisor decides which agent handles each part of a task, an aggregator decides how the results of several agents become one. In its most common form the work is fanned out to agents that run independently, often in parallel, and their outputs are collected by an aggregator that combines them into a single result. The division can take two forms [21]: the task may be split into independent sub-tasks, each handled by a different agent, or the same task may be given to several agents so that their answers can be compared, for instance by voting, to obtain a more reliable result. Aggregation need not be mechanical. In the **Mixture-of-Agents** approach, several models act as proposers of candidate responses and a further model acts as aggregator, synthesising them into a single output of higher quality than the individual proposals [22]. Orches-

tration frameworks support the pattern directly, typically by dispatching the same state to several nodes and merging their outputs into a shared field read by an aggregator node [23].

The pattern pays off when the parts are genuinely independent, since they can then run concurrently, and when several perspectives increase confidence in the result. Its weaknesses mirror its strengths. Parts that depend on one another cannot be separated without losing that dependency, and an aggregator that synthesises rather than merely collects can introduce into the combined result something that none of its inputs contained.

Other families

Agent architectures predate language models. The Belief–Desire–Intention (BDI) model, rooted in theories of human practical reasoning, describes an agent through its beliefs about the world, the goals it could pursue and the subset it commits to, revised in a continuous deliberation cycle [24]. Hybrid reactive–deliberative architectures run a fast loop for time-critical responses alongside a slower loop that maintains goals and plans, with an arbitration mechanism between the two, and suit agents that must act continuously in real time. Layered neuro-symbolic architectures combine neural components for perception with a symbolic planner, trading some flexibility for decisions that can be traced and explained. These families remain influential, but the systems considered in this thesis are built from the patterns above.

2.4.4 Keeping agents within bounds

The capabilities described so far are also what makes an agentic system unreliable. A language model asked to produce a configuration will produce something plausible whether or not it has the information needed to produce something correct, and an agent able to invoke tools can act on such an answer before anyone has examined it. Three families of mechanisms bound this behaviour, and they act on different things: the form of what the model produces, its content, and what the system is allowed to do with it.

Constrained generation Rather than accepting free text and parsing it afterwards, the caller declares the shape of the answer as a schema, and the model is required to emit an instance of it. This is the mechanism behind function calling, where the parameters of a tool are described in advance and the model fills them in, and in its strictest form the decoding itself is restricted, so that at each step only the tokens compatible with the schema or grammar can be generated [25]. The gain is not stylistic. An answer bound to a schema has fields whose presence, type and admissible values can be checked mechanically, so a malformed or impossible answer fails at the boundary instead of propagating into the system as a plausible-looking one. When a field admits only an enumerated set of values, the model cannot select an option that does not exist, and each decision it is asked to take becomes narrower and easier to keep correct.

Grounding Grounding supplies the facts an answer must be built from, instead of relying on what the model retained from its training. Retrieval-augmented generation is its familiar form, in which relevant documents are retrieved and placed in the context of the model before it answers [26], but the principle extends to any external source of

truth, including a catalogue, a schema or the current state of the system being acted upon. Grounding decides the content of an answer, constraining decides its form, and neither replaces the other: a system that applies only the first can produce a well-formed statement about equipment that does not exist, and one that applies only the second can produce a true statement in a shape nothing downstream can consume.

Guardrails A guardrail is a control placed around a model rather than inside it: code that determines what may reach the model, what may leave it and which actions may follow, enforced independently of what the model itself would do. The distinction from prompting is essential. An instruction in a prompt asks the model to behave in a certain way; a guardrail makes the unwanted behaviour impossible, or at least detectable, whatever the model produces. It is equally distinct from the alignment a model acquires in training, which resides in its weights and cannot be inspected or adjusted by the application that uses it. Guardrails are commonly grouped by what they control:

- **Input guardrails** examine what reaches the model, for instance to detect attempts to override its instructions or requests outside the scope of the application.
- **Output guardrails** examine what the model returns before it is used, checking it against a schema, against the facts it should be grounded in, or against content policies; constrained generation and grounding can be seen as particular cases of this kind.
- **Action guardrails** restrict what the system can do: which tools exist at all, which arguments they accept and with which permissions they run.
- **Human approval** makes a consequential action wait for an explicit decision by a person, and is the strongest guardrail available for actions that cannot be undone.

Dedicated toolkits allow such controls to be declared as programmable rails that sit between the application and the model [27], and several agent frameworks expose guardrails as a native concept. Whatever their form, a principle recurs: guardrails should *fail closed*, so that when a check cannot establish that an action is allowed, the action does not take place.

Taken together, these mechanisms change what a language model is within an architecture. It stops being the component that decides what is true and what is done, and becomes the component that translates between an informal request and structures defined and enforced elsewhere, which is what makes its output something that can be validated deterministically rather than merely read.

2.5 Agent interoperability protocols

The advancement of Multi-Agent Systems (MAS) has transitioned from isolated Large Language Models (LLMs) to complex ecosystems where autonomous agents must interact with diverse tools, data sources, and other agents. Traditionally, these interactions were managed through "glue-code": custom, brittle integrations that are difficult to scale and secure [3]. To move toward a more scalable "Agent Economy," several standardized protocols have been proposed to handle different tiers of interaction: vertical (agent-to-infrastructure) and horizontal (agent-to-agent) [3] [28].

2.5.1 Model Context Protocol (MCP)

The Model Context Protocol (MCP), introduced by Anthropic, is a vertical integration standard designed to provide a universal interface between AI models and data sources [3]. Often described as the "USB-C for AI," MCP replaces the need for bespoke connectors by using a model-agnostic JSON-RPC client-server architecture [29][28].

This protocol allows applications to expose "resources" (static or dynamic data), "tools" (executable functions), and "prompts" (predefined templates) to an LLM [29]. By standardizing these interfaces, MCP enables agents to interact with complex environments, such as IoT data spaces and industrial context brokers, without requiring the developer to write custom code for every new integration.

Architectural Model

MCP employs a Client-Server architecture where the components are decoupled to ensure security and modularity[3]:

- **MCP Host:** The primary application (an IDE, a desktop assistant, or a custom LLM app...) that orchestrates the overall workflow and manages the user's session
- **MCP Client:** A component within the host that handles the protocol-specific logic, establishing and maintaining connections to one or more servers
- **MCP Server:** A lightweight provider that exposes specific datasets or functionalities to the model. A single host can connect to multiple servers simultaneously to aggregate different capabilities

Core Components

The protocol distinguishes the features a server offers from those a client may offer in return [30]. Servers expose three primitives:

- **Resources:** read-only data sources, such as file contents, database records or API logs, provided as context to the model
- **Tools:** executable functions that allow the model to take actions in the external world, defined with schemas (for example JSON Schema) so that the model knows how to parameterise each call
- **Prompts:** predefined interaction templates provided by the server to help the model perform specific tasks

Clients, in turn, may offer features that servers invoke. The most notable is **Sampling**, which lets a server request a completion from the model of the host, so that the server can use the model's reasoning for its own logic; **Roots** tell a server the boundaries it may operate within, and **Elicitation** lets a server ask the user for additional information during an interaction.

2.5.2 Agent To Agent(A2A)

While MCP handles how an agent accesses tools, Google's Agent-to-Agent (A2A) protocol focuses on horizontal communication, enabling agents from different vendors and

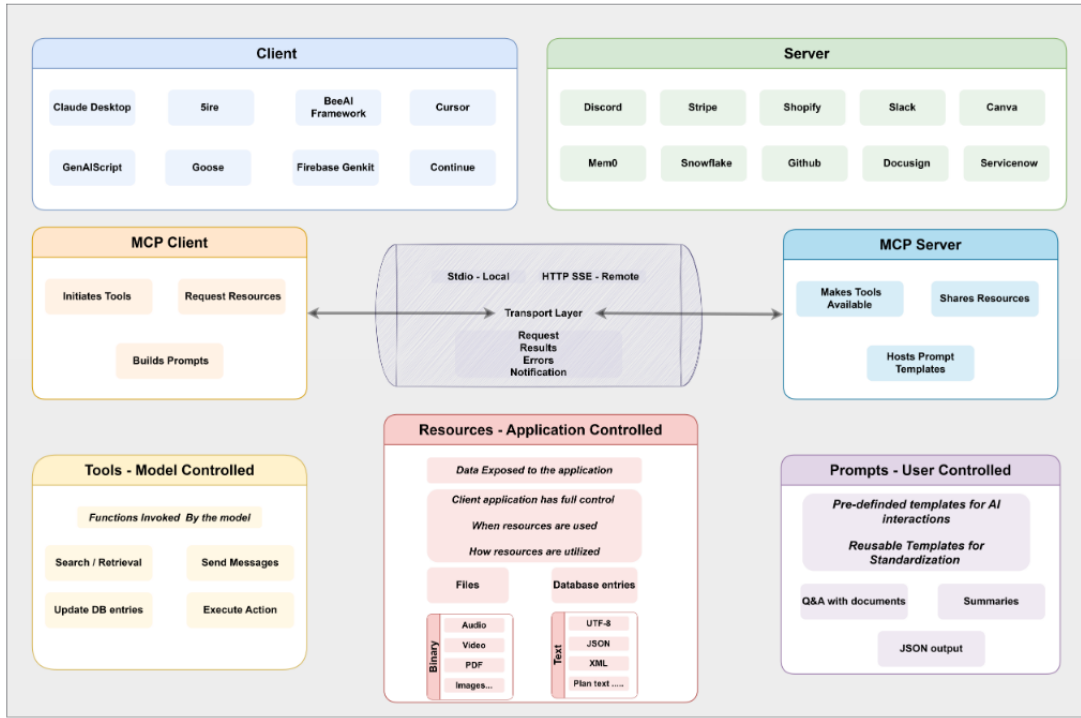


Figure 2.7: MCP architecture[28]

frameworks to collaborate [28]. The A2A framework utilizes “Agent Cards”(standardized descriptions of an agent’s capabilities and security requirements)to facilitate discovery and negotiation between autonomous entities. This protocol primarily utilizes HTTP and Server-Sent Events (SSE) to manage asynchronous task delegation and multimodal information exchange [3].

Architectural model

A2A utilizes a Peer-like relationship between two distinct agent roles:

- **Client Agent:** The agent that represents the user’s initial intent. It is responsible for planning the workflow and delegating specialized sub-tasks to other agents
- **Remote Agent:** A specialist agent that receives task delegations. It exposes its capabilities through standardized discovery mechanisms and executes the work on behalf of the Client Agent

Core components

The protocol is built around several key structural elements:

- **Agent Card:** A structured JSON metadata document that serves as the agent’s “business card.” it declares the agent’s identity, supported skills, input/output formats, and authentication requirements
- **Tasks:** The fundamental unit of work in A2A. A task encapsulates the request parameters and has a unique lifecycle, moving from initiation to execution and finally to completion

- **Artifacts:** The formal outputs produced by the Remote Agent. Artifacts can include structured data, text, or file references that are returned to the Client Agent
- **Communication Layer:** A2A utilizes HTTP POST for initiating requests and Server-Sent Events (SSE) for streaming progress updates and intermediate status reports, enabling long-running, asynchronous collaborations

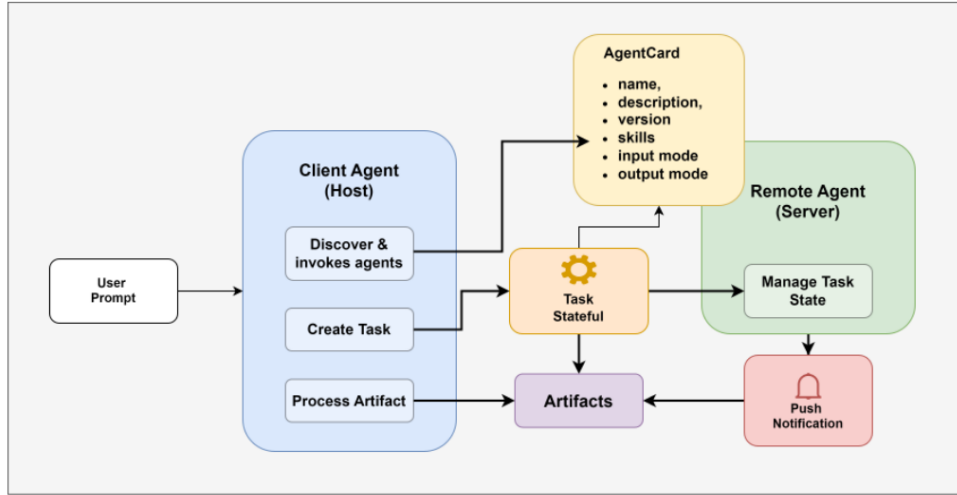


Figure 2.8: A2A architecture[28]

2.5.3 Agent Communication Protocol (ACP)

The Agent Communication Protocol (ACP) , introduced within the BeeAI open-source project, is a modular, HTTP-based system designed to facilitate interaction between clients and agents. It is composed of three primary components: the ACP Client, the ACP Server, and one or more ACP Agents[28].

The **ACP Client** initiates communication by submitting requests in ACP-compliant format.

The **ACP Server** acts as a middleware component, translating external HTTP requests into internal agent executions. It manages agent lifecycle phases including execution and optional session persistence.

The **ACP Agent** is the execution component defined using a decorator-based configuration.

Central to this architecture is the **Agent Detail**, a self-descriptive JSON or YAML document that serves as the agent’s public identity and capability profile. Complementing this, **Discovery Mechanisms** allow clients to locate agents dynamically at runtime. Once an agent is located, clients issue a **Task Request**, a structured unit of delegated work. All requests and responses conform to ACP’s **Message Structure**, which standardizes the communication envelope. Finally, the result of agent execution is encapsulated in one or more **Artifacts**. In August 2025 ACP was merged into A2A under the governance of the Linux Foundation [31], so that its concepts now continue within that protocol rather than as a separate standard.

2.5.4 Agent Network Protocol (ANP)

The Agent Network Protocol (ANP) represents a decentralized approach to agent interoperability, focused on creating an open, global “Agent Economy” [3]. Unlike the client-server models of MCP or ACP, ANP adopts a pure peer-to-peer (P2P) architecture that eliminates the need for centralized intermediaries or platform-specific gateways [28].

The protocol relies on W3C Decentralized Identifiers (DIDs) for secure, self-sovereign authentication and JSON-LD graphs for discovery. This allows agents to autonomously find and negotiate services with one another over the open internet, using verifiable credentials to establish trust dynamically. ANP is designed for scenarios where agents from disparate organizations must collaborate without prior manual configuration or a shared governing entity.

2.5.5 Structural patterns and combinations

The effectiveness of these protocols is maximized when they are applied in specific structural combinations, creating a tiered hierarchy of agent capabilities[3]

MCP and A2A synergy

The most common structural pattern is the A2A-MCP Hybrid, which separates high-level orchestration from low-level execution:

- **Orchestration Layer (Horizontal/A2A):** A Client Agent (the Manager) uses A2A to discover a Remote Agent (the Specialist). The Manager delegates a high-level task through an A2A request.
- **Execution Layer (Vertical/MCP):** The Specialist Agent, upon receiving the task, acts as an MCP Host. It invokes its internal MCP Servers to access the specific local data or tools required to solve the sub-task.
- **Result Aggregation:** The results from the MCP tool calls are synthesized by the Specialist and returned to the Manager as structured A2A Artifacts

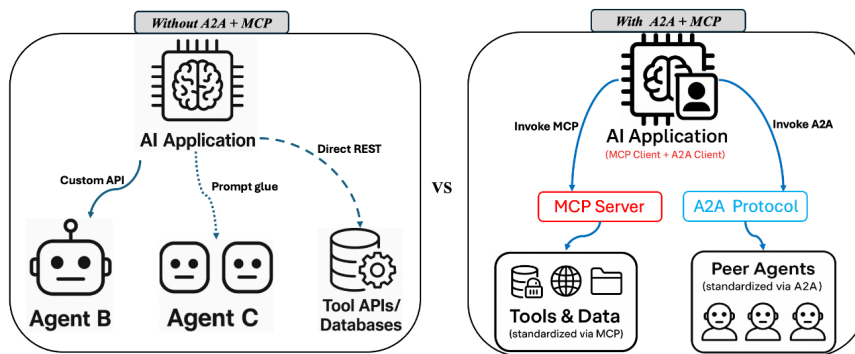


Figure 2.9: MCP+A2A integration[3]

This pattern can be implemented in 3 different ways:

1. **A2A Agent Utilizing MCP Internally (Primary Pattern):** An A2A server agent uses MCP internally.
2. **Exposing MCP Tools via A2A Agent Card:** A2A skills directly represent MCP tools.
3. **A2A for Tool Orchestration (Alternative/Edge Case):** Using A2A directly for complex "tools".

Protocol Evolution and Maturity Pattern

Knowing all the protocols it's important to question their application in their proper context. Research suggests a phased structural roadmap for organizational adoption, moving from internal tool standardization to external network participation[28]:

- **Phase I (Resource Access):** Implementation of MCP to standardize how internal models interact with legacy databases and proprietary APIs. **Phase II (Service Interaction):** structured, multimodal communication between internal agent services, originally the scope of ACP and now covered by A2A after the merger of the two protocols.
- **Phase III (Inter-Agent Collaboration):** Integration of A2A to allow agents to discover one another and delegate tasks across departmental or organizational boundaries.
- **Phase IV (Global Economy):** Transition to ANP to permit these agents to participate in decentralized, internet-scale marketplaces.

2.6 Agentic AI frameworks

While interoperability protocols define the external communication standards and boundaries between independent systems, agentic AI frameworks provide the internal execution runtimes, programmatic abstractions and state management mechanisms required to construct the agents themselves. In a distributed multi-agent deployment, these frameworks implement the internal reasoning loops and application logic of an agent, which can then be exposed to other systems through protocols such as MCP or A2A. The frameworks below differ above all in three respects, which also organise the comparison at the end of the section: how the flow of control is expressed, where the state of a run lives, and how a person can intervene while it is running.

2.6.1 LangGraph

LangGraph is a low-level orchestration framework for building long-running, stateful agents [23]. Unlike linear pipelines, it represents an agentic application as a state machine expressed as a directed graph, cyclic where needed. **Nodes** are the units of work: a reasoning step, a tool invocation, a sub-agent. **Edges** define the routing between them, either statically or through conditional functions that inspect the current state and decide where execution goes next. A graph is defined, compiled and then invoked, and each invocation walks it from an entry point until it reaches a terminal node.

The shared state. What distinguishes the model is that the graph carries an explicit, typed state object, passed from node to node, which every node reads and may update. A node does not return a message to the next node; it returns a partial update to the shared state, and the framework merges it. How the merge happens is declared per field through reducers: a field may be replaced by the newest value, or accumulated, which is how conversation histories and activity traces grow instead of overwriting themselves. The components of the application therefore communicate through a single object with a declared schema rather than through the arguments of one another's calls, so what the system knows at any moment is inspectable in one place.

Persistence and threads. A compiled graph can be given a checkpoint, which saves the state as execution progresses to a durable store. Each execution belongs to a thread, identified by a thread identifier supplied by the caller, and the checkpoints of a thread form its history. This turns a run from a transient call into something that can be left and resumed: invoking the graph again with the same thread identifier continues from the last checkpoint, in a later process or a later session, and a run that fails can resume from where it stopped rather than from the start, a property the framework calls durable execution. Checkpointers provide memory within a thread; for knowledge that must outlive a single thread, such as facts shared across conversations, the framework offers separate stores. The history of checkpoints also allows earlier states to be inspected, which makes the execution of a long workflow auditable after the fact rather than only while it runs.

Interruption and human-in-the-loop. Persistence is also what makes human intervention structural rather than improvised. A node may call an interrupt, which suspends the graph at that point and surfaces a payload to the caller; the state is saved as it stands, and the run ends. Nothing is held open and nothing waits: the application may terminate entirely. When the person answers, the graph is resumed on the same thread with a command carrying the reply, which becomes the value returned by the interrupt inside the node. This is the mechanism behind approval gates, disambiguation questions and any step that must not proceed without a decision that only a human can take.

Two properties of this mechanism shape how a node that interrupts must be written. Resumption re-executes the interrupted node from its beginning, not from the line that suspended it, so everything preceding the interrupt runs a second time; side effects placed there should therefore be idempotent, and work already accomplished should be recognisable as such [23]. And because execution stops at the interrupt rather than blocking on it, whatever follows the interrupt in the node has certainly not happened yet when the person is asked: the suspension is the guarantee that the decision precedes its effects.

Control flow and multi-agent structures. Because routing is explicit, hierarchical and multi-agent patterns are expressed directly in the graph. A supervisor is a node whose output names the next node or nodes, so delegation becomes a routing decision over a declared set of destinations instead of an emergent behaviour of a prompt. Specialised agents are ordinary nodes, and a whole graph can be embedded as a node of another one, which allows a subsystem to be developed and tested on its own and then composed. Cycles are permitted, so revision loops, retries and iterative refinement are

ordinary structures rather than exceptions, and the compiled graph exposes its execution as a stream of events, which allows a caller to show what is happening while it happens.

Interoperability. Nodes are arbitrary code, so a node can equally well invoke tools exposed by a remote server through the Model Context Protocol described in Section 2.5.1. Adapters maintained within the LangChain ecosystem convert MCP tools into tools that LangGraph agents can call [32], letting the graph orchestrate capabilities that live outside the process while keeping routing, state and checkpointing local.

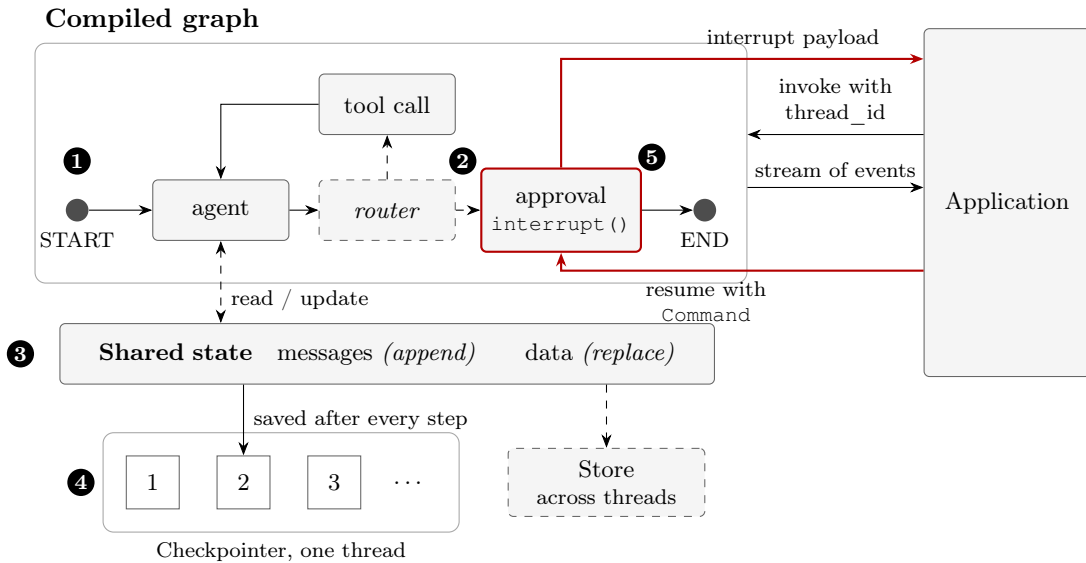


Figure 2.10: How LangGraph executes an application

2.6.2 LangChain

LangChain is an open-source framework for building agents and applications powered by large language models [33]. Rather than forcing developers to write custom wrappers for model APIs and prompt formatting, it provides a standardised interface to the primitives such applications are made of, and a broad set of integrations with model providers, tools and data sources.

The framework structures application logic around interoperable building blocks, including unified model interfaces, prompt templates, structured output parsers and tool definitions. Its original defining concept is the **chain**, which links components sequentially so that the output of one step becomes the input of the next, and it also offers higher-level agent abstractions for common patterns. For fine-grained orchestration, with cycles, persistent state and interruption, the LangChain ecosystem relies on LangGraph, so the two are commonly used together: LangChain for access to models and tools, LangGraph for the control flow around them.

2.6.3 LlamaIndex

LlamaIndex is a data-centric framework focused on connecting language models with private, heterogeneous data [34]. It serves as a data ingestion, indexing and retrieval

layer designed to support retrieval-augmented generation and data-intensive agent applications.

Its architecture is organised around three subsystems: **data connectors**, **index structures** and **query engines**. Data connectors ingest raw data from disparate sources, such as local documents, cloud APIs and relational databases, and parse them into standard document objects. The indexing system transforms these documents into structures suited to retrieval, including vector stores, hierarchical indices and knowledge graphs, which can be persisted and reloaded. For agentic applications, queries can be routed to the index or engine best suited to the request, and event-driven workflows compose retrieval and reasoning steps into multi-step applications. The company behind the framework has since shifted its primary focus to document parsing and extraction, while keeping the open-source framework available.

2.6.4 Microsoft Agent Framework

The Microsoft Agent Framework is an open, multi-language framework, available for .NET and Python, for building agents and multi-agent workflows intended for production [35]. It is the successor of two earlier Microsoft projects, Semantic Kernel and AutoGen, for both of which migration guides are provided.

The framework combines individual agents, which can be backed by different model providers, with graph-based workflows that support sequential, concurrent, hand-off and group-collaboration patterns. Workflows provide checkpointing, streaming, human-in-the-loop control and the ability to return to earlier states, and observability is built in through OpenTelemetry. Agents can be hosted so that they are reachable through the A2A protocol.

2.6.5 CrewAI

CrewAI is an open-source Python framework for multi-agent applications that distinguishes two levels of control [36]. At the higher level, agents are defined as personas characterised by an explicit role, a goal and a backstory that shape their reasoning, and are grouped into a **crew** that carries out a set of tasks. A crew executes its tasks through a process, either sequentially, with each task passed down a line, or hierarchically, with a manager agent that delegates tasks and reviews their outputs. At the lower level, **flows** define event-driven workflows that give precise control over the sequence of steps and can invoke crews where autonomous collaboration is wanted. Tasks can declare structured outputs, guardrails and human review, and the framework is frequently used to automate business processes that map naturally onto roles.

2.6.6 OpenAI Agents SDK

The OpenAI Agents SDK is an official, open-source Python framework for building multi-agent workflows [37]. Despite its origin it is provider-agnostic, supporting OpenAI's own interfaces as well as a large number of other models.

The SDK is built around a small set of primitives: **agents**, which combine a model with instructions and tools; **hand-offs**, through which an agent transfers control of the conversation to another, and the alternative of invoking an agent as a tool; **tools**, which include ordinary functions, hosted tools and the tools of MCP servers; **guardrails**, configurable checks on input and output in the sense of Section 2.4.4; mechanisms for

involving humans during a run; **sessions**, which manage conversation history across runs; and built-in **tracing**. By favouring a small runner loop that automates tool execution and message history over explicit state abstractions, the SDK offers a compact and traceable execution model, with control flow decided by the agents themselves through hand-offs.

2.6.7 Comparative evaluation

Table 2.2 compares the frameworks on the three respects introduced at the beginning of the section, together with the interoperability protocols each supports natively. The comparison is based on the official documentation of each framework at the time of writing, and all of them evolve quickly.

Framework	Control flow	State and persistence	Human in the loop	Protocols
LangGraph	Explicit graph of nodes and edges, cycles allowed, declared by the developer	Checkpoints per thread; stores across threads	Native interrupts, resumed by command	MCP via adapters; A2A via Agent Server
LangChain	Chains and prebuilt agents; orchestration delegated to LangGraph	Short-term memory; durable state through LangGraph	Through LangGraph	MCP via adapters
LlamaIndex	Routing over indices and query engines; event-driven workflows	Persistent indices and storage	Human input events in workflows	MCP
Microsoft Agent Framework	Graph-based workflows: sequential, concurrent, hand-off, group	Workflow checkpoints; return to earlier states	Native in workflows	MCP, A2A
CrewAI	Crews run by a sequential or hierarchical process; event-driven flows	Agent memory	Human review of tasks	MCP, A2A
OpenAI Agents SDK	Runner loop; control passed by agents through hand-offs	Sessions holding conversation history	Built-in	MCP, built-in

Table 2.2: Comparison of agentic AI frameworks.

The frameworks divide along the first column. In LangGraph and in the workflows of the Microsoft Agent Framework the path of execution is declared in advance and the agents act within it; in CrewAI’s hierarchical process and in the OpenAI Agents SDK a larger part of the path is decided at run time by the agents themselves. The first approach is better suited to workflows that must stop, resume and be audited at known points; the second to open-ended collaboration where the path cannot be foreseen. Chapter 7 states which of these properties the artifact requires and why LangGraph was adopted.

3. Literature Review

The rapid expansion of the Internet of Things (IoT) has made its infrastructures larger and more heterogeneous, and configuring them has become a significant bottleneck. At the root of this difficulty lies a semantic gap between the abstract, high-level intent of an operator and the low-level actions that each platform requires, and bridging it is a primary challenge in modern IoT ecosystems.

To systematically evaluate the state-of-the-art solutions attempting to address this gap, this literature review analyzes existing frameworks and approaches. The goal of this review is to identify the current capabilities and limitations of existing systems, highlighting the necessity for a novel agentic architecture. To ensure a rigorous and structured comparison, the reviewed works are evaluated against a predefined matrix of characteristics. These characteristics represent the core functional requirements necessary to achieve the objectives of this thesis (empowering IoT architects to deploy complex infrastructures autonomously using Agentic AI).

The comparative evaluation is grounded in the following characteristics:

- **C1 Natural Language Intent Support:** Does the system accept abstract, unstructured natural language as the primary method to initiate the deployment process?
- **C2 Hierarchical Multi-Agent Architecture:** Does the system distribute deployment tasks across a hierarchical network of specialized, context-sharing entities rather than a centralized execution?
- **C3 Automated IoT Deployment:** Does the system automatically execute the physical deployment of IoT configurations and infrastructure, eliminating the need for manual, device-by-device provisioning by human operators?
- **C4 Dynamic Workflow Resilience:** When encountering a disruption or dynamic change during execution, does the orchestration engine retain its workflow state and dynamically adapt its path, rather than failing and requiring a complete system restart?
- **C5 Context and Memory Tracking:** Does the system implement persistent context and memory tracking across multi-step workflows, allowing it to remember past actions, delegated sub-tasks, and environmental history without losing the thread of execution?
- **C6 AI-Driven Reasoning Capability:** Does the system employ an AI-driven reasoning engine to derive the sequence of deployment actions from the request, rather than requiring the operator to select a pre-programmed template or script? The criterion concerns where the interpretation of the request comes from; it does not exclude that what the reasoning produces is then checked deterministically.

3.1 Data collection methodologies

In order to collect papers relevant for the literature review, the methodology used is to exploit a search query in Scopus. The definition of a search query is based on the so-called keywords, elementary terms that, joined together, allow extracting the desired results: they are joined by means of logical operators (such as AND, OR, NOT), so as to define the way in which the same terms will be searched:

("Agentic AI" OR "LLM Agent*" OR "Autonomous Agent*" OR "Multi-Agent" OR "MAS") AND ("Internet of Things Deployment" OR "IoT Deployment") AND ("Architecture" OR "Framework" OR "Design")

The first query, centred on multi-agent approaches to IoT deployment, returned no system accepting natural language as its input, which is the capability C1 describes. Since that absence could reflect the query rather than the literature, a second query was run restricted to titles and abstracts of works published from 2023 onwards:

("large language model" OR "large language models" OR "LLM") AND ("Internet of Things" OR "IoT" OR "smart home" OR "smart building") AND ("natural language") AND ("configuration" OR "provisioning" OR "deployment" OR "automation" OR "orchestration")

The query returned 195 records in September 2026. The records were sorted by number of citations, and the titles and abstracts of the forty most cited were screened against the same criteria described below; four works were retained and read in full.

3.1.1 Inclusion and exclusion criteria

To evaluate the state of the art in Intelligent IoT and Agentic architectures, a systematic selection process was applied to the collected bibliography.

The inclusion of research was governed by three primary criteria:

- architectural relevance to multi-agent orchestration
- alignment with established system characteristics (mentioned previously in the introduction of the literature review)
- methodological diversity, meaning that at most two works adopting the same technique (for instance reinforcement learning) were retained

Papers were prioritized if they presented alternative orchestration strategies (such as Hierarchical Deep Multi-Agent Reinforcement Learning) which serve as essential baselines for contrasting traditional statistical optimization against generative, intent-based agentic workflows.

Conversely, papers were excluded if they focused exclusively on low-level physical layer metrics (for example raw packet scheduling or specific hardware-level energy consumption) without addressing higher-level semantic reasoning or system-wide orchestration. Furthermore, studies that repeated existing, well-documented methodologies already established in the background chapter were omitted to ensure the literature review maintains a focus on innovative approaches to agentic IoT deployment.

3.2 Discussion

This section compares the selected works against the characteristics C1–C6, whose assessment is summarised in Table 3.1.

X-IoT [38] makes IoT applications portable across platforms through model-driven engineering, and Softwarized Edge Intelligence [39] deploys containerised industrial IoT services across the cloud/edge continuum. Hi-MAD [40] and Lin et al. [41] use reinforcement learning, respectively a hierarchy of agents for resilient IoT services and cascaded loops for resource orchestration in dense deployments. The other works rely on language models: Deng et al. [42] handle urban sensor data for traffic navigation, dMCP [43] generates quality-of-service and security policies through the Model Context Protocol, RuleBot++ [7] creates Home Assistant automations through conversation, TARGE [44] translates user intents into IFTTT rules, Jahanbakhsh et al. [45] generate services from natural language and deploy them at runtime, and LUMEN [8] uses multiple agents and a knowledge graph to analyse IoT systems.

Table 3.1: Comparative evaluation of literature approaches based on system characteristics.

Approach	C1	C2	C3	C4	C5	C6
X-IoT[38]	✗	✗	✓	✗	✗	✗
Hi-MAD[40]	✗	✓	✓	✓	✓	✓
LLM for traffic navigation[42]	✗	✗	✗	✓	✓	✓
Resources orchestration for IoT[41]	✗	✗	✗	✓	✓	✓
dMCP Framework[43]	✗	✗	✓	✓	✗	✓
Softwarized Edge Intelligence[39]	✗	✗	✓	✓	✗	✗
RuleBot++ [7]	✓	✗	✗	✓	✓	✓
TARGE [44]	✓	✗	✗	✗	✗	✓
RAG smart home orchestration [45]	✓	✗	✓	✓	✗	✓
LUMEN [8]	✓	✓	✗	✓	✓	✓

C1: Natural Language Intent Support

C2: Hierarchical Multi-Agent Architecture

C3: Automated IoT Deployment

C4: Dynamic Workflow Resilience

C5: Context and Memory Tracking

C6: AI-Driven Reasoning Capability

3.2.1 Natural Language Intent and Architectural Hierarchy

A significant gap exists across the literature of the first query regarding **natural language intent (C1)**, as none of the analyzed systems natively accept abstract, unstructured text to initiate workflows. Instead, the Hi-MAD framework[40] and the MARL

orchestration system [41] rely strictly on structured, numerical metrics like Quality of Service or channel states. While the LLM platform for traffic navigation [42] integrates natural language models, it restricts their use to parsing routing patterns rather than driving broader infrastructure provisioning workflows. Similarly, both the dMCP framework [43] and the softwarized edge architecture [39] process technical system parameters or telemetry inputs directly rather than intuitive human-centric prose.

To manage operational workloads, the **architectural paradigms (C2)** differ significantly. The Hi-MAD framework [40] explicitly implements a strict vertical hierarchy that divides tasks across global, context, and device-level layers. The remaining approaches—including the model-driven X-IoT [38], the traffic navigation model [42], the spectrum management framework [41], the dMCP framework [43], and the data-driven continuum plane [39] operate without this specific hierarchical multi-agent orchestration setup, relying instead on specialized peer nodes, standalone software clients, or direct agent-to-infrastructure communication interfaces.

The works retrieved by the second query change this picture with respect to **C1**: all four accept natural language as their primary input. RuleBot++ lets end users create and modify smart home automations by conversing with a GPT-4 based agent [7]; TARGE maps user intents onto trigger-action rules for the IFTTT platform [44]; the orchestrator of Jahanbakhsh et al. turns requirements expressed in natural language into executable services [45]; and LUMEN answers operators' questions about an IoT system, asking follow-up questions when a request is incomplete [8]. In each case, however, the language describes an automation, a rule or a question about the system, not an installation to be configured. As for **C2**, LUMEN is the only one to organise its work hierarchically: a group manager coordinates specialised agents, and a task planner decomposes each request into subtasks and revises the plan when execution fails [8]. RuleBot++ relies on a single agent that invokes tools through function calling, while TARGE and the orchestrator of Jahanbakhsh et al. are pipelines of successive stages.

3.2.2 Deployment Automation and Workflow Resilience

When examining how these systems **automate IoT deployment (C3)**, capabilities vary. Portable model-driven tools like X-IoT [38] and dynamic runtime composition frameworks like Hi-MAD [40] successfully execute automated infrastructure orchestration to eliminate manual device-by-device provisioning. This deployment capability is also shared by the dMCP protocol [43], which dynamically maps intents into executable network policy vectors, and the softwarized edge deployment [39], which utilizes programmable infrastructures and containerized microservices to trigger dynamic adaptations across the cloud/edge continuum. Conversely, the frameworks presented in [41] and [42] focus on edge-layer data processing and physical layer channel coordination, without autonomously deploying systemic infrastructure configurations.

Despite these differences in deployment, **dynamic workflow resilience (C4)** is strongly represented. Except for the static configurations of X-IoT [38], all analyzed frameworks retain workflow execution states under operational drift. The optimization loops in [40], [42], and [41] systematically handle network disturbances and traffic changes. This resilience is replicated in dMCP [43] via its event-driven trigger mechanism that re-evaluates configurations during environmental drops, and within the softwarized architecture [39] through a modular control plane that handles ongoing telemetry variations on-the-fly.

Among the works of the second query, only the orchestrator of Jahanbakhsh et al. deploys what it generates (**C3**): the service produced by the language model is compiled into an OSGi bundle and installed at runtime, without restarting the system [45]. What is deployed, however, is service logic on a local runtime rather than the entities of an installation on an IoT platform. RuleBot++ produces automations that Home Assistant executes, but for devices already installed and described in the configuration of the home [7]; TARGE recommends rules without executing them, and is evaluated on a dataset [44]; LUMEN generates and executes analysis code, not configurations [8]. Resilience (**C4**) is again the most common characteristic. The orchestrator isolates a generated bundle that fails, so that the rest of the environment remains stable [45]; RuleBot++ verifies each automation and, when the code is invalid, asks the model to regenerate it instead of aborting [7]; LUMEN evaluates the output of its agents and requests a revised plan when errors are found [8]. TARGE, which generates rules in a single pass, has no workflow to recover.

3.2.3 Context Tracking and Autonomous Reasoning

The capability to **maintain persistent context and memory tracking (C5)** remains restricted to specific platforms. The Hi-MAD framework [40], the traffic navigation system [42], and the resource orchestration loop [41] actively track environmental history and historical channel parameters over time. However, the remaining setups—including X-IoT [38], the dMCP framework [43], and the softwarized edge architecture [39], do not preserve persistent context layers or execution histories across sequential multi-step deployment operations.

This stateful gap impacts the corresponding **AI-driven reasoning capabilities (C6)**. Moving away from traditional templates and deterministic scripts, autonomous reasoning is leveraged by Hi-MAD [40] via Deep Reinforcement Learning, by the navigation platform [42] through domain-adaptive language models, by the spectrum manager [41] via cascaded reinforcement loops, and by dMCP [43] through its integrated LLM-driven cognitive engine. On the other hand, the model-driven parameters of X-IoT [38] and the ML-based inference services of the softwarized edge infrastructure [39] (which rely on fixed classification boundaries rather than flexible reasoning algorithms) lack the capacity to independently deduce novel deployment paths.

Persistent context (**C5**) is present in two of the four works. RuleBot++ stores the automations it creates, together with the conversation logs, in a database from which they are retrieved when the user asks to modify or explain them [7]; LUMEN models the monitored system as a knowledge graph of devices and relationships, which its agents query throughout the analysis [8]. The orchestrator of Jahanbakhsh et al. retrieves knowledge about device capabilities from a vector database, which grounds generation but does not record the actions performed [45], and TARGE keeps no state between requests [44]. All four rely on a language model for reasoning (**C6**): GPT-4 in RuleBot++, OpenAI models guided by prompt templates and retrieved context in the orchestrator, a fine-tuned open model in TARGE, and collaborating LLM agents in LUMEN.

3.3 Key findings and gaps

3.3.1 Key findings

The systematic evaluation of current literature reveals that dynamic workflow resilience is a well-established strength among modern IoT frameworks. Most state-of-the-art systems successfully retain their execution states and dynamically adapt to operational drift, network disturbances, or environmental changes without requiring a complete manual restart. Furthermore, several frameworks demonstrate robust capabilities in automating physical deployments, effectively eliminating the need for manual, device-by-device provisioning through the use of dynamic runtime compositions and event-driven trigger mechanisms. There is also a notable integration of AI-driven optimization techniques—such as Deep Reinforcement Learning and domain-adaptive models—which are actively employed to handle specific edge-layer data processing and traffic adaptations.

The second query shows that natural language input is no longer rare: recent systems based on large language models accept it for smart home automations, trigger-action rules, service generation and system observability, and several of them pair it with verification or self-correction of what the model produces.

3.3.2 Identified gaps

Despite the strengths observed in existing architectures, several critical limitations prevent current systems from achieving fully autonomous, intent-driven IoT deployment. These gaps can be categorized into the following core areas:

- **The Semantic Communication Barrier:** The systems retrieved by the first query lack native support for abstract, unstructured natural language. Current frameworks force operators to initiate workflows using structured numerical metrics, rigid technical parameters, or direct telemetry inputs. This leaves a significant semantic gap between intuitive, human-centric design goals and the low-level execution required by the system. The works retrieved by the second query overcome this barrier, but for automations, rules and analytical questions rather than for the description of an installation to be provisioned.
- **Absence of Unified Hierarchical Orchestration:** The prevailing architectural trend favors specialized peer nodes, standalone clients, or direct software-to-infrastructure interfaces rather than a centralized, multi-agent hierarchy. With rare exceptions, systems fail to distribute complex deployment tasks across a managed network of specialized, context-sharing entities, significantly limiting their ability to decompose overarching goals into coordinated sub-tasks.
- **Inconsistent Infrastructure Automation:** While some frameworks achieve automated provisioning, a large portion of the literature restricts automation to localized edge-layer data processing or physical layer channel coordination. These systems lack the broader capability to autonomously orchestrate and deploy systemic virtual infrastructure configurations across IoT platforms.
- **State Management and Autonomous Reasoning Deficits:** Many frameworks fail to implement persistent context tracking or retain execution histories across sequential, multi-step deployment operations. This inability to maintain a

stateful memory directly hampers advanced reasoning capabilities. Where reasoning is present, it is either left unconstrained or replaced altogether by fixed scripts, classification boundaries or model-driven templates; none of the reviewed systems combines a model that interprets the request with deterministic checks on what it produces.

- **No combination of intent, hierarchy and deployment:** natural language input, hierarchical multi-agent orchestration and automated deployment each appear in the literature, but never together. Hi-MAD combines hierarchy and deployment without natural language, LUMEN combines natural language and hierarchy without deployment, and the orchestrator of Jahanbakhsh et al. combines natural language and deployment without a hierarchy of agents.

4. Research Methodology

The contribution of this work is an artifact: a system built to solve a problem observed in practice. Building it is not a result in itself. Three things have to be shown: the problem is real, the design is justified, and the system’s behaviour can be assessed against the problem. This chapter describes how the research is set up to show them. It covers the paradigm, the process model, the evaluation strategy and the resulting limitations.

4.1 Design science as the research frame

Research in information systems usually follows one of two paradigms. Behavioural science explains or predicts how people and organisations behave. Design science builds and evaluates artifacts that solve identified problems [46]. The first aims at truth, the second at utility.

The research questions of Section 1.3 are constructive and evaluative, not explanatory. They ask which architecture can turn abstract requirements into deployments, how a reasoning engine can coordinate such workflows, and how well the resulting loop works. Observing existing systems cannot answer them, because the object they concern does not exist yet. The work therefore adopts design science and its three cycles [47]:

- a *relevance* cycle, linking the work to its environment: the IoT architect as user, a real building as context and two operational IoT platforms as deployment targets;
- a *rigour* cycle, grounding it in the literature of Chapters 2 and 3;
- a *design* cycle, in which building and assessing alternate.

Following March and Smith [48], the work produces four outputs: *constructs*, the vocabulary for describing installations; a *model* of an installation; a *method*, the agentic loop that goes from a natural-language description to a provisioned platform; and an *instantiation*, the prototype that demonstrates the other three.

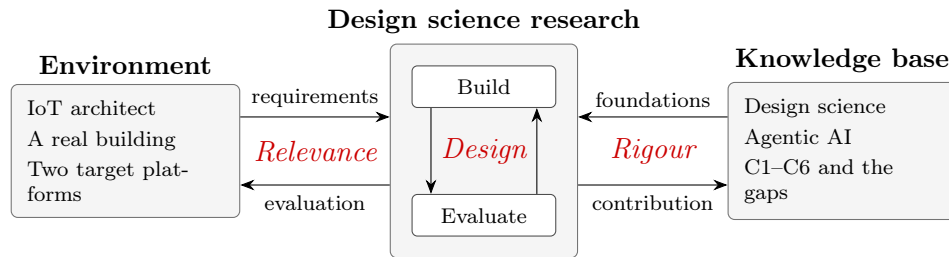


Figure 4.1: The three cycles of design science [47] as instantiated in this work.

4.2 Process model and structure of the work

The work is organised as the sequence of phases that design science research conventionally follows [49]. It begins with *awareness of the problem*, which means establishing that a real difficulty exists and turning it into requirements. *Suggestion* proposes a design capable of satisfying them. *Development* builds that design into a working system. *Evaluation* confronts the system with the requirements it was meant to satisfy. *Conclusion* interprets what the outcome shows. The order is not rigid: a later phase can send the work back to an earlier one, and this happened repeatedly here, since a design decision often revealed its consequences only once implemented, and the design was revised rather than the implementation forced to follow it.

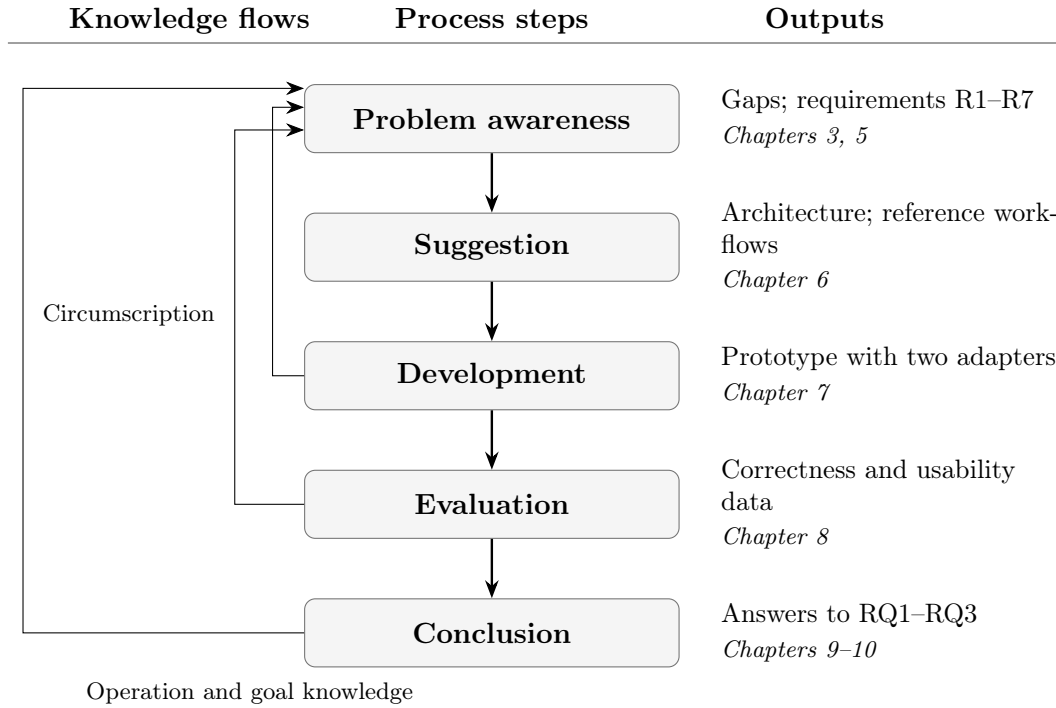


Figure 4.2: The design cycle followed by this work [49], with what each phase produced and where it is reported.

Where the sequence starts also matters, because it determines what the work is answerable to. Research of this kind can equally begin from an available technology and look for an application of it [50]. This one begins from a problem observed beforehand, the manual configuration of a real installation, and agentic AI was chosen afterwards as the means of addressing it.

Two provisions keep this sequence traceable. The first concerns the review. The six characteristics C1–C6 against which Chapter 3 compares existing approaches, from natural-language intent to AI-driven reasoning, are stated before any work is examined against them. The gaps that emerge therefore justify the design, instead of describing after the fact what was built anyway. The second concerns the requirements. R1–R7 are derived from those gaps together with the experience of configuring a real building by hand, so each one answers both a documented limitation of existing work and a practical need.

4.3 Design, development and evaluation

Chapter 6 presents the design as an ordered sequence of decisions. Each decision states the question at issue, the available alternatives, the criteria applied and the option chosen. The rejected alternatives are recorded so that a reader who might have chosen differently can evaluate the design, and not only understand it.

Development is iterative. The three reference workflows of Section 6.6 are defined during design and then serve as the behavioural specification of the prototype. They are used in two ways, and each produces a different kind of evidence:

- during construction, the author uses them formatively to find where the implementation departs from the design. This shows that the artifact behaves as designed;
- in Chapter 8, two of them become tasks for participants. Only this second kind of evidence shows whether the design serves its intended users.

Design science evaluations differ in purpose, formative or summative, and in setting, artificial or naturalistic [51]. Here the risk is acceptance rather than feasibility: whether an architect can describe an installation in their own words, whether the system asks for what is missing instead of inventing it, and whether what reaches the platform is what was intended. The plan therefore moves from formative artificial checks during construction to a study with users in a realistic setting, instead of extending the artificial testing. Correctness is measured by comparing the entities created on the platform with the internal model, which serves as ground truth; perceived usability, with standardised instruments. Section 8.1 gives the protocol, and Table 4.1 shows which part of the work answers each research question.

RQ	How it is addressed	Chapter	Form of the answer
RQ1	Derivation of a hierarchical orchestration architecture from the requirements, with the alternatives and the criteria of choice stated	6	Justified design
RQ2	Realisation of the architecture on a multi-agent framework, with constrained generation, external grounding, deterministic validation and persistent state	7	Working instantiation
RQ3	Study with representative users on tasks derived from the reference workflows, combining extracted correctness with usability instruments	8 and 9	Empirical data and its interpretation

Table 4.1: Correspondence between research questions and the parts of the work that address them.

4.4 Ethical considerations and limitations

The study involves human participants. Participation is voluntary and follows written informed consent stating its purpose, what a session involves, what is recorded and

how the data is handled, and making clear that the artifact and not the participant is under evaluation. Data is held under a participant code whose link to the person is kept separately and destroyed once the analysis is complete, and no name or identifying image appears in the thesis.

The choices described above bound what the work can claim:

- **No baseline.** The study has a single condition and no manual comparison, so it can report how accurately and how comfortably the artifact was used, not that it beats configuring by hand.
- **Scope.** The requirements come from one scenario and the tasks are modest beside a real installation, so both reflect one building rather than the field.
- **Platforms.** Two targets support the claim that platform-specific knowledge stays inside the adapters, but do not extend it to platforms with a very different provisioning model.
- **Non-determinism.** The reasoning layer is non-deterministic. The implementation commit, the models and the sampling parameters are fixed and declared before data collection, so the conditions are reproducible even where individual runs are not.
- **Author as evaluator.** The formative checks during construction were run by the author, who is also the designer. They show conformance to the design; the assessment of utility rests on the study with external participants.

5. Problem Awareness

The configuration of a distributed IoT installation inside a real building is a complex, time-consuming and error-prone activity. Before a single measurement reaches a dashboard, someone must describe the building, decide which devices belong in which spaces, define the relationships connecting gateways, devices, rooms and the cloud platform, and reproduce all of this on the platform that will operate the installation.

The effort grows quickly with the size of the environment, but the real fragility is not the volume of work. The conceptual representation of the building, the physical deployment of the devices and the configuration actually present on the platform are three descriptions of the same reality, produced separately and kept aligned only by the diligence of the operator. When they drift apart the installation keeps producing data, but that data is no longer reliably interpretable.

Heterogeneity amplifies the difficulty[1]: devices differ in capabilities, provisioning procedures and required metadata, platforms in the abstractions they expose and in the APIs that manipulate them. Lacking universal standardisation, integration is habitually achieved through custom and brittle “glue code”[3], so that configuration knowledge remains bound to individual practitioners and platforms.

Underneath these obstacles lies a **semantic problem**. The architect reasons in terms of **what** the installation should achieve: which building, how organised, which scenario, which spaces covered. The platform requires **how** this is realised: assets and relations to be created, device profiles to be instantiated, credentials and telemetry keys to be declared. Conventional programmatic approaches cannot bridge the two, since they lack both the reasoning needed to turn an abstract description into a coherent sequence of configuration actions and the ability to ask for what is missing when that description is incomplete, as natural language descriptions almost always are.

What is needed is an explicit intermediation layer between the intent of the architect and the configuration of the target platform: a system that lets the environment be described in natural language, progressively builds a coherent model of the building and its equipment, verifies its consistency, and translates the result into operational deployment on platforms such as ThingsBoard or OpenRemote, guiding the design decisions of the architect rather than replacing them. The separation of intent planning from behaviour realisation promoted by the SAIBA framework[9], together with the standardised interfaces offered by agent interoperability protocols[28, 29], provides the foundations on which this layer can be built.

The **goal** of the solution proposed in this thesis is consequently twofold: to **remove the manual, platform-specific effort** that currently separates the description of a building from a running IoT installation, and to make the resulting configuration explicit, verifiable and reusable, **so that the architect can reason about installations** rather than about the individual platform operations that implement them.

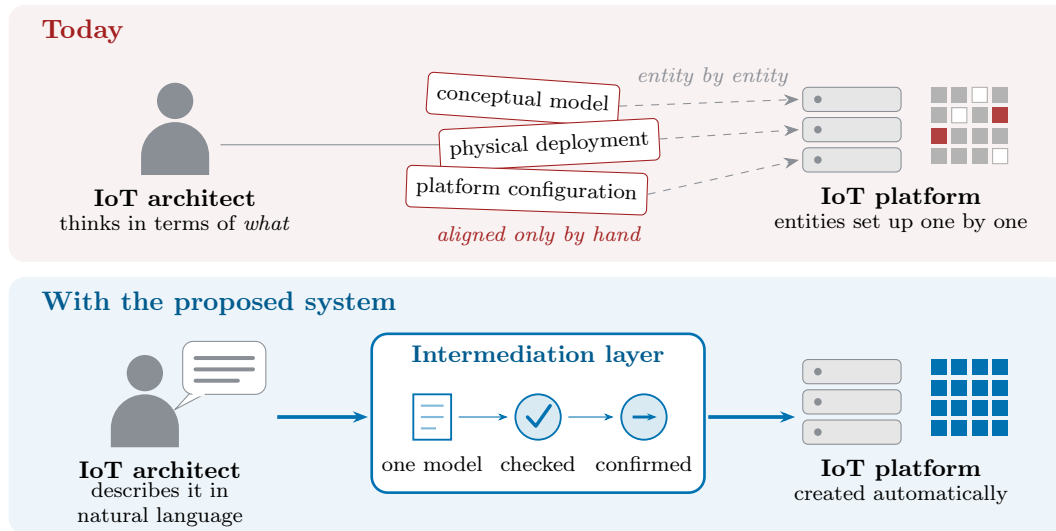


Figure 5.1: The configuration of an IoT installation today and with the proposed system

5.1 Reference scenario

To make the problem concrete, the Computer Science Building of the University of Camerino is adopted as the reference scenario. The building already hosts a set of IoT devices for environmental monitoring and an operational ThingsBoard instance, which makes it suitable both as a source of realistic requirements and as a target for the validation of the prototype. Its configuration was carried out manually, and it is precisely that experience that motivates the present work: spaces had to be represented correctly, each device associated with the room in which it was installed, gateways configured and the platform populated with all the information needed to operate the installation. Table 5.1 summarises the characteristics of the scenario that are relevant to the problem addressed here.

Figure 5.2 shows the building and one of its classrooms, where the environmental sensor of the existing installation can be seen.

Aspect	Description
Site	Computer Science Building, University of Camerino
Environment	Multi-floor building with classrooms, offices, laboratories, meeting rooms, corridors and technical rooms
Existing infrastructure	Environmental monitoring devices, local gateways and edge components
Target platform	ThingsBoard instance in operation, the reference deployment target
Current practice	Manual: spaces, devices, gateways and platform entities created one by one
Observed criticalities	Slow, repetitive work; devices assigned to the wrong space; drift between model, deployment and platform
Role in this thesis	Pilot scenario for design, implementation and validation, without restricting the approach to this building

Table 5.1: Characteristics of the reference scenario.

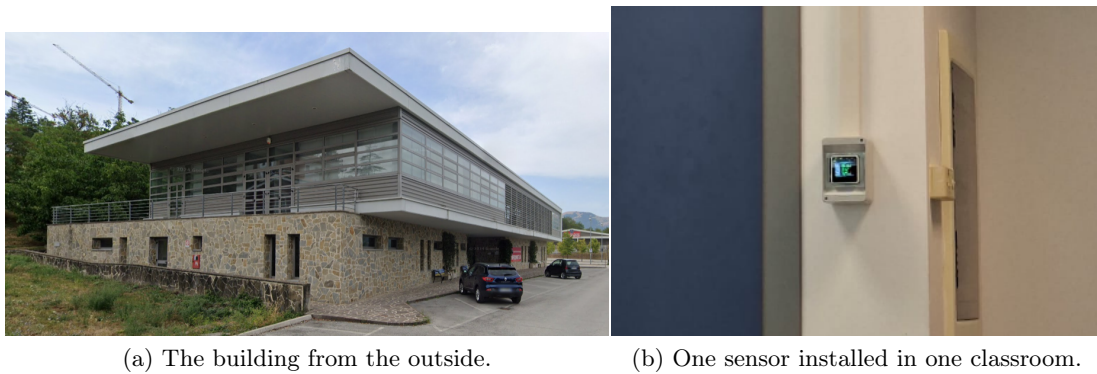


Figure 5.2: The Computer Science Building of the University of Camerino, the reference scenario of this work.

The same building can host installations that pursue different objectives, and the objective determines which devices are meaningful and when a configuration may be considered complete. Three profiles are considered in this work, summarised in Table 5.2.

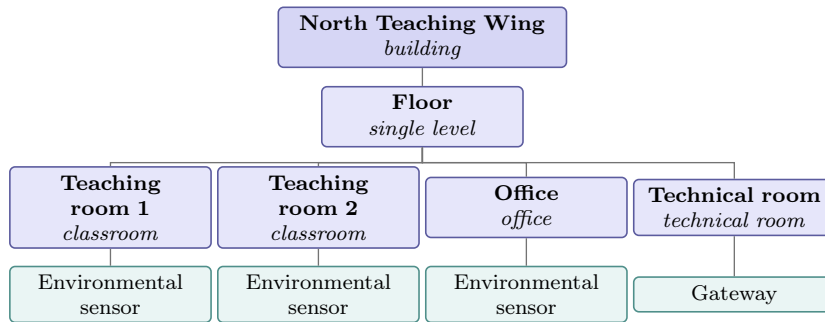
Use case	Description	Configuration implication
Wellness	Monitoring of environmental conditions and occupancy in occupied spaces	Environmental sensing in the selected spaces, people counters where access points are modelled, a gateway
Seismic safety	Preparedness and emergency monitoring in seismic-risk contexts	Early-warning devices bound to specific spaces, stricter association and coverage rules
General	Open scenario in which the architect places catalogue devices directly	Flexible configuration, only the basic completeness rules

Table 5.2: Use-case profiles considered in the reference scenario.

These profiles differ in the equipment involved and in the rules that determine completeness, but they share the same activity: describing an environment, equipping it and reproducing the result on a platform. The Computer Science Building is therefore used as a pilot, while the approach remains general enough for other buildings and use cases.

Since the reference building is too large to be configured within a single session of the user study, the evaluation relies on two reduced scenarios with the same structure and vocabulary (Figure 5.3). The North Teaching Wing is described in full, while the Branch Site, which concerns seismic safety, omits how its offices and laboratories are distributed across floors, so that information has to be obtained by asking. What the participants configure is therefore a portion of the same problem rather than a different one.

(a) Task S1: a complete description



(b) Task S2: an incomplete description

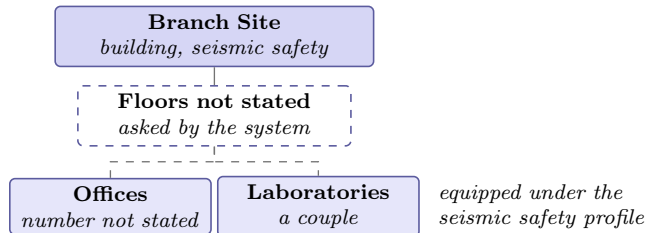


Figure 5.3: The two scenarios used in the evaluation, in the terms of the conceptual model.

5.2 Practical configuration needs of an IoT architect

The main expected user of the system is an IoT architect: a technical or techno-functional profile who understands the purpose of the installation and is responsible for configuring it, who knows the application domain and the physical environment, but who does not wish to declare every entity manually on the target platform. Based on the reference scenario described in Section 5.1 and on the information available in the platforms and device families involved, the following are representative of the practical needs such a user brings to the system:

- **Describe the building in natural language**

The architect should be able to state what the environment looks like in the terms in which it is normally discussed: the name and location of the building, its floors, the rooms present on each of them and their general organisation. This is the entry point of the whole activity.

- **Complete and disambiguate the description through clarification**

A description given in natural language is almost never complete. The system must recognise what is missing with respect to a coherent topology and ask for it, rather than silently filling the gaps with assumptions the architect never made.

- **Obtain a summary of the configuration built so far**

Because the configuration is constructed progressively across a long interaction, the architect needs to see its current state at any moment, in textual form or through a diagrammatic representation of the building and its spaces.

- **Enrich spaces with the metadata that deployment depends on**

Beyond its name and floor, a space carries information that determines what can be installed in it: its type, intended use, accessibility and priority within the scenario. Since this metadata conditions both device selection and the later validation, the system should ask for it explicitly rather than expecting the architect to anticipate which attributes will turn out to matter.

- **Select the use case to be configured**

Wellness, seismic safety and generic installations imply different devices, coverage expectations and completeness rules. Declaring the use case allows the system to bring preliminary knowledge to bear on the configuration, suggesting coherent equipment where the scenario is well characterised.

- **Know which devices are available and what they provide**

The architect needs to consult a catalogue describing which devices can be deployed, which capabilities each offers, which space types they are meaningful in, which metadata they require before commissioning and how they are represented on the target platform.

- **Associate devices with spaces, individually or by class**

Some devices must be placed in a specific room; many others apply uniformly to every space of a given kind, such as all classrooms of a floor. Supporting both modes is what distinguishes a usable assistant from a faster form of manual work, since it is the repetition across homogeneous spaces that dominates the effort in a real installation.

- **Verify completeness and consistency before deploying**

Before anything is written to a platform, the architect needs to know whether the configuration holds together: whether spaces required by the use case are uncovered, whether every device has been associated with a space, and so on. These checks need not be exhaustive to be valuable; they must surface the obvious omissions that manual configuration tends to produce.

- **Choose the target platform and confirm that it is reachable**

The architect must be able to indicate towards which platform the deployment is directed and to supply the information required to reach it, such as endpoint and credentials.

- **Review the planned operations and confirm the deployment**

Autonomy is acceptable only if it is accountable. The architect should see a summary of what the system intends to create, be able to correct it, and give explicit confirmation before execution begins; an equivalent summary should then report what was actually performed.

- **Resume and revise an earlier configuration**

Configuring a building is not a single sitting, and installations evolve. The architect should be able to retrieve a previous configuration, inspect it and continue from where it was left, which requires the system to retain the state of the interaction rather than treating each session as an independent exchange.

5.3 Derived requirements

The needs collected in Section 5.2 can be consolidated into a set of functional requirements for the proposed solution.

- **R1 Natural language as the entry point:** The system must accept an unstructured, platform-agnostic description of a building and of the installation to be realised, and derive from it a functional specification of the desired configuration, in line with the intent planning stage of the SAIBA framework[9].
- **R2 Guided conceptualisation of the environment:** The system must build a coherent model of the building incrementally, requesting the information that is missing with respect to minimal topology rules instead of inferring it silently, and must expose that model to the architect for review at any point of the interaction.
- **R3 Hierarchical decomposition of the workflow:** A supervising agent must decompose the configuration and deployment activity into sub-tasks and delegate them to specialised agents, coordinating their execution and reconciling their outputs, rather than resolving the whole workflow in a single monolithic reasoning step.
- **R4 Catalogue-driven device selection:** Device knowledge must reside in an externally editable catalogue that describes capabilities, compatible space types, required metadata and platform mapping, so that the behaviour of the system can be extended to new equipment without modifying its source code.
- **R5 Validation before execution:** The system must perform completeness and consistency checks against the rules associated with the selected use case, and

present their outcome, together with a summary of the planned operations, before any write operation is performed on the target platform.

- **R6 Automated, platform-mediated deployment:** The system must perform the provisioning and configuration operations itself, through the interfaces exposed by the target platform, eliminating manual entity-by-entity intervention, while remaining structured so that support for an additional platform does not require redesigning the configuration layer.
- **R7 Stateful, resilient and traceable orchestration:** The execution state must persist across steps and across sessions, allowing a configuration to be resumed and a partial failure to be isolated and retried; every reasoning step, delegated sub-task and tool invocation must remain observable to the architect who requested it [6].

Req.	Primary need addressed	Related character- istic
R1	Describing the building in natural language	C1
R2	Clarification, summary, metadata enrichment	C1, C5
R3	Decomposition of a multi-step configuration	C2
R4	Device availability and coherent selection	C6
R5	Pre-deployment completeness and consistency	C3, C6
R6	Execution of the deployment on the platform	C3
R7	Resuming, recovering and auditing the workflow	C4, C5

Table 5.3: Mapping between the derived requirements and the evaluation characteristics introduced in Chapter 3

6. Artifact Design

Chapter 5 established what the system must do: accept a description of a building in natural language, build a coherent model of it, associate devices with its spaces, check that the result is complete and consistent, and deploy it on a target IoT platform, while remaining stateful, resumable and reviewable. This chapter derives the design of the artifact from those requirements through a sequence of decisions: how the installation is represented (Section 6.1), where the knowledge about devices resides (Section 6.2), how natural language is turned into that representation (Section 6.3), how the resulting work is orchestrated (Section 6.4) and . The resulting architecture is presented in Section 6.5 and the last section(6.6) reports three example workflows.

Figure 6.1 anticipates how these decisions fit together, following the three stages of the SAIBA framework; the sections below justify each element in turn.

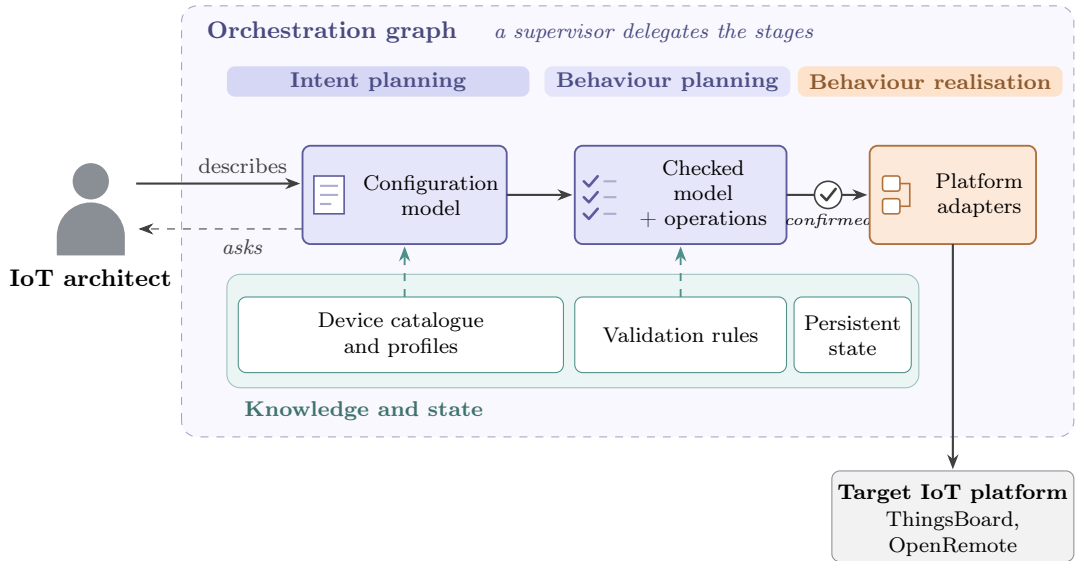


Figure 6.1: An abstract view of the architecture designed in this chapter

6.1 Conceptual model of an IoT installation

The first design decision concerns the representation of the installation, because every other component depends on it: the reasoning layer produces it, the validation logic checks it, the deployment layer translates it and the architect reviews it. A shared, explicit model is what allows these components to operate on the same object rather than on independent interpretations of the same conversation.

Several standardised vocabularies already describe this domain. The Building To-

pology Ontology (BOT)[15] provides a minimal and extensible vocabulary for sites, buildings, storeys, spaces and the containment relations among them; SAREF and its extensions[52] describe devices, their functions and the properties they measure; Brick[17] offers a richer schema for the equipment of smart buildings. Adopting one of these directly would bring semantic interoperability and the possibility of reasoning over the resulting graph, at the cost of introducing an RDF storage layer and a query language into the artifact, and of requiring the language model to emit valid triples rather than a structure it can be constrained to.

Since the objective of this work is the transition from an informal description to a platform configuration, and not the semantic publication of building data, the design adopts a lightweight internal schema serialised as JSON, whose concepts are deliberately named after the corresponding BOT and SAREF terms so that a later mapping towards those ontologies remains a translation exercise rather than a redesign. Table 6.1 lists the entities of the model and their role.

Entity	Description
Tenant	Organisational owner of one or more installations
Installation	A deployment project within a tenant, bound to a use-case profile
Building	The physical building or site being configured
Floor	A level within a building
Space	A room or area within a floor: classroom, office, laboratory, meeting room, corridor, technical room, common area or outdoor space
Access point	A door, passage or entrance associated with a space, where devices such as people counters may be installed
Device	An instance of a catalogue device type, associated with the space or access point in which it is to be installed
Gateway	A device that collects and forwards the data produced by other devices of the installation

Table 6.1: Entities of the conceptual model of an IoT installation.

The relationships among these entities form a containment hierarchy: a tenant owns installations, an installation contains a building, a building contains floors, a floor contains spaces, and devices are associated with the spaces or access points in which they are installed. On top of this hierarchy the model defines a small set of minimal topology completeness rules, which state the conditions under which a model may be considered well-formed regardless of the use case: every building must contain at least one floor, every floor at least one space, and every space must be assigned a type drawn from the controlled vocabulary listed in Table 6.1.

Two properties of this decision matter for the rest of the design. First, the model is the shared state of the system: it is what agents read and modify, what is checkpointed between steps and what is presented to the architect when a summary is requested. Second, having an explicit schema gives the reasoning layer a target to be constrained to, which is what distinguishes an answer that can be validated from an answer that can only be read.

6.2 Device catalogue

The second decision concerns where the knowledge about devices resides. Embedding it in the prompts of the reasoning layer would make it invisible, unverifiable and dependent on the model; embedding it in the code would make every extension of the supported equipment a development activity. Requirement R4 therefore leads to an externalised, declarative catalogue: a YAML artifact that can be edited without touching the source code and that constitutes the knowledge base against which devices are suggested, configurations are validated and platform entities are generated. Table 6.2 describes the fields associated with each device type.

Field	Meaning
<code>device_type_id</code>	Stable identifier of the device type
<code>display_name</code>	Human-readable name presented to the architect
<code>category</code>	Class of equipment: multisensor, people counter, early warning, gateway, coordinator, generic sensor
<code>capabilities</code>	Functions provided: environmental monitoring, presence detection, occupancy counting, structural warning, commandable mode
<code>installation_target</code>	Kind of element the device is installed on: space, access point, structural location, technical room
<code>compatible_space_types</code>	Space types in which the device is meaningful
<code>required_metadata</code>	Fields that must be provided before deployment, such as physical label, network identifier or gateway association
<code>telemetry_model</code>	Telemetry keys the device is expected to publish
<code>command_model</code>	Commands or operating modes supported, where applicable

Table 6.2: Structure of a device type in the catalogue.

The catalogue serves three distinct purposes in the design:

1. During **configuration**, it bounds the set of devices the system may propose, so that suggestions are grounded in equipment that actually exists rather than generated from the parametric memory of the language model
2. During **validation**, the fields `compatible_space_types` and `required_metadata` make the completeness checks deterministic: whether a device may be placed in a space, and whether it carries the information needed to be commissioned, are questions answered by the catalogue rather than by the model
3. During **deployment**, the platform mappings, kept in a file of their own, determine how each device type is represented on the target platform, so that the same model yields the right entities on either of them.

Alongside device types, the catalogue carries the use-case profiles introduced in Section 5.1. A profile associates a scenario with the device categories it recommends, the metadata it makes mandatory and the validation rules that determine when an installation of that kind is complete: the wellness profile requires at least one environmental

monitoring source in every selected space and admits people counters only where an access point has been modelled; the seismic safety profile additionally requires an early-warning source, or an explicit and documented exclusion, and the association of every safety device with a space; the general profile enforces only the minimal rules. In all cases at least one gateway must be present.

The catalogue and the profiles could have been an RDF knowledge graph built on the ontologies of Section 6.1, with the profile rules written as shapes, for instance in SHACL, a standard way to express constraints on existing graph data [53]. They are kept in YAML for three reasons.

First, the code only reads, filters and matches the catalogue’s fields and infers nothing from it, and the profile rules are completeness and consistency checks evaluated directly by the validation code.

Second, the language model never queries this knowledge: it receives lists and admissible identifiers generated by code, so it never has to write SPARQL, and writing semantically correct SPARQL SELECT queries remains difficult in several cases for out-of-the-box models [54].

Third, a review that modelled a prototypical building with five ontologies, among them BOT, SAREF and Brick, found gaps in their support for three energy use cases and concluded that, for such use cases, creating, maintaining and using semantic building models with existing ontologies is difficult, labour intensive and costly[55]. The price is that this knowledge is not interoperable with other semantic data; translating the catalogue to RDF, partly with SAREF, and the rules to shapes is left as future work.

6.3 From natural language to a configuration

The third decision concerns the mechanism that turns the description given by the architect into the model of Section 6.1. Requirement R1 excludes the solution adopted by most configuration tools, in which the architect fills a form or a wizard: such interfaces require the environment to be enumerated in the vocabulary of the platform and offer no support when the description is partial, which is precisely the situation the system is meant to address.

Two families of techniques have traditionally been used to interpret natural language in technical domains. Rule-based processing relies on patterns crafted for a specific domain: it is predictable, but the customisation effort must be repeated for every new vocabulary and it degrades as soon as the phrasing departs from the anticipated patterns. Intent classification adapts better to variation, but requires substantial labelled data, scarce for the configuration of building installations, and still reduces an utterance to one of a predefined set of intents.

Large Language Models remove both constraints, interpreting unconstrained phrasing without domain-specific grammars or task-specific training data and producing structured output rather than a classification label. Their known limitations, however, are directly relevant here: they may generate plausible but incorrect content, offer limited transparency about how a conclusion was reached, and do not retain state between invocations. Left unconstrained, a model asked to configure an installation would readily invent device types, place them in spaces where they do not belong and report a deployment it never performed.

The design therefore does not delegate the task to the model, but confines it to the parts where flexible interpretation is required, surrounding it with deterministic

components. Four provisions implement this arrangement:

- **Constrained generation** The model never emits free text where a structure is expected: the output of the interpretation step is a fragment of the schema of Section 6.1, validated on reception and rejected when malformed.
- **Grounding in the catalogue** Devices are selected from the catalogue, supplied as context rather than recalled by the model, so that the space of admissible choices is defined by an external artifact and not by parametric memory.
- **Explicit clarification** When the description is incomplete with respect to the topology rules or the profile in force, the missing information is requested from the architect. Incompleteness is detected by rules, not by the judgement of the model, which prevents plausible defaults from silently replacing decisions the architect never took (R2).
- **Deterministic validation and human confirmation** Completeness and consistency are evaluated by the validation logic against the catalogue and the profile, and the resulting plan is submitted to the architect before any write operation reaches the platform (R5).

The result is a layered arrangement of the kind described in Section 2.4.3: language understanding and generation are handled statistically, while the constraints that determine whether a configuration is admissible, and the actions that modify the platform, remain symbolic and traceable.

6.4 Orchestration architecture

The activity described in Chapter 5 spans several distinct competences: conducting a dialogue and maintaining a topology, reasoning about equipment, evaluating validation rules, and interacting with a platform API. A single ReAct agent could in principle carry all of them, but it would have to hold the conversation, the model, the catalogue and the platform tools in one context over an interaction that may extend across sessions, with the loss of reliability that context saturation and tool proliferation entail. These lead to a supervisor architecture, in which a coordinating agent decomposes the activity and delegates it to specialised agents.

The design distributes the work over the following agents, all operating on the shared configuration model:

- **Supervisor:** Interprets the request of the architect, determines which stages of the activity it involves and delegates them accordingly.
- **Conceptualisation agent:** Translates the description of the environment into the topology model, refining a single model across successive descriptions rather than replacing it, and detects what is missing with respect to the completeness rules in order to formulate clarification requests.
- **Configuration agent:** Applies the use-case profile, consults the catalogue, proposes or accepts device selections, and associates devices with individual spaces or with every space of a given type.

- **Summary agent:** Renders the configuration built so far, in tabular and diagrammatic form, as a projection of the model presented to the architect for review.
- **Validation agent:** Evaluates the minimal topology rules and the rules of the profile in force against the current model, reports uncovered spaces, unassociated devices, missing metadata and absent gateways, and pronounces on whether the model may proceed to deployment.
- **Deployment agent:** Verifies the reachability of the target platform, derives the sequence of provisioning operations from the model and the platform mapping, and executes it once confirmation has been given, reporting what was performed.
- **Response node:** Not an agent the supervisor can delegate to, but the step that closes every turn, turning what the agents did into the single reply the architect reads. It is the only component that addresses the architect directly.

This decomposition maps onto the three stages of the SAIBA framework [9], as summarised in Table 6.3, and preserves the property that motivated its adoption: what the installation must achieve is expressed independently of how any particular platform realises it, and the platform-specific translation is confined to the final stage.

SAIBA stage	Component	Produced artifact
Intent planning	Supervisor, conceptualisation and configuration agents	Platform-agnostic configuration model of the installation
Behaviour planning	Validation and summary agents, planning phase of the Deployment agent	Model admitted to deployment, and the ordered set of platform operations derived from it
Behaviour realisation	Realisation phase of the deployment agent, platform adapters	Entities and relations created on the target platform

Table 6.3: Mapping of the proposed components onto the stages of the SAIBA framework.

The configuration model is checkpointed after every step and associated with a session, so that an interaction can be interrupted and resumed, and a failure occurring during deployment can be isolated and retried without discarding the work already performed. The transition from behaviour planning to behaviour realisation is an explicit interruption point: the plan is presented, and execution proceeds only on confirmation, which is what keeps an autonomous deployment accountable to the architect who requested it. Confirmation authorises the model as it stood when it was given; the deployment agent establishes anew that the model still satisfies the rules at the moment of execution, so that authorisation is not carried forward across a modification of the configuration.

6.5 Final design

Based on the above considerations, the architecture illustrated in Figure 6.2 is proposed. It is organised in four layers: an interaction layer through which the architect converses with the system, an orchestration layer hosting the supervisor and the specialised agents, a knowledge and state layer holding the configuration model and the device catalogue, and an execution layer that reaches the target platform through MCP servers or REST interfaces, depending on the platform. The components are the following:

- **Conversational user interface** Provides the interaction through which the architect describes the building, answers clarification requests, inspects summaries and confirms the deployment, and allows previous configurations to be retrieved and resumed
- **Orchestration graph** Implements the supervisor and the specialised agents as nodes of a stateful graph, governs the routing between them, and manages the interruption points at which the architect is consulted.
- **Configuration state store** Holds the model of the installation and its checkpoints, providing persistence across steps and sessions and the basis for resuming an interrupted activity.
- **Device catalogue** Supplies the declarative knowledge about device types and use-case profiles, consumed by the configuration agent for selection, by the validation agent for verification and by the deployment agent for platform mapping.
- **Validation engine** Evaluates the minimal topology rules and the rules of the active profile, producing the report presented to the architect before deployment.
- **Platform adapters** Expose the provisioning and management capabilities of each supported platform as typed tools, isolating platform-specific behaviour from the rest of the system.
- **Target IoT platform** The system on which the installation is finally realised; ThingsBoard is adopted as the reference target and OpenRemote as a second one, without the architecture depending on either.

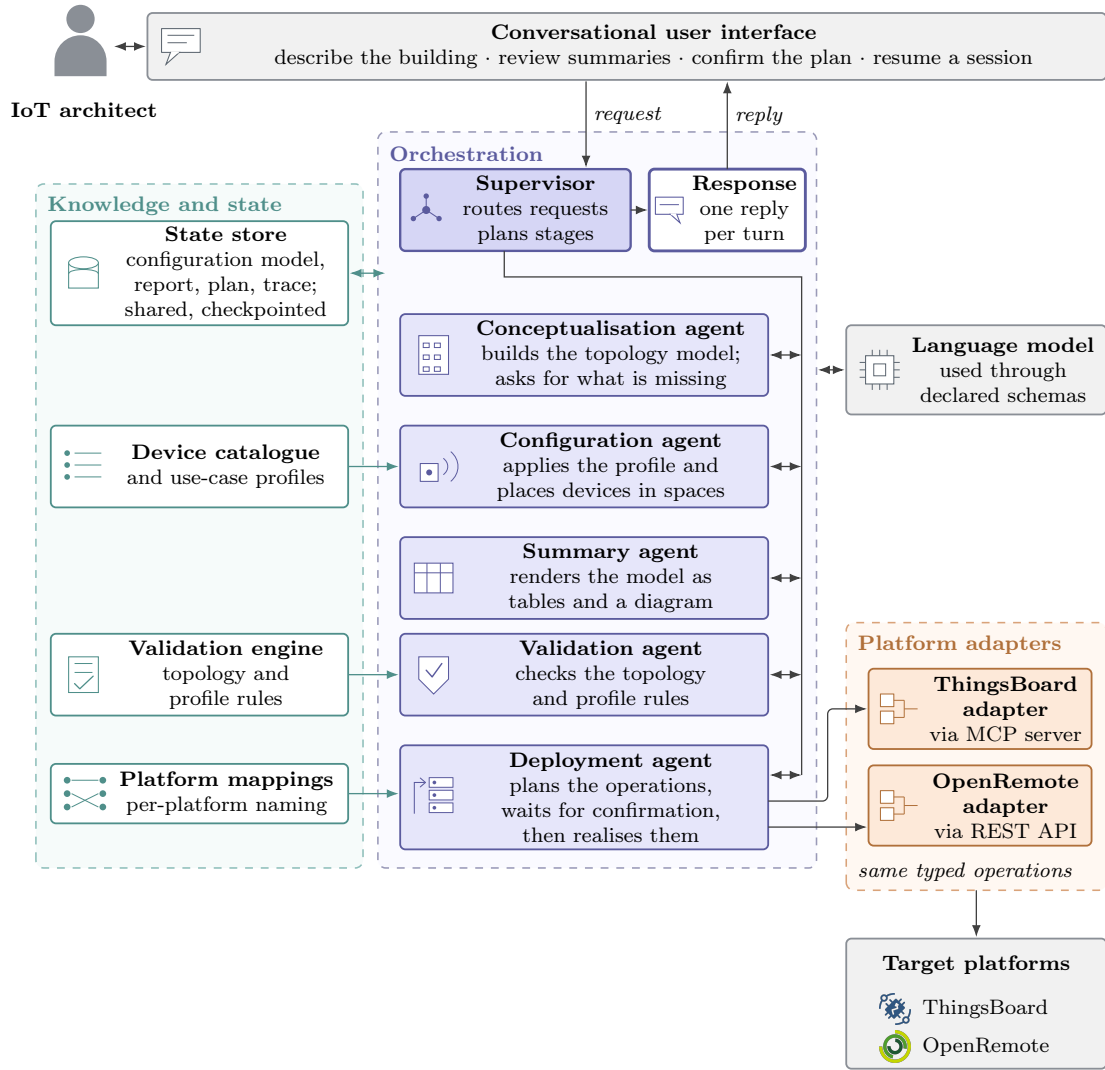


Figure 6.2: Proposed architecture of the artifact.

6.6 Example workflows

The traversal described above is the nominal one, but the components are exercised differently depending on what the architect asks and on what the environment returns. Three representative workflows are reported here: a configuration carried out without incident, a configuration in which the description is incomplete and the validation sends the activity back to an earlier stage, and an interaction that is suspended, resumed and affected by a failure during execution. Each is described as the sequence of delegations performed by the supervisor, since it is that sequence, rather than the individual capability of a single agent, that the architecture is meant to support.

6.6.1 Nominal configuration of a wellness installation

1. The architect states that a wellness installation must be configured in the Computer Science Building, which has three floors. The supervisor delegates to the conceptualisation agent.

2. The agent creates the building and its floors, observes that no space has been declared, and returns a clarification request asking which rooms are present on each floor.
3. The architect enumerates the rooms and declares the main entrance as an access point. The agent assigns a type to each of them, updates the model and returns a summary of the resulting topology, which now satisfies the minimal completeness rules.
4. The architect selects the wellness profile. The configuration agent loads the corresponding rules, consults the catalogue and proposes an environmental multisensor in every classroom and office, a people counter at the main entrance and a gateway in the technical room.
5. The architect accepts the proposal and asks that the multisensors also be applied to every office of the second floor. The agent resolves the space type and creates the associations in a single operation.
6. The validation agent evaluates the model: every selected space is covered and a gateway is present, but two devices lack the physical label required by their catalogue entry. The outcome is reported to the architect.
7. The architect supplies the missing metadata and validation succeeds.
8. The architect select the platform and authenticates to it.
9. The deployment agent verifies that the ThingsBoard instance is reachable, derives the sequence of operations from the platform mapping (assets for the building, floors and spaces, the relations among them, device entities with their credentials and attributes) and presents the plan.
10. The architect confirms, the operations are executed through the platform adapter, and a final summary reports what was created.

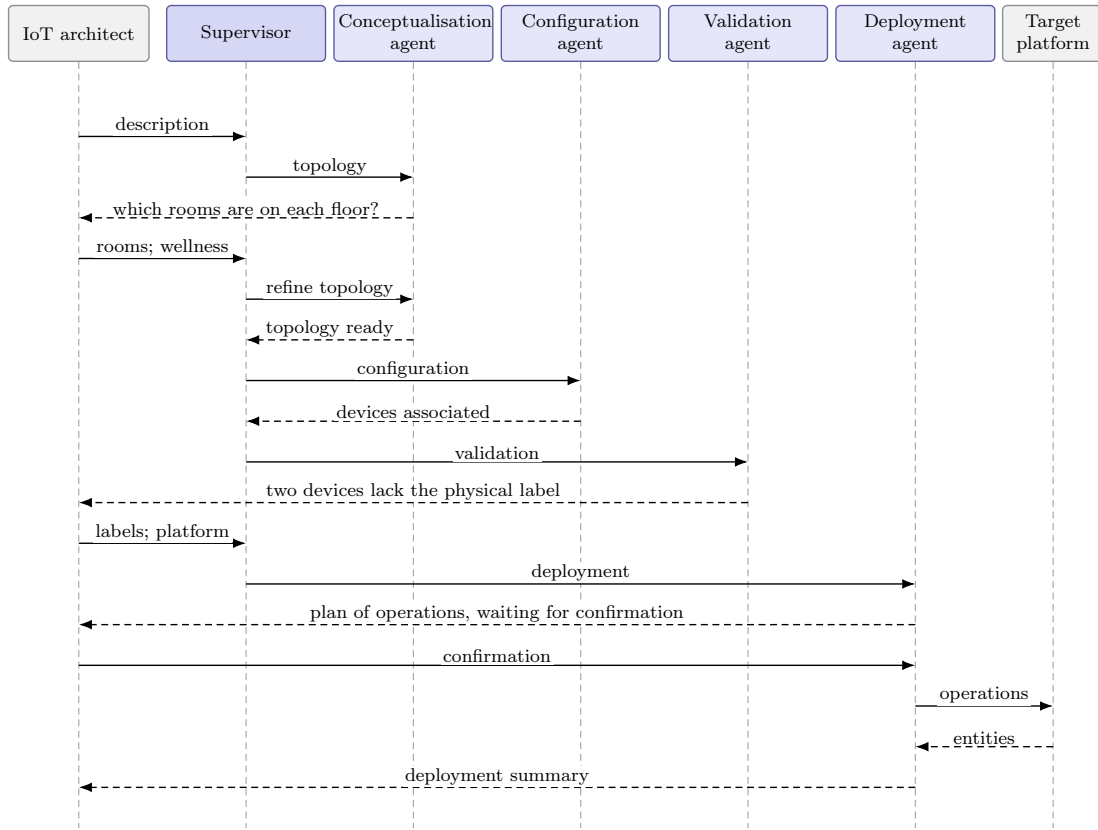


Figure 6.3: Nominal configuration of a wellness installation.

6.6.2 Incomplete description and failed validation

1. The architect describes the building as containing “some offices and a couple of laboratories”, without mentioning floors. The conceptualisation agent cannot construct a well-formed topology and asks how the spaces are distributed across levels.
2. The architect answers partially, indicating that the laboratories are in the basement. The agent adds the corresponding floor and repeats the request for the remaining spaces, which are then assigned to the ground floor.
3. Before proceeding, the architect asks for a summary; the summary agent returns the current state of the model, and the architect corrects the type of one space that had been classified as an office instead of a technical room.
4. The architect selects the seismic safety profile. The configuration agent proposes the corresponding equipment and the associations are created.
5. The validation agent reports that two corridors are not covered by any early-warning source, as required by the profile. The activity returns to the configuration stage.
6. The architect adds the missing devices in one of the corridors and records an explicit exclusion for the other, which is documented in the model rather than silently ignored. Validation is repeated and succeeds, and the workflow proceeds to deployment.

This traversal shows why the control flow of the orchestration layer is cyclic rather than linear: clarification and validation both return the activity to a stage it has already visited, and the state accumulated in the meantime must survive that return.

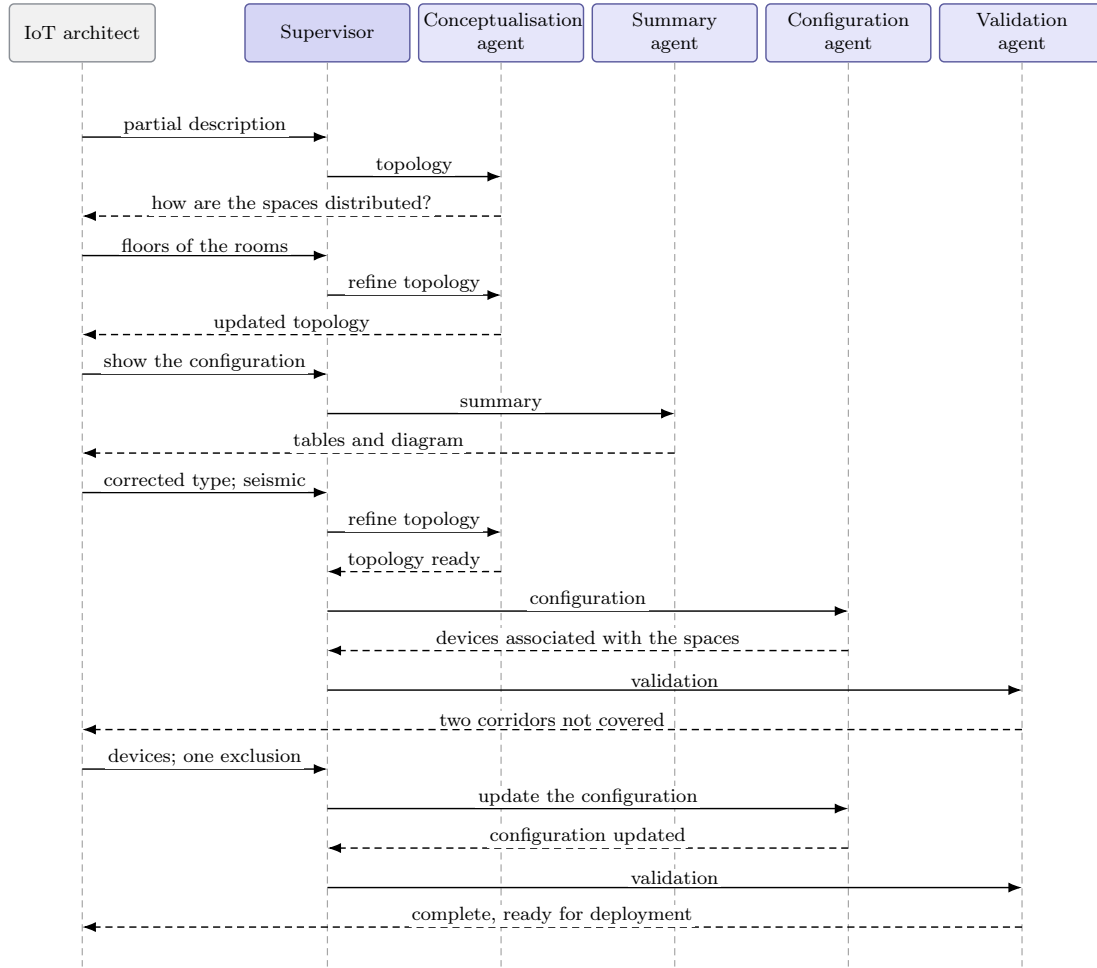


Figure 6.4: Incomplete description and failed validation.

6.6.3 Suspension, resumption and partial failure

1. The architect interrupts the interaction after the configuration stage. The state of the model is checkpointed and associated with the session.
2. Some days later the architect retrieves the session and asks for a summary. The model is restored from the checkpoint and the activity continues from the point at which it had been left, without the building having to be described again.
3. The deployment agent presents the plan, the architect confirms, and execution begins.
4. The creation of one device fails, because an entity with the same name already exists on the platform. The agent stops and reports which operations were completed and which was rejected, with the reason given by the platform.
5. The architect removes the conflict and confirms again. Execution resumes from the failed operation, leaving the entities already created untouched; assets whose

name already exists are reused rather than duplicated. The final summary reports the outcome of both attempts.

The trace of the interaction records which agent handled each stage, which actions were performed and with what outcome, so that the resulting installation can be reviewed after the fact. These three traversals constitute the reference behaviour against which the implementation is exercised in the following chapters.

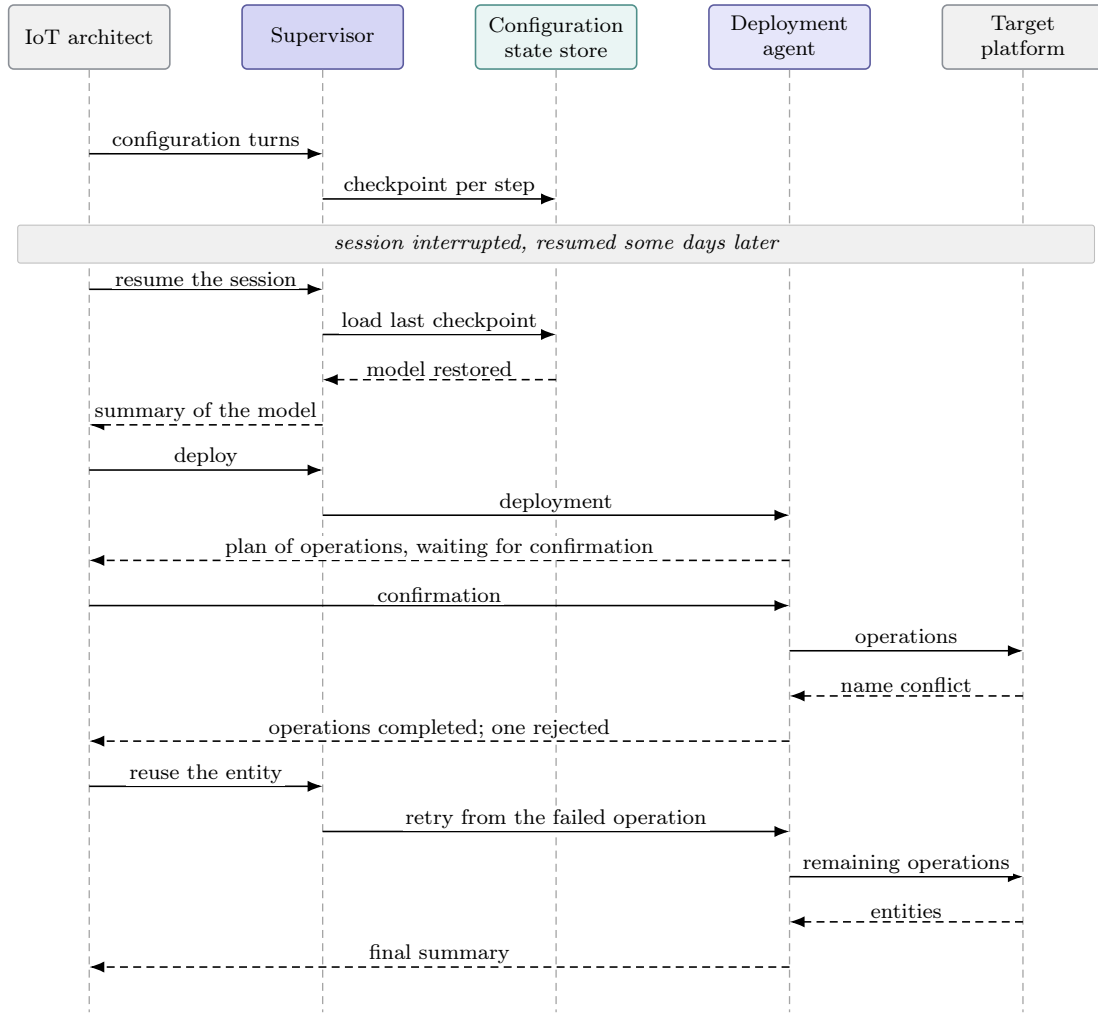


Figure 6.5: Suspension, resumption and partial failure. Solid arrows leaving the supervisor are delegations; dashed arrows are returns, to the supervisor within a turn or to the architect at its end.

7. Artifact Implementation

Chapter 6 established *what* the artifact is: a conceptual model of an installation shared by a hierarchy of agents, a declarative device catalogue, a validation engine and a deployment stage separated from the platform by an adapter. This chapter reports *how* that design was realised. Section 7.1 states which technologies were adopted and what each of them is answering for; Section 7.2 walks through the structure of the code, commenting the parts that carry a design decision rather than the whole of it; Section 7.3 reports what the resulting system does, with the corresponding output of the running prototype.

7.1 Technologies

The technology decisions are not independent of the requirements derived in Section 5.3. Three of them constrain the choice almost entirely: the workflow must be decomposed and delegated rather than resolved in one reasoning step (R3), the state must persist across steps and across sessions so that a configuration can be resumed and a partial failure retried (R7), and no write may reach a platform before the architect has confirmed a plan (R5). What follows justifies each choice against those constraints; Table 7.1 summarises the resulting stack.

Technology	Version	Role in the artifact
Python	3.13	Implementation language
LangGraph	1.2.6	Stateful cyclic orchestration, checkpointing, interruption (R3, R5, R7)
LangGraph SQLite checkpointer	3.1.1	Persistence of the model across steps and sessions (R7)
LangChain	1.3.10	Model client and structured output through function calling (R1)
OpenRouter	—	Provider-agnostic access to the language model
Pydantic	2.12	Conceptual model and constrained generation schemas (R1, R2)
Streamlit	1.60	Conversational interface, session sidebar, configuration panels
Mermaid	11	Client-side rendering of the topology diagram
Docker Compose	—	ThingsBoard, its MCP server and OpenRemote as local targets

Table 7.1: Technologies adopted, and what each answers for.

7.1.1 Orchestration runtime

The runtime is LangGraph 1.2.6. The comparison of Table 2.2 already isolates the properties that matter here, and the artifact needs all four of them at once.

- **Cyclic control flow.** The workflows of Section 6.6 are not linear: a failed validation returns the activity to the configuration stage, and a clarification returns it to conceptualisation, in both cases with the state accumulated in the meantime intact. A framework centred on chains expresses a pipeline; expressing a cycle in it means writing the loop outside the framework and keeping the state by hand, which is why LangChain itself delegates this kind of orchestration to LangGraph (Figure 7.1).
- **Transactional checkpointing.** R7 asks that the model survive an interrupted session and that a deployment resume from the operation that failed. LangGraph checkpoints the whole shared state after every step, keyed by a thread identifier, which is exactly the granularity required: a session is a thread, and resuming it is reading its last checkpoint.
- **A first-class interruption point.** The transition from behaviour planning to behaviour realisation is where the artifact must stop and wait for the architect. LangGraph's `interrupt()` suspends the graph and saves its state; the answer resumes it by re-running the node from its beginning, so the plan is derived again from the current model and operations already completed are not repeated. Without such a primitive, a confirmation would be a flag in a database and a control flow rebuilt on the next message.
- **Integration with MCP.** The tools published by an MCP server are loaded through `langchain-mcp-adapters` as ordinary tools that a node can call, which is how the deployment stage reaches ThingsBoard.

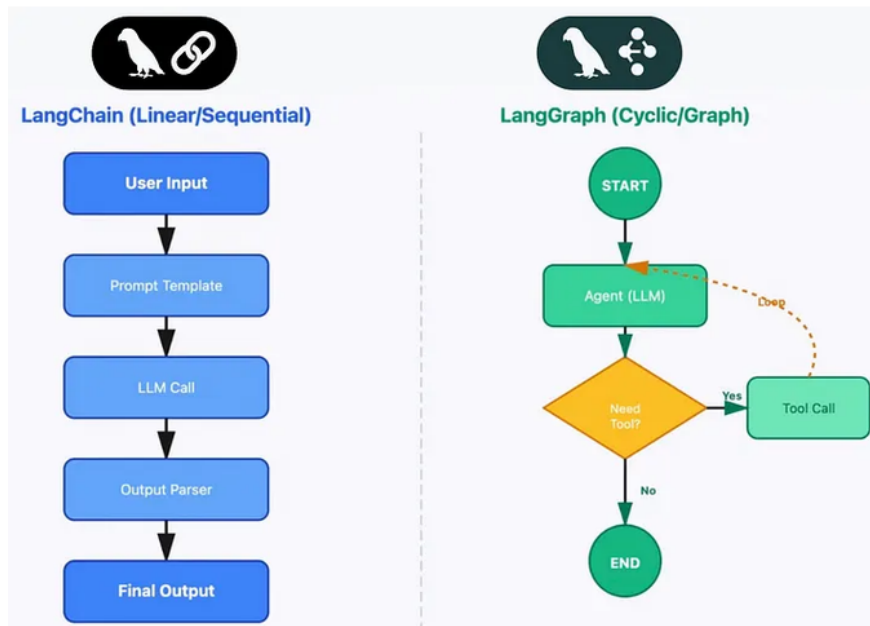


Figure 7.1: Linear execution in LangChain and cyclic execution in LangGraph. Reproduced from [56].

The alternatives were weighed against the same properties. CrewAI and the OpenAI Agents SDK are built around abstractions, role-based crews and hand-offs between agents, that leave a larger share of the path of execution to the agents at run time, whereas the artifact needs a path fixed at known points: validation before deployment, suspension before any write. The sessions of the OpenAI Agents SDK, moreover, preserve conversation history rather than a checkpointed workflow state from which a deployment could resume. LlamaIndex addresses retrieval over private data, which is not the problem here. The Microsoft Agent Framework offers comparable capabilities, with graph-based workflows, checkpointing and human-in-the-loop control, and would have been a credible choice; what decided in favour of LangGraph was the ecosystem the artifact already relied on, since model access and structured output go through LangChain and the ThingsBoard tools through LangChain's MCP adapters, both of which LangGraph consumes without an additional integration layer.

7.1.2 Language model access

OpenRouter is an aggregation gateway (figure 7.2): it exposes the models of many different providers behind a single OpenAI-compatible endpoint and one credential, with the model chosen by an identifier of the form `vendor/model`.

The reasoning layer is reached through it, using the client of `langchain-openai`. The reason is substitutability rather than any property of a particular model: the model identifier is an environment variable, so the same artifact runs on a different provider without a code change, and the evaluation of Chapter 8 can fix and declare one model for every session. The model used in the configuration is `anthropic/claude-haiku-4.5`.

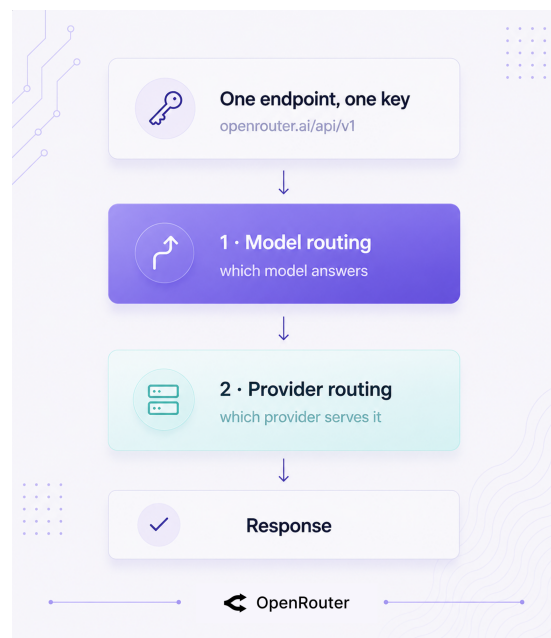


Figure 7.2: Openrouter functioning [57]

Three separate handles onto the model are constructed rather than one, because the three calls fail differently and are worth tuning apart:

- the **general** handle, with a ceiling of 8 192 output tokens, because extracting the topology of a large building is the largest single generation the artifact performs;

- the **router** handle (1 024 tokens), used once per turn for the supervisor’s routing decision, which is a short structured answer that determines which agents run at all;
- the **response** handle (2 048 tokens), used for the single reply the architect actually reads, which is where a weaker model’s inventions would become visible.

Both specialised handles fall back to the general model when their environment variable is unset, so the separation costs nothing when it is not wanted.

Every call that must produce a structure uses **structured output through function calling**: the model does not return prose to be parsed, it returns an instance of a declared schema, which is validated on reception and rejected or repaired when it does not conform.

7.1.3 Data modelling and the knowledge base

The conceptual model of Section 6.1 and every schema the language model is asked to fill are declared with **Pydantic**. The choice is what makes the two coincide: the same type system describes the installation the code manipulates and the shape the model must emit, so a value that would be invalid in the model cannot be returned by the model either. Constrained vocabularies are expressed as `Literal` types, and this is load-bearing rather than stylistic.

The device catalogue, the use-case profiles and the per-platform naming live in three **YAML** files loaded and cached at first use. **YAML** rather than code is what **R4** asks for: supporting new equipment, a new profile or a new deployment target is an edit to a declarative artifact, and the schemas that bound the model’s choices are built from those files at start-up.

7.1.4 Platform access

The two supported platforms are reached in deliberately different ways, and the asymmetry is informative rather than accidental.

ThingsBoard is reached through its official **MCP server**, run as a container alongside the platform and addressed over SSE with `langchain-mcp-adapters`. The provisioning surface is exposed as typed tools maintained by the platform vendor, and the artifact consumes them instead of reimplementing a REST client and tracking its evolution.

OpenRemote publishes no MCP server, and is therefore reached through its own **REST API**, authenticating against Keycloak with the client credentials of a service user. The two platforms consequently differ not only in their vocabulary but in the very protocol by which they are driven. The planning layer above the adapter does not know it, that is the strongest available check on the boundary **R6** postulates.

7.1.5 Interface and persistence

The conversational interface is **Streamlit**. The requirement it answers is modest and specific: the evaluation of Chapter 8 puts a participant in front of the artifact for thirty minutes, so what is needed is a chat surface with a session sidebar and panels that redraw from the stored model, not a production front end. Topology diagrams are emitted as Mermaid and rendered client-side.

Persistence is **SQLite**, through LangGraph’s SQLite checkpointer: a single file, no server to run, and one row per checkpoint per session. The only state kept outside it is a small JSON index of the titles shown in the sidebar; the configuration itself is never duplicated there, so what the sidebar lists and what the graph restores cannot disagree.

7.2 Structure of the code and its implementation

The artifact is built out of patterns that are common in agent systems today, and it is useful to name them before looking at the code: once they are named, the parts that belong to this work in particular are easier to see.

Five of them carry the structure. A *supervisor* decomposes a request and delegates it to specialised workers, which is the hierarchical architecture of Subsection 2.4.3. A single *shared state object* is passed from node to node and checkpointed after each one, the state model the orchestration runtimes are built around. Every call that must produce a decision uses *structured output*, so the model fills a declared schema instead of emitting prose for someone else to parse. Actions on the outside world are reached through *typed tools behind a protocol*, MCP wherever the platform publishes a server. And the one irreversible step is placed behind a *human-in-the-loop interruption*, which the runtime supports as a first-class primitive rather than as a flag the application has to manage. A sixth pattern follows from the first: once a turn has run several agents, their outputs must be *aggregated* into the single answer the user reads.

7.2.1 Layout and requirements

The structure of the code is coherent with the components of the final design of Section 6.5, so that a component of the architecture can be located in the source without a mapping document¹.

Two properties of this arrangement are worth stating because they are the ones the rest of the chapter relies on. First, `domain/` contains no reference to a language model and no reference to a platform: it is the part of the artifact whose behaviour is decidable. Second, everything platform-specific is confined to `platforms/` and to `platform_mappings.yaml`; nothing above that boundary names ThingsBoard or OpenRemote.

```
src/iot_agentic_deployer/
  workflow.py orchestration graph
  agents/ one module per agent
  domain/
    models.py conceptual model of an installation
    state.py shared state, trace, routing schema
    validation.py validation engine
    catalog/ all the yamls files
  platforms/
    base.py adapter contract, operation type, registry
    planning.py derivation of the deployment plan
    thingsboard.py adapter over the ThingsBoard MCP server
    openremote.py adapter over the OpenRemote REST API
    mcp_client.py MCP session handling
  rendering/ tables and diagrams built from the model
```

¹The source of the artifact is available at <https://github.com/atomicSteiner/IoTAgenticDeployer>.

```
ui/chat.py Streamlit interface
```

Listing 7.1: Package layout.

All the requirements are satisfied by a specific part of the code as listed in the table 7.2.

Requirement	Realised by
R1 Natural language entry point	Constrained extraction into the topology schema (agents/conceptualization.py)
R2 Guided conceptualisation	missing_topology_information over the topology rules, plus grounding of the extraction in what was said
R3 Hierarchical decomposition	Supervisor node and the ordered queue of stages (agents/supervisor.py, workflow.py)
R4 Catalogue-driven selection	domain/catalog/: YAML catalogue, profiles and mappings, from which the intent schemas are generated
R5 Validation before execution	domain/validation.py, re-evaluated inside the deployment node above the interrupt
R6 Automated, platform-mediated deployment	platforms/: plan, adapter contract, ThingsBoard over MCP, OpenRemote over REST
R7 Stateful, resilient, traceable orchestration	SQLite checkpointer, model-derived operation identifiers, trace in the shared state

Table 7.2: Where each requirement of Section 5.3 is realised.

7.2.2 The shared state and the graph

Every agent reads and writes one state object, declared as a typed dictionary. Two of its fields carry a reducer, which is how LangGraph is told that concurrent or successive writes accumulate instead of overwriting one another.

```
class IoTDeploymentState(TypedDict, total=False):
    """Everything the agents share as they work."""
    messages: Annotated[list, add_messages]

    installation: dict # models.Installation, serialised
    validation_report: dict # whatever the validation engine last said
    deployment_plan: list # the operations, in the order they run

    # Every hand-off and every tool call stays visible
    trace: Annotated[list, append]

    next_node: str
    pending_nodes: list # stages still to run for this turn, in order
```

Listing 7.2: The shared state (domain/state.py).

The installation is held serialised and revalidated by whichever node reads it, so a node that writes back a malformed model fails at its own boundary rather than corrupting the state for the ones that follow. The `trace` field is used for traceability: every agent appends one entry per action, in the same shape, so the session reads as a single account rather than six sets of notes.

The graph itself is a star. The supervisor is the entry point, a conditional edge sends

control to the agent it selected, and every agent returns to the supervisor; the turn is terminated by a node that is never delegated to.

```
def _router(self, state: IoTDeploymentState) -> str:
    destination = state.get("next_node", "WaitUser")
    return "Response" if destination == "WaitUser" else destination

def _setup_edges(self):
    self.workflow.set_entry_point("Supervisor")
    routing_map = {name: name for name in self.agents}
    routing_map["Response"] = "Response"
    self.workflow.add_conditional_edges("Supervisor", self._router,
                                       routing_map)
    for name in self.agents:
        self.workflow.add_edge(name, "Supervisor")
    # The turn ends with the reply, never by falling out of the supervisor
    self.workflow.add_edge("Response", END)
```

Listing 7.3: Nodes and edges (workflow.py).

The cyclic behaviour the design calls for is a consequence of this topology rather than a mechanism added on top of it: because every agent returns to the supervisor, a validation that sends the activity back to configuration is an ordinary routing decision on the next pass, and the state it returns to is the one the intervening steps left behind. The Response node is deliberately excluded from the dictionary of agents, which is also the list of destinations offered to the supervisor, so it cannot be delegated to by mistake.

A session is a thread identifier and nothing else. The whole session API is four calls onto the compiled graph:

```
def _config(self, session_id: str) -> dict:
    return {"configurable": {"thread_id": session_id}}

def run_turn(self, session_id, user_input, on_step=None) -> str:
    return self._drain(self.system.stream(
        {"messages": [HumanMessage(content=user_input)]},
        config=self._config(session_id), stream_mode="values"), on_step)

def resume(self, session_id, value, on_step=None) -> str:
    return self._drain(self.system.stream(Command(resume=value),
        config=self._config(session_id),
        stream_mode="values"), on_step)

def pending_approval(self, session_id: str) -> dict | None:
    snapshot = self.system.get_state(self._config(session_id))
    return snapshot.interrupts[0].value if snapshot.interrupts else None
```

Listing 7.4: Session API (workflow.py).

Resuming a configuration days later therefore requires no restoration logic of its own: selecting the session in the sidebar selects a thread, and the graph reads its last checkpoint.

7.2.3 The conceptual model

The entities of Table 6.1 are Pydantic models nested along the containment hierarchy, with two traversal helpers that the rest of the artifact uses instead of walking the structure by hand.

```
SpaceType = Literal[
    "classroom", "office", "laboratory", "meeting_room",
    "corridor", "technical_room", "common_area", "outdoor",
    "unknown",
]

class Device(BaseModel):
    """One real device, of a type taken from the catalogue."""
    device_type_id: str
    instance_name: str
    metadata: dict = Field(default_factory=dict)

class Space(BaseModel):
    name: str
    type: SpaceType = "unknown"
    intended_use: Optional[str] = None
    accessibility: Accessibility = "unknown"
    priority: Priority = "normal"
    devices: list[Device] = Field(default_factory=list)
    access_points: list[AccessPoint] = Field(default_factory=list)

class Installation(BaseModel):
    """The shared configuration every agent works on."""
    tenant: str = "default-tenant"
    name: str = "unnamed-installation"
    use_case: Optional[str] = None
    building: Optional[Building] = None
    exclusions: list[Exclusion] = Field(default_factory=list)
    target: PlatformTarget = Field(default_factory=PlatformTarget)

    def iter_devices(self):
        """Walks every device, as (floor, space, access_point or None, device)"""
        for floor, space in self.iter_spaces():
            for device in space.devices:
                yield floor, space, None, device
            for ap in space.access_points:
                for device in ap.devices:
                    yield floor, space, ap, device
```

Listing 7.5: The model, abridged (domain/models.py).

Two decisions of Chapter 6 are visible in these thirty lines. The space vocabulary is a closed Literal, so a room type outside the controlled vocabulary cannot enter the model from any direction. And Exclusion is a first-class entity: a decision not to satisfy a profile rule is written into the model with its justification, which is what lets validation later distinguish a deliberate choice from an oversight.

7.2.4 Routing

The supervisor's decision is itself a constrained generation. The schema is built at start-up from the list of registered agents, so a new agent becomes routable by being added

to one dictionary in `workflow.py`.

```
def build_routing_decision(valid_destinations: list[str]) -> Type[BaseModel]:
    return create_model(
        "RoutingDecision",
        next_nodes=(
            list[Literal[tuple(valid_destinations + ["WaitUser"])]],
            Field(description="The stages this message asks for, in the order "
                             "they run - usually one. ['WaitUser'] to stop "
                             "and await the architect."),
        ),
        response_to_user=(str, Field(description="Message shown to the IoT
        architect.")),
    )
```

Listing 7.6: The routing schema (`domain/state.py`).

The order of the two fields is not cosmetic. A model that writes its prose first and its routing second tends to route towards whatever the prose has just promised; declaring the routing first makes the delegation the decision and the message its consequence.

A single message can legitimately ask for more than one stage (“use the wellness profile and add sensors to the classrooms”). The supervisor therefore returns an ordered queue, decided once, of which the first element is executed immediately:

```
@staticmethod
def _queue(decision) -> list[str]:
    """The stages to run, in order, out of a decision that may be missing
    altogether. Deployment always ends the queue: it stops on its own
    interrupt, so anything behind it would run off the confirmation"""
    if decision is None:
        return []
    queue = [node for node in decision.next_nodes if node != "WaitUser"][:
        MAX_STAGES]
    if "Deployment" in queue:
        queue = queue[:queue.index("Deployment") + 1]
    return queue
```

Listing 7.7: Building the queue of stages (`agents/supervisor.py`).

Deciding once is what keeps the account of the turn honest. Re-deciding on the way back (with the new messages) produced turns in which work that never ran was reported as done, because the second decision read the first agent’s output as though it were the architect’s request.

The supervisor is also the only agent that answers without delegating, and it does so from an explicitly assembled statement of ground truth rather than from the conversation:

```
@staticmethod
def _model_status(state: IoTDeploymentState) -> str:
    inst = Installation(**(state.get("installation") or {}))
    if not inst.building:
        return "Model status: no building has been modelled yet."
    spaces = sum(1 for _ in inst.iter_spaces())
    return (
        f"Model status: building '{inst.building.name}' modelled with "
```

```

f"{len(inst.building.floors)} floor(s) and {spaces} space(s); "
f"use case: {inst.use_case or 'not selected'}; "
f"{len(inst.all_devices())} device(s) assigned; "
f"target platform: {inst.target.platform} "
f"(adapters available: {', '.join(available_platforms())}). "
f"Room types that exist: {', '.join(SPACE_TYPES)}. There are no others
    ."
)

```

Listing 7.8: Model status, read off the stored model (`agents/supervisor.py`).

The last two lines are there because of observed failures: asked which platforms it could deploy onto, or which kinds of room existed, the model answered plausibly and wrongly. Anything the artifact does not state, the model supplies on its own.

7.2.5 Conceptualisation: constrained extraction and grounding

The conceptualisation agent is where natural language becomes a structure, and it is built around two restrictions.

The first is the shape it is allowed to produce. The extraction schema mirrors the spatial part of the model and has *no device field at all*, so equipment cannot enter the model through a description of the building: it enters only through the configuration agent, which checks it against the catalogue. The call is bound to that schema, and the prompt is left to carry only what a schema cannot state.

```

# Shapes used only for extraction, with no 'devices' field on purpose.
# Equipment belongs to Configuration, which checks it against the catalogue
class TopologySpace(BaseModel):
    name: str = Field(description="What this room is called, e.g. 'Classroom 1'.")
    type: SpaceType = "unknown"
    intended_use: Optional[str] = None
    accessibility: Accessibility = "unknown"
    priority: Priority = "normal"
    access_points: list[TopologyAccessPoint] = Field(default_factory=list)

class TopologyBuilding(BaseModel):
    building_name: str = Field(description=(
        "Just the building's name, as the architect said it - for example "
        "'Computer Science Department'. Not the installation, not the use "
        "case, not the name of this schema.))
    location: Optional[str] = Field(default=None, description=...)
    floors: list[TopologyFloor] = Field(default_factory=list)

# The answer is an instance of the schema, never prose to be parsed
self.modeller = llm.with_structured_output(TopologyBuilding,
                                             method="function_calling")

```

Listing 7.9: The extraction schema, abridged (`agents/conceptualization.py`).

Successive descriptions refine one model rather than replacing it, and that is implemented by handing the current model back to the call, projected into the very shape the answer must take. What comes back is then refused if it would discard what the model already held:

```

# The conversation, prefixed by the prompt and followed by the model itself,

```

```

# shown in the shape the answer is expected to take
context = [{"role": "system", "content": self.system_prompt}] + state["messages"]
if inst.building:
    context.append({"role": "system", "content": "Model to refine:\n"
        + _to_topology(inst.building).model_dump_json()})

extracted = self._extract(
    context, had_floors=bool(inst.building and inst.building.floors))

def _extract(self, context: list, had_floors: bool) -> Optional[
    TopologyBuilding]:
    """Runs the extraction, and refuses to hand back a result that would
    throw the model away. The extraction is repeated for better results"""
    extracted = self.modeller.invoke(context)
    if extracted.floors or not had_floors:
        return extracted

    extracted = self.modeller.invoke(context) # once more, then give up
    if extracted.floors:
        return extracted
    return None

```

Listing 7.10: The extraction call, and the answer it will not accept (agents/conceptualization.py).

The second restriction is *grounding*. Left to itself, a model asked to type a set of rooms will type all of them, confidently and sometimes wrongly; asked to model a building it will add the doors it expects such a building to have. Anything the architect never said, and the model did not already hold, is therefore removed from the extraction before it is written.

What counts as said is decided by a second call that reports the same evidence in any language, with the regex patterns kept as the fallback for when it fails.

```

class Mentioned(BaseModel):
    """What the architect's own words name, in whatever language"""
    space_types: list[SpaceType] = Field(description=(
        "Every room type the message itself names. 'three classrooms and an office' "
        "-> ['classroom', 'office']; 'tre aule e un ufficio' -> the same. "
        "Empty when "
        "the message names no room type at all. Never a type merely implied."))
    access_points: bool = Field(description=(
        "True only when the message names doors, entrances, exits, gates or "
        "passages of its own accord."))

def _mentions(self, said: str) -> Optional[Mentioned]:
    """Reads the evidence out of the architect's words. None when the call
    fails, and grounding falls back to the English patterns"""
    try:
        return self.mentions_llm.invoke([
            {"role": "system", "content": (
                "List the room types this message names, and whether it names any "
                "door or entrance. Report only what it says, in any language,

```

```

        never "
        "what it suggests."}},
        {"role": "user", "content": said},
    ])
except Exception:
    return None

# Only the architect's own words can justify a door or a room type
said = " ".join(m.content for m in state["messages"])
    if isinstance(m, HumanMessage))
extracted, dropped_types, dropped_aps = _ground(
    extracted, said, inst.building, self._mentions(said))

```

Listing 7.11: The evidence call, and where its answer enters grounding (agents/conceptualization.py).

The two answers meet in the grounding pass, which walks the extraction and drops whatever neither the architect's words nor the stored model support:

```

def named_type(space_type: str) -> bool:
    # the model's reading of the words, or the English patterns when it
    # failed
    if mentioned is not None:
        return space_type in mentioned.space_types
    return _mentioned(said, TYPE_EVIDENCE.get(space_type, ()))

for fi, floor in enumerate(extracted.floors):
    for si, space in enumerate(floor.spaces):
        # the name first; if it no longer matches it was renamed, so fall
        # back on where it sits
        here = (floor.name, space.name)
        if here not in known_types and by_position:
            here = (fi, si)
        # a type different from unknown survives only if it is mentioned in
        # text
        if (space.type != "unknown"
            and not named_type(space.type)
            and known_types.get(here) != space.type):
            space.type = "unknown"
            dropped_types += 1
        # the access points likewise
        if (space.access_points
            and not named_access_points()
            and here not in known_aps):
            dropped_aps += len(space.access_points)
            space.access_points = []

```

Listing 7.12: Grounding the extraction in what was actually said (agents/conceptualization.py).

A dropped type is not an error but a type invented and kept would be a decision the architect never took. The arrangement is worth stating in general terms, because it recurs: the language model is asked what the words say, never what should follow from them, and the consequence is drawn in code.

Indexing spaces *both* by name and by position exists for a related reason. Devices belong to spaces, and a re-description that renames a room must not orphan its equipment; matching by name first and falling back on position when the shape of the building is

unchanged preserves the association across a rename, and any device that could not be matched is counted and reported in the trace rather than lost in silence.

Finally, what is missing is decided by the rules and not by the model. The agent calls the same function the validation engine uses, and turns its findings into questions:

```
def missing_topology_information(inst: Installation) -> list[str]:
    """What conceptualisation still has to ask about: from the topology
    rules, not from the model's own opinion"""
    questions = []
    for f in check_topology(inst):
        if f.rule == "building_present":
            questions.append("Which building is being configured, and where is
            it located?")
        elif f.rule == "floor_per_building":
            questions.append(f"How many floors does '{inst.building.name}' have
            ?")
        elif f.rule == "space_per_floor":
            questions.append(f"Which rooms are present on floor '{f.scope}'?")
        elif f.rule == "space_type_assigned":
            questions.append(f"What kind of space is '{f.scope}'?")
    return questions
```

Listing 7.13: Clarification requests derived from the topology rules (domain/validation.py).

7.2.6 Configuration: intents bounded by the catalogue

The configuration agent is the largest module of the artifact, and the one where the separation between interpretation and execution is sharpest: a language model decides *what was asked*, and plain Python decides *what happens*.

The intent schema is built at runtime from the catalogue. Device identifiers, use cases, metadata keys and platform names are `Literal` types generated from the YAML files and the adapter registry, which is what grounds selection in equipment that actually exists (Table 6.2) and what makes R4 operational: adding a device type to `devices.yaml` widens the schema, with no code touched.

```
def build_configuration_intent(device_ids: list[str], use_cases: list[str])
    -> Type[BaseModel]:
    return create_model(
        "ConfigurationIntent",
        # A Literal rather than a bare str: matching none of these, the model
        # had nothing to settle on and ran until the token cap cut it off
        action=(Literal[ACTIONS], Field(description=(...))),
        use_case=(Optional[Literal[tuple(use_cases)]], Field(default=None)),
        device_type_id=(Optional[Literal[tuple(device_ids)]], Field(default=
            None,
            description="The catalogue id of the device concerned. ...")),
        target_space_types=(list[str], Field(default_factory=list,
            description="Space types to apply to, e.g. ['classroom'] for 'all
            classrooms'.")),
        target_floor_names=(list[str], Field(default_factory=list, ...)),
        quantity=(int, Field(default=1, ...)),
        quantity_mode=(Optional[Literal["total", "additional"]], Field(default
            =None, ...)),
        # A Literal: left a bare string, 'rename the building to X' arrived
        # here as metadata_key 'building_name', written onto a device
```

```

        metadata_key=(Optional[Literal[tuple(metadata_keys())]], Field(default
            =None, ...)),
        # A Literal for the same reason: a bare string was answered with a
        # platform nobody has an adapter for
        platform=(Optional[Literal[tuple(available_platforms())]], Field(
            default=None, ...)),
        ...
    )

```

Listing 7.14: The intent schema, built from the catalogue (agents/configuration.py).

The comments in the previous listing record two failures that motivated the constraint and are worth reporting as findings. An unconstrained field is an invitation to invent, and the fix is a closed vocabulary rather than a firmer instruction.

Because a structured call can still return a value outside its own vocabulary, the agent asks for the raw answer alongside the parsed one and repairs it instead of losing the turn:

```

for entry in calls[0].get("args", {}).get("actions", []):
    if entry.get("action") not in ACTIONS:
        continue
    for field, values in allowed.items():
        if entry.get(field) is not None and entry[field] not in values:
            guessable = (field == "device_type_id" and named
                and entry["action"] == "assign_devices")
            entry[field] = named[0] if guessable else None

```

Listing 7.15: Repairing an out-of-vocabulary answer (agents/configuration.py).

An invented device identifier is replaced by one the architect demonstrably named (only for an assignment, since an empty removal already means *every device*). Which devices the architect named is itself decided in code, by a ranking over the catalogue that matches names, aliases and, at lower weight, the measurements a device provides, so that “a temperature sensor” resolves to every entry that measures temperature, simplest first.

The handlers themselves contain no model call. They share one targeting rule (a named floor *narrows* the selected spaces rather than adding to them, so “the offices on the second floor” is a conjunction) and they are written to be idempotent, which matters because an architect repeats themselves:

```

# Counted over the room and not over one door: the quantity is what
# the space is to hold, wherever in it the devices hang
present = sum(held(t) for t in targets)
# 'another sensor' adds to what is there; a bare number is what the
# room is to end up with, which is what keeps a repeat idempotent
wanted = min(present + quantity, MAX_PER_SPACE) if adding else quantity
if present >= wanted:
    already += 1
    continue

```

Listing 7.16: Quantities, and why repeating a request is harmless (agents/configuration.py).

Two further behaviours of this agent are worth recording because they implement

design commitments rather than convenience. When the profile in force requires a gateway and none has been placed, one is added automatically to a technical room and every device still lacking a `gateway_id` is bound to it (this is obviously reported to the architect). And the metadata a device type declares as required is pre-filled from templates declared with every defaulted field named to the architect and every placeholder that could not be resolved left empty so that validation still reports it. A default is thus a starting point offered explicitly, never a decision taken on the architect's behalf.

7.2.7 The validation engine

The validation engine is ordinary Python, and deliberately so: it is the component that decides whether a configuration may reach a platform, and that decision must be reproducible.

```
class Finding:
    def __init__(self, level: str, message: str, rule: str = "", scope: str =
        "installation"):
        self.level = level # "error" | "warning"
        self.message = message
        self.rule = rule # rule id, so an exclusion can reference it
        self.scope = scope

    @property
    def blocking(self) -> bool:
        return self.level == "error"

def validate_installation(inst: Installation) -> dict:
    findings = check_topology(inst)
    # Device and profile rules presuppose a well-formed topology.
    if not any(f.blocking for f in findings):
        findings += _check_devices(inst) + _check_profile(inst)

    return {
        "findings": [f.to_dict() for f in findings],
        "errors": sum(1 for f in findings if f.level == "error"),
        "warnings": sum(1 for f in findings if f.level == "warning"),
        "deployable": not any(f.blocking for f in findings),
    }
```

Listing 7.17: Findings and the order of the checks (`domain/validation.py`).

Three properties follow from this shape. Findings carry a rule identifier and a scope, which is what allows a recorded exclusion to reference the rule it excuses and be reported as a documented exclusion rather than as an omission. Checks are layered, so a building with no floors produces one comprehensible error instead of a cascade of consequential ones. And the profile rules themselves are read from `use_cases.yaml` rather than written in the engine: for example what *wellness* requires (a gateway, an environmental monitoring source in every selected space, people counters only where a door has been modelled) is a declaration, and the engine is the interpreter of that declaration.

7.2.8 Behaviour planning

Planning turns the model into an ordered list of platform operations without executing any of them. Two decisions in it carry the resilience the design asks for.

```
def derive_plan(inst: Installation) -> list[PlatformOperation]:
    """Turns the installation into platform operations, in containment order
    so
    nothing refers to a parent that does not exist yet. op_ids come from the
    model, so planning twice gives the same ids and a retry finds its work"""
    ...
    building_op = "asset:building"
    ops.append(PlatformOperation(
        op_id=building_op,
        kind="create_asset",
        description=f"Create building asset '{inst.building.name}'",
        params={"name": inst.building.name, "asset_type": BUILDING_ASSET_TYPE
        },
    ))

    for floor in inst.building.floors:
        floor_op = f"asset:floor:{floor.name}"
        ...
        ops.append(_relation(f"rel:{floor_op}", building_op, "ASSET",
            floor_op, "ASSET", ...))
```

Listing 7.18: Deriving the plan, abridged (platforms/planning.py).

First, operations are emitted in containment order, so nothing ever refers to a parent that does not yet exist. Second an operation's identifier is derived from the model (to make possible a retry), not from the run: the operation that creates *Classroom 1 of the ground floor* has the same identifier in every plan derived from the same building. Identifiers a platform assigns are unknown at planning time, so an operation refers to the output of an earlier one through a placeholder resolved during execution:

```
def execute(self, op: PlatformOperation, refs: dict) -> str | None:
    """Runs one operation. 'refs' maps op_id to the platform id it
    produced, which is how an operation points at something an earlier
    one created."""
    p = dict(op.params)

    # Fill in any {"$ref": "<op_id>"} placeholders
    for key, value in list(p.items()):
        if isinstance(value, dict) and "$ref" in value:
            p[key] = refs.get(value["$ref"])

    if op.kind == "create_asset":
        return self.create_asset(p["name"], p["asset_type"])
    ...
```

Listing 7.19: Resolving references at execution time (platforms/base.py).

Whether a catalogue device becomes a platform *device* or one more *asset* is read from the platform mapping, so the same model yields a plan of the right shape for either target.

7.2.9 The platform adapters

An adapter implements four operations and a reachability check, and registers itself by name; the deployment agent obtains one from the registry and never learns which class it received.

```

class PlatformAdapter(ABC):
    """What every target platform has to be able to do."""
    name: str = "abstract"

    @abstractmethod
    def check_reachability(self) -> tuple[bool, str]: ...
    @abstractmethod
    def create_asset(self, name: str, asset_type: str) -> str: ...
    @abstractmethod
    def create_device(self, name: str, device_type: str, label: str | None =
        None) -> str: ...
    @abstractmethod
    def set_attributes(self, entity_id, entity_type, scope, attributes: dict)
        -> None: ...
    @abstractmethod
    def create_relation(self, from_id, from_type, to_id, to_type) -> None:
        ...

    def batch(self):
        """Somewhere to hold a connection open for a run of operations."""
        return nullcontext()

```

Listing 7.20: The adapter contract (platforms/base.py).

The ThingsBoard adapter implements these over MCP tools. One detail of that integration is worth reporting, because it is the kind of thing a boundary is for: the MCP tools do not raise on failure, they *return* the failure (a dictionary with an error status, or a bare string where an entity was expected). Taken at face value, a refused operation is recorded as completed with an identifier it never received, so the adapter converts both shapes into exceptions at its own edge. The client also holds one MCP session open for the duration of a deployment instead of opening one per operation.

The OpenRemote adapter is the one that tested the boundary. OpenRemote has neither a device entity nor a relation entity: everything is an asset, and containment is not something one creates but where the child sits.

```

def create_relation(self, from_id: str, from_type: str,
    to_id: str, to_type: str) -> None:
    """Containment is not a thing you create here, it is where the child
    sits: read it, move it under its parent, write it back"""
    child = self._call("GET", f"/asset/{to_id}")
    child["parentId"] = from_id
    self._call("PUT", f"/asset/{to_id}", json=child)

```

Listing 7.21: Containment on a platform that has no relations (platforms/openremote.py).

Nothing above the adapter changed to accommodate this. The plan still contains a `create_relation` operation; the adapter reinterprets it. The exercise did force one change in the catalogue, and it was a change in the right direction: per-platform naming, originally a block inside each device entry, grew as devices multiplied by platforms, so it was moved into a file of its own indexed by platform. A third target is now one new section rather than an edit to every catalogue entry.

Two smaller findings from the same adapter. OpenRemote refuses an asset whose type declares mandatory attributes unless they are present, so the adapter reads the platform's asset model once and declares them empty rather than letting the reasoning

layer invent values. And reachability authenticates first, since reading needs no token while writing does: a check that only queried would call the platform reachable and then fail on the first write.

7.2.10 Deployment: interruption, confirmation and resumption

The deployment agent is where the SAIBA split between behaviour planning and behaviour realisation becomes control flow.

```
def __call__(self, state: IoTDeploymentState) -> dict:
    inst = Installation(**state.get("installation", {}))

    # Everything above the interrupt runs again when the architect answers,
    # so rules and operations follow the model as it stands now
    planned = self._plan(inst)
    if "deployment_plan" not in planned: # not valid, or unreachable
        return planned

    plan = self._carry_completed(
        [PlatformOperation(**op) for op in state.get("deployment_plan", [])],
        [PlatformOperation(**op) for op in planned["deployment_plan"]])

    # Suspends here with the plan; what comes back is whatever the
    # architect wrote next, and reading it is this agent's own business
    answer = interrupt({"operations": [op.model_dump() for op in plan],
                       "platform": inst.target.platform})
    if not self._confirms(answer):
        return {**planned, "messages": [AIMessage(content=(
            "[Deployment] Not executed. Nothing has been written.\n\n"
            + plan_to_markdown(plan))), ...]}
    return {**planned, **self._realise(inst, plan)}
```

Listing 7.22: The two phases, separated by the interrupt (agents/deployment.py).

The mechanics of `interrupt()` give the design property stated in Section 6.4: on resumption the node re-executes from its beginning, so validation and planning are performed *again*, against the model as it stands at the moment of execution. A confirmation therefore authorises a plan, not a session: if the configuration changed while the plan was waiting, what runs is the plan derived from the changed configuration, and the architect sees it.

Confirmation is read by a single structured call with a two-valued answer, and anything that is not a plain go-ahead leaves the platform untouched, whether it is a condition, a question or an unreadable reply. The default is inaction, which is the correct default for an irreversible operation.

Realisation stops at the first refusal rather than pressing on:

```
# One connection for the whole run, instead of one per operation
with adapter.batch():
    for op in plan:
        if op.status == "completed":
            continue
        try:
            op.result_ref = adapter.execute(op, refs)
            op.status = "completed"
            refs[op.op_id] = op.result_ref
        except Exception as e:
```

```

op.status = "failed"
op.error = f"{type(e).__name__}: {e}"
failure = op
break # stop at the first refusal; the rest stay pending

```

Listing 7.23: Execution, and what a failure leaves behind (`agents/deployment.py`).

Because operations carry model-derived identifiers, the completed ones are carried into the freshly derived plan on the next attempt and skipped, and the identifiers they produced are restored into the reference table so that later operations can still resolve their parents. The scenario of failure Section 6.6 is exactly this mechanism : a name conflict on one device, the entities already created left untouched, execution resuming from the operation that failed. The adapters close the loop from their own side: an asset whose name already exists is reused rather than duplicated, so confirming again after a conflict converges instead of colliding.

7.2.11 One reply per turn

Every supervisor architecture meets the same problem at the end of a turn: several agents have run, and the architect is owed one answer rather than a transcript of the delegation. Aggregating them is a standard step, and it is usually done by handing the agents' messages to a language model and asking for a summary.

This artifact keeps the step but changes what the model is given. Summarising prose means rewording it, and a reworded claim is a new claim that the model has no way of checking. The failures were not stylistic ones: a turn in which nothing ran was reported as done, and rooms that did not exist were named. This is also the last call before the architect reads anything, so whatever it invents arrives with the credibility of everything the system got right. The model is therefore confined to two short fields, and what the code rendered is placed around its answer rather than passed through it:

```

class Reply(BaseModel):
    account: str = Field(description=(
        "What you HAVE DONE this turn, in the language the architect wrote in. "
        "... Only what the facts state: no count, room or device that is not "
        "among them, and nothing about steps that did not run. Two or three "
        "sentences, no lists and no tables - the tables are added after you."
    ))
    next_step: str = Field(description=(
        "What the ARCHITECT can usefully do next, worked out from the facts "
        "and "
        "the order of the stages below. ... Never a stage the facts rule out: "
        "no deploying while there are errors, no devices before a use case."
    ))

# The model writes the two fields; the rendering is concatenated around them
# never passed through them. Only the last one: the earlier are the same
model
body = "\n\n".join(
    [self._clean(reply.account)] + rendered[-1:]
    + [self._clean(reply.next_step)])

```

Listing 7.24: What the model may write, and what is placed around it, abridged (`agents/response.py`).

Where an agent's prose ends and its rendering begins is decided in code, by the opening line each rendering is known to start with, so a table never reaches a language model at all. What the two fields may be written from is assembled in the same node:

```
lines = [SupervisorNode._model_status(state)]

if steps:
    lines.append("What ran this turn:")
    lines += [f"- {s['agent']} {s['action']}: {s['detail']}" for s in steps]
else:
    # Spelled out: 'no stage ran' was read as room to report what the
    # architect asked for as though it had actually happened
    lines.append("No stage ran this turn: nothing was created, changed,
        removed "
        "or deployed. ...")

# Which rooms there are, by name: told only how many, the model invents
# the ones it cannot name (a Classroom 3 that never existed)
if inst.building:
    for floor in inst.building.floors:
        named = ", ".join(f"{s.name} ({s.type})" for s in floor.spaces[:15])
        lines.append(f"Rooms on {floor.name}: {named or 'none'}."
            + (" And no others." if len(floor.spaces) <= 15 else ""))
```

Listing 7.25: The facts a reply may be written from, abridged (agents/response.py).

Each line guards against something the model had filled in on its own. Told how many rooms exist but not which, it named rooms that did not exist; told only that no stage ran, it reported the request as done anyway. Saying everything that is true, including what did *not* happen, is what leaves nothing to be guessed at.

That is the general point: an aggregating call is safe in proportion to how little it has to work out for itself. Given a closed list of true statements it is only choosing words, which is what language models are reliable at.

The reply is written in the language the architect used, decided by a small classification call rather than by an instruction in the prompt: told to answer “in the architect’s language”, the model answered in English. The agents’ own messages are then removed from the transcript, so the conversation holds one exchange per turn.

7.2.12 The interface

Streamlit re-runs the whole script on every interaction, so the interface cannot hold anything of its own: the panels are a redraw of what is stored. Two things must survive a rerun, the graph, which is slow to build, and one snapshot of the state, so that the panels agree with one another.

```
@st.cache_resource
def get_workflow() -> IoTAgenticWorkflow:
    """Builds the graph once and reuses it for every session. The only
    per-session thing is the configuration itself, and that lives in the
    checkpointer"""
    return IoTAgenticWorkflow()

def _values(self) -> dict:
    # Streamlit rebuilds this class on every rerun, so the cache lasts one
```

```
# rerun, which is what makes every panel see the same state
if self._cached_values is None:
    self._cached_values = self.workflow.get_state_values(self._session)
return self._cached_values
```

Listing 7.26: What must not be rebuilt on a rerun (ui/chat.py).

From that snapshot it draws the sidebar of previous configurations, the current installation and its diagram, the deployment plan, the conversation and the activity trace.

The only decision it makes is what a message means. The same box either starts a turn or answers a plan waiting on the interrupt; the interface asks the graph which of the two it is and routes it, without reading the answer itself.

```
# A plan waiting on the architect turns the next message into its answer;
# reading that is the deployment agent's job, so it passes through
if self.workflow.pending_approval(self._session):
    return self.workflow.resume(self._session, user_input, on_step)
return self.workflow.run_turn(self._session, user_input, on_step)

# While the graph is suspended the plan lives in the interrupt payload,
# not in the state: nothing written before an interrupt is kept
approval = self._approval()
plan = approval["operations"] if approval else self._values().get("
    deployment_plan")
```

Listing 7.27: One input box, two meanings (ui/chat.py).

The second half of that code follows from the graph: a node suspended on an interrupt has returned nothing, so the plan being confirmed is not in the stored state yet. It is read from the interrupt payload and drawn live.

A turn can run several agents, so the interface says which one is running, and takes that from the activity trace. The entries R7 asks for so that a session can be audited afterwards are also what reports progress while it is running.

```
def step(entry):
    agent = entry.get("agent")
    if agent == "Supervisor":
        going = (entry.get("detail") or "").split(" -> ")[0]
        status.update(label=f"Running {going}...")
    else:
        status.write(f"**{agent}** {entry.get('action')}")

reply = self._run_turn(user_input, on_step=step)
```

Listing 7.28: The trace, read as progress, abridged (ui/chat.py).

7.3 Results

This section reports what the implemented system does, in the order in which an architect encounters it. The figures are taken from the running prototype across different scenarios and runs, to show each capability.

7.3.1 Describing a building and completing the description

The activity begins with an unstructured description. The system extracts the topology, assigns a type to each space where the description supports one, and returns both the model it built and the questions that remain, expanding quantities into individually named spaces, so “twelve classrooms” becomes twelve rooms that can each be equipped, corrected and deployed.

The screenshot shows a dark-themed interface. At the top, a message from a user (indicated by a red speech bubble icon) describes a building: "The Computer Science Building in Camerino has 2 floors, each floor has 2 classrooms and a corridor the ground floor also has a technical room". Below this is a timestamp "12:45".

A second message from the system (indicated by an orange speech bubble icon) responds: "I modelled the Computer Science Building with its two floors and seven spaces: two classrooms and a corridor on the ground floor along with the technical room, and two classrooms and a corridor on the first floor."

Below the system message, three lines of text provide details:

- Installation:** Computer Science Building installation
- Building:** Computer Science Building (Camerino)
- Use case:** *not selected*

A table follows, summarizing the model:

Floor	Space	Type	Devices	Ways in
Ground floor	Classroom 1	classroom	<i>none</i>	<i>none</i>
Ground floor	Classroom 2	classroom	<i>none</i>	<i>none</i>
Ground floor	Corridor	corridor	<i>none</i>	<i>none</i>
Ground floor	Technical room	technical_room	<i>none</i>	<i>none</i>
First floor	Classroom 3	classroom	<i>none</i>	<i>none</i>
First floor	Classroom 4	classroom	<i>none</i>	<i>none</i>
First floor	Corridor	corridor	<i>none</i>	<i>none</i>

Below the table, a summary line states: "7 space(s), 0 way(s) in, 0 device(s) configured."

At the bottom, a prompt with a lightbulb icon asks: "Choose a use case for the building." followed by another timestamp "12:45".

Figure 7.3: A building described in natural language, and the model derived from it.

Where the description is incomplete, the missing information is requested rather than assumed. The questions come from the topology rules, so they are the same ones that would later block deployment, and a description given in several messages refines one model instead of restarting it.

 The Branch Site has one floor with Room A, Room B and Room C.
13:19

 I modelled the Branch Site with one floor and set up three spaces: Room A, Room B, and Room C.
Installation: Branch Site installation
Building: Branch Site
Use case: *not selected*

Floor	Space	Type	Devices	Ways in
Ground floor	Room A	unknown	<i>none</i>	<i>none</i>
Ground floor	Room B	unknown	<i>none</i>	<i>none</i>
Ground floor	Room C	unknown	<i>none</i>	<i>none</i>

3 space(s), 0 way(s) in, 0 device(s) configured.

 Tell me what kind of space each room is—choose from classroom, office, laboratory, meeting_room, corridor, technical_room, common_area, or outdoor.
13:19

 Room A and Room B are laboratories, Room C is a technical room.
13:19

 I updated the building model with the room types: Room A and Room B are now set as laboratories, and Room C as a technical room.
Installation: Branch Site installation
Building: Branch Site
Use case: *not selected*

Floor	Space	Type	Devices	Ways in
Ground floor	Room A	laboratory	<i>none</i>	<i>none</i>
Ground floor	Room B	laboratory	<i>none</i>	<i>none</i>
Ground floor	Room C	technical_room	<i>none</i>	<i>none</i>

3 space(s), 0 way(s) in, 0 device(s) configured.

 Choose a use case for the Branch Site deployment.
13:20

Figure 7.4: Clarification requested from the rules, and the model after the answer.

7.3.2 Reviewing the configuration

The configuration built so far can be requested at any point and is presented both as a table and as a topology diagram, each drawn from the stored model.

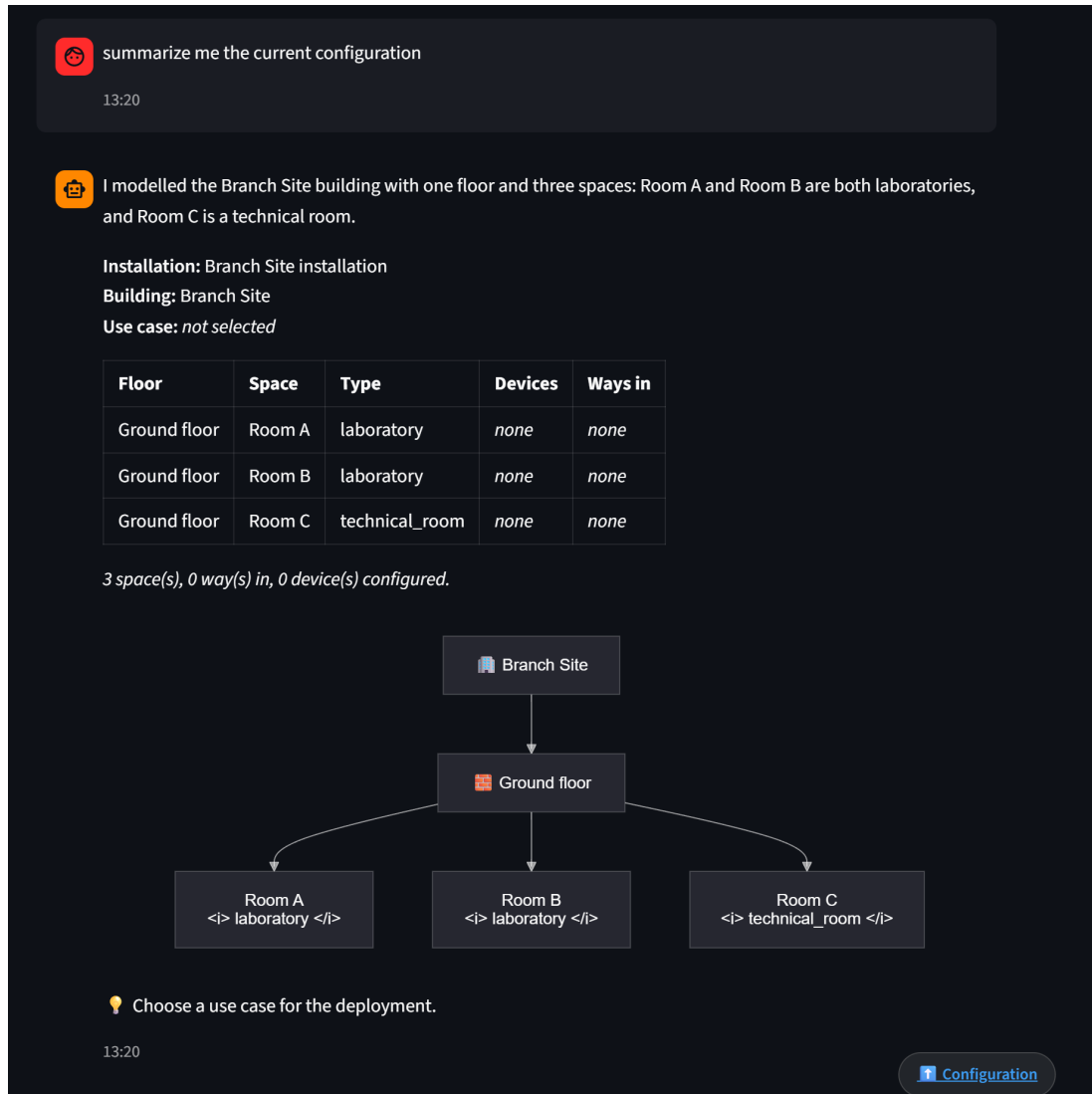


Figure 7.5: The configuration rendered as a table and as a topology diagram.

7.3.3 Consulting the catalogue and equipping the spaces

Selecting a use case loads its profile and the equipment it recommends; asked for the catalogue before a profile is chosen, the system lists all of it, as in Figure 7.6. The catalogue can then be consulted directly: which devices exist, the category and capabilities of each, and what kind of element it is installed on. Once a profile is in force the listing is narrowed to the equipment that profile recommends. Devices are assigned either to a named room or to every space of a kind, which is what removes the repetition that dominates a real installation.

 which devices are available??
13:21

 I listed the device catalog and found 7 devices available for the site.

Id	Name	Category	Capabilities	Installed on
dipme.monitoring_safety_multisensor	DIPME Monitoring & Safety Multisensor	multisensor	environmental_monitoring, structural_warning, commandable_mode	space
generic.environmental_sensor	Environmental Sensor	generic_sensor	environmental_monitoring	space
dipme.people_counter	People Counter	people_counter	presence_detection, occupancy_counting	access_point

Figure 7.6: The catalogue, answered out of the declarative knowledge base

 add an environmental sensor to all classrooms
12:49

 I placed an Environmental Sensor in each of the four classrooms: Classroom 1 and 2 on the Ground floor, and Classroom 3 and 4 on the First floor. The Edge Gateway needed for the wellness profile is in the Technical room on the Ground floor.

Installation: Computer Science Building installation
Building: Computer Science Building (Camerino)
Use case: wellness

Floor	Space	Type	Devices	Ways in
Ground floor	Classroom 1	classroom	Environmental Sensor	none
Ground floor	Classroom 2	classroom	Environmental Sensor	none
Ground floor	Corridor	corridor	none	none
Ground floor	Technical room	technical_room	Edge Gateway	none
First floor	Classroom 3	classroom	Environmental Sensor	none
First floor	Classroom 4	classroom	Environmental Sensor	none
First floor	Corridor	corridor	none	none

7 space(s), 0 way(s) in, 5 device(s) configured.

 Ask for a validation check when you are ready.

12:49

 Configuration

Figure 7.7: Assignment across every space of a type, with the defaults applied stated explicitly.

7.3.4 Checking before deploying

Validation reports errors and warnings separately against the rules of the active profile and pronounces on whether the configuration may proceed. Uncovered spaces, missing metadata, an absent gateway and a device placed where its type does not belong are all reported with the space they concern; a documented exclusion appears as such rather than as an omission.

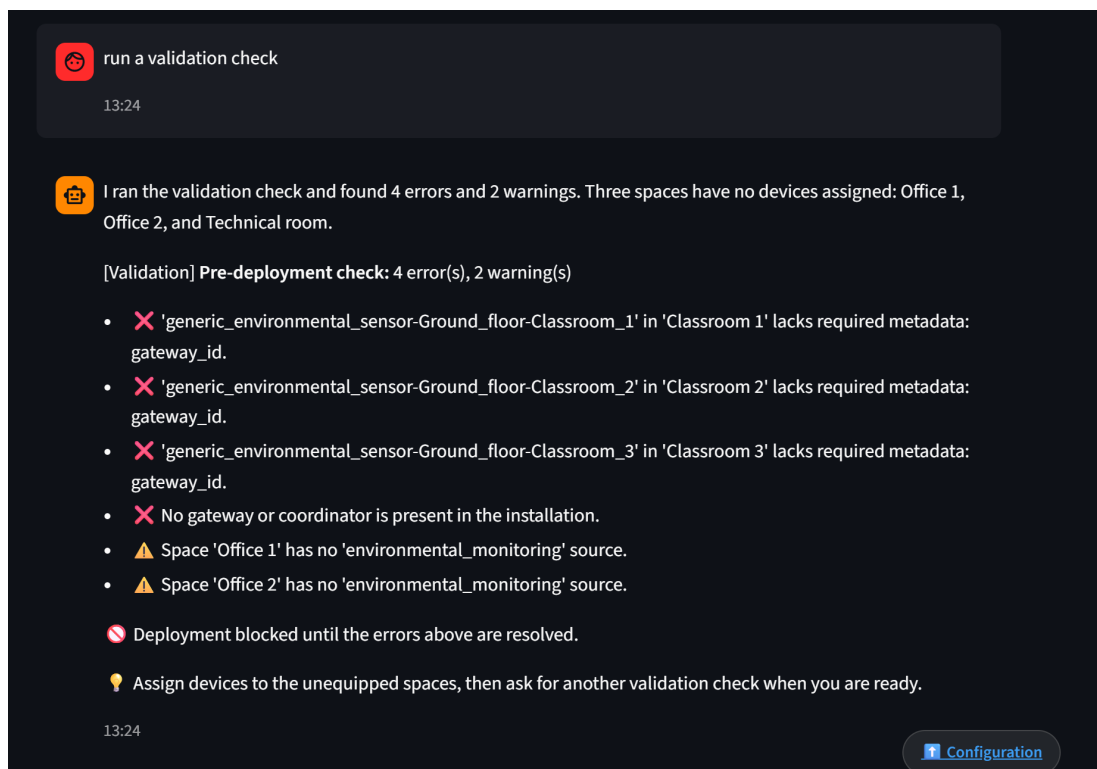


Figure 7.8: The pre-deployment check, with blocking errors distinguished from warnings.

7.3.5 Planning, confirming and deploying

Asking to deploy produces the ordered list of operations that would realise the configuration on the selected platform, and stops there. Nothing is written until the architect confirms in the conversation; a reply that refuses, postpones or attaches a condition leaves the platform untouched.

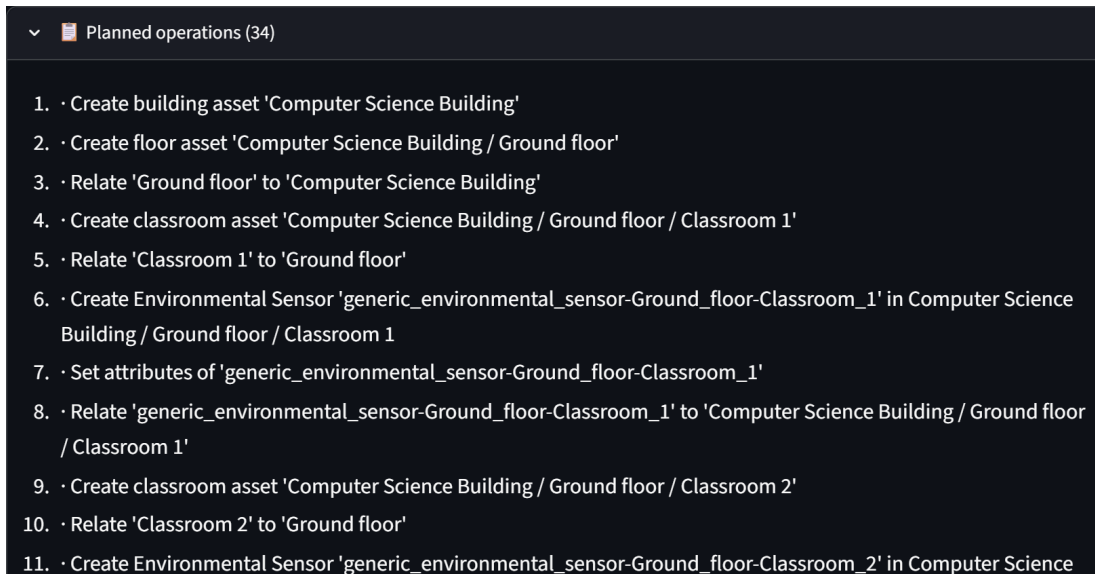


Figure 7.9: The deployment plan, suspended on the architect's confirmation, abridged.

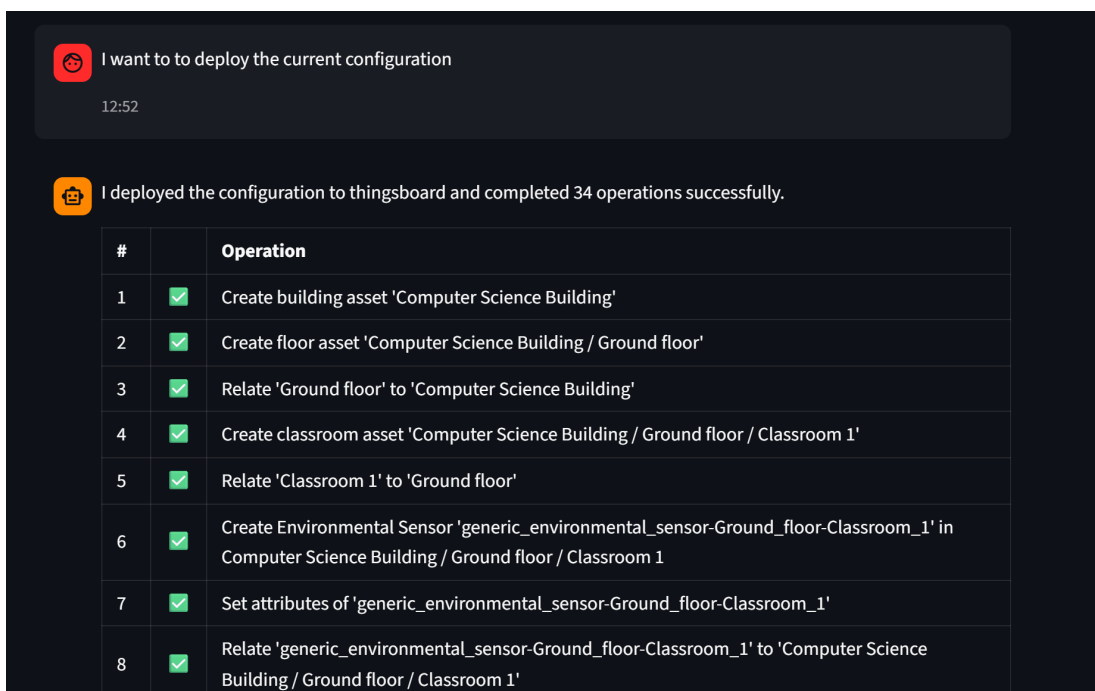
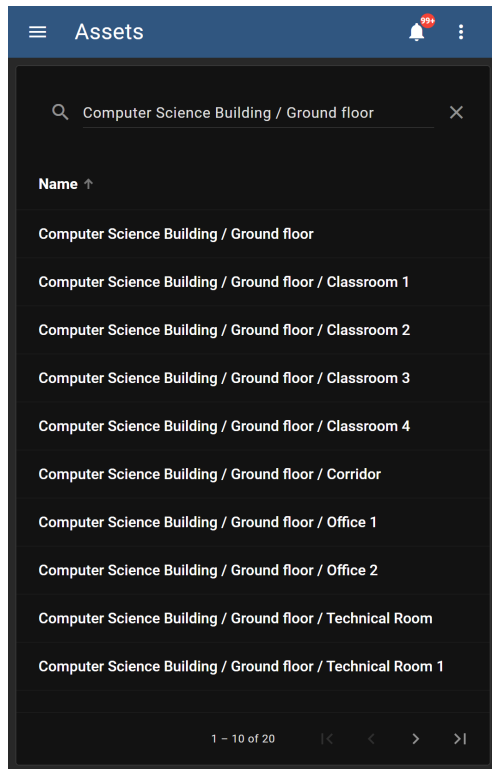


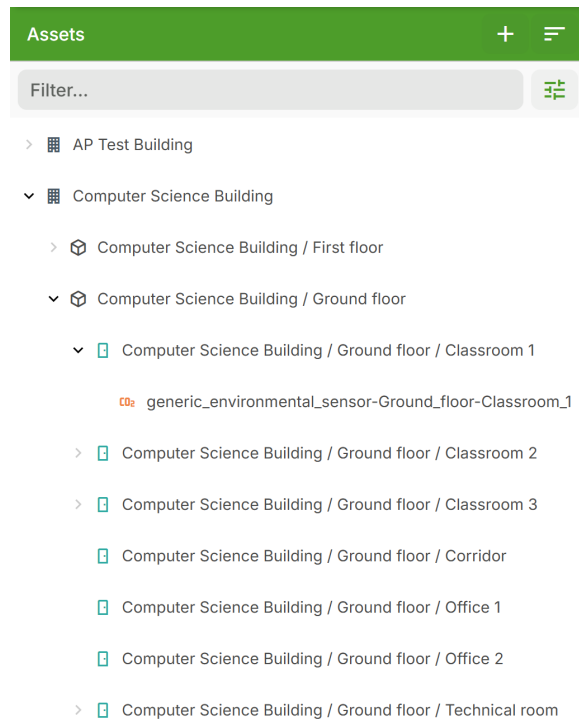
Figure 7.10: The plan after confirmation, with the outcome of each operation, abridged

7.3.6 The same model on two platforms

The architect can change between the platform on which the installation can be deployed. The installation reaches ThingsBoard as assets for the building, its floors and its spaces, related by containment, and as device entities carrying the metadata held in the model. The same model, deployed onto OpenRemote, becomes assets throughout, nested by parentage, a difference confined to the adapter and invisible in the plan the architect confirmed.



(a) ThingsBoard



(b) OpenRemote

Figure 7.11: The same installation as realised on ThingsBoard (a) and on OpenRemote (b).

7.3.7 Interruption and resumption

Configurations persist. A session is listed in the sidebar and reopened with its model, its validation report and its plan intact, and the activity continues from where it stopped without the building being described again.

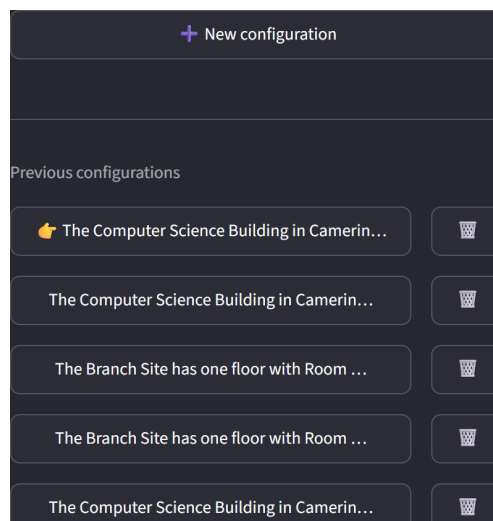
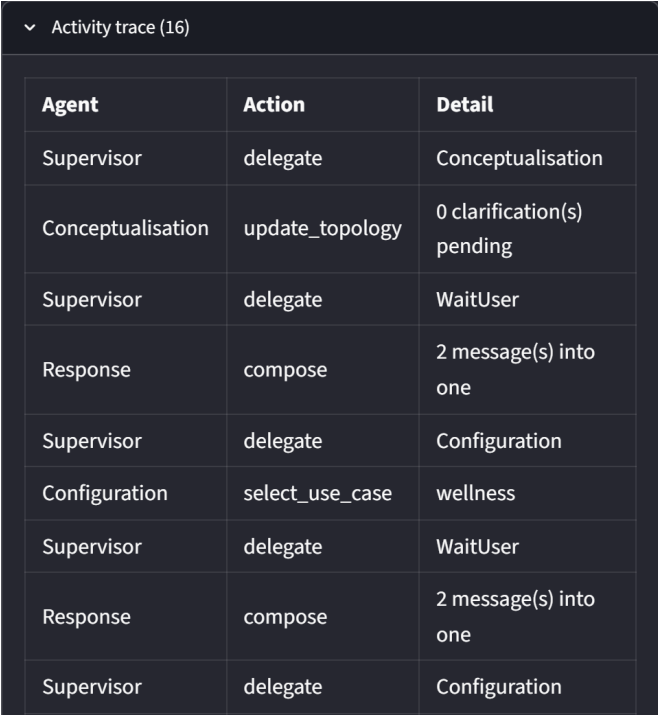


Figure 7.12: A configuration can be retrieved and continued in a later session through the sidebar selection

7.3.8 Traceability

Every delegation, every action and every outcome is recorded in the activity trace and remains available to the architect for the whole session, which is the auditability of R7: what the installation is can be read from the model, and how it came to be that can be read from the trace.



▼ Activity trace (16)		
Agent	Action	Detail
Supervisor	delegate	Conceptualisation
Conceptualisation	update_topology	0 clarification(s) pending
Supervisor	delegate	WaitUser
Response	compose	2 message(s) into one
Supervisor	delegate	Configuration
Configuration	select_use_case	wellness
Supervisor	delegate	WaitUser
Response	compose	2 message(s) into one
Supervisor	delegate	Configuration

Figure 7.13: The activity trace of a session.

7.3.9 Summary of the realised capabilities

Table 7.3 states what the implemented system does, against the needs collected in Section 5.2. Whether these capabilities are usable by their intended users, and whether the resulting installation is correct, is the subject of Chapter 8.

Capability	Requirement	Figure
Describe a building in natural language, in any of the supported languages	R1	7.3
Complete an incomplete description through clarification derived from the rules	R2	7.4
Review the configuration as a table and as a topology diagram	R2	7.5
Consult the catalogue and select a use-case profile	R4	7.6
Associate devices with one space or with every space of a type	R4	7.7
Check completeness and consistency before deploying	R5	7.8
Review the planned operations and confirm before any write	R5	7.9
Deploy onto ThingsBoard or OpenRemote without manual intervention	R6	7.11
Resume a configuration in a later session	R7	7.12
Inspect the trace of everything that ran	R7	7.13

Table 7.3: Capabilities of the implemented artifact.

8. Evaluation

8.1 Evaluation protocol

This section reports the protocol as it was planned before the first session: ten participants, two tasks, thirty minutes each, every participant using the assistant and no manual baseline. Section 8.2 states where the study departed from it.

8.1.1 Research question

RQ3. To what extent does the proposed agentic deployment loop allow a natural-language description to be translated into an installation that is correctly configured and realised on the target platform, and how do its intended users judge its usability?

The two halves are reported separately. *Correctness* is how many of the expected entities reached the platform and how often the system asserted something untrue; the stored model is the ground truth, so both are extracted automatically. *Usability* is the SUS at the end of the session and a difficulty question after each task.

There is no comparison condition. Configuring the same building by hand on the platform would roughly double the length of the session, and a study that fits in thirty minutes per participant is one that can actually be run with the people available. The study therefore describes the assistant rather than showing it to be better than manual configuration, and that limit is stated as such rather than argued away.

8.1.2 Participants

Ten participants, plus one pilot session excluded from the analysis. The only requirement is knowing what a sensor, a gateway and a IoT management platform are. Anyone who contributed to the artifact or has used it before is excluded, and prior use of ThingsBoard is recorded for each participant.

The language model and the commit of the artifact are fixed before the first session; the platform tenant and the assistant’s stored sessions are emptied between participants.

8.1.3 Tasks

Both tasks derive from the reference workflows of Section 6.6. Briefs are handed over printed, so that the facilitator’s phrasing does not end up in the participant’s own words.

Table 8.1 states the parameters that separate the two tasks, so that a replication can reproduce them rather than merely repeat their wording, and so that a difference in the results between S1 and S2 can be attributed to something stated in advance.

Parameter	S1	S2
Spaces to be modelled	4, all stated	4–6, none stated
Description completeness	Complete	Floors omitted
Device types involved	2	2
Expected platform entities	19 (10 entities, 9 relations)	Not fixed
Clarification turns required	0	At least 1
Check expected to report	Nothing	At least one finding
Time allowed	12 min	6 min

Table 8.1: Task parameters, stated so that the two tasks can be compared and reproduced.

S1 – Nominal configuration. Covers R1, R2, R4, R5 and R6.

North Teaching Wing. One floor. Two teaching rooms, one office, one technical room.

Environmental conditions need monitoring in the occupied spaces: an environmental sensor in each teaching room and in the office, and in the technical room the gateway the sensors report to.

Build this installation by conversing with the assistant, until it has been written to the platform. There is no right way to phrase things: use your own words.

Expected: one building, one floor, four spaces, three environmental sensors and one gateway, with the nine containment relations that link them (19 elements in total: 10 entities and 9 relations).

S2 – Incomplete description. Covers R2 and R5. The brief omits how the spaces are distributed across floors, so the system has to ask rather than infer, and the pre-deployment check has something to report.

Branch Site. There are some offices and a couple of laboratories. The installation concerns **seismic safety**.

Start from this description as it stands and get to a configuration the system considers ready for deployment. Supply the missing information as it is asked of you: you decide how the spaces are distributed.

8.1.4 Session

The facilitator does not help and does not comment on what the system does. Requests for help are answered with “it is your decision” and recorded, since each one is a datum.

8.1.5 Procedure

Consent The form states what the study is and who runs it, what the participant is asked to do and for how long, that the screen and the conversation are recorded, that the tool and not the participant is being evaluated, that data is held under a code whose link to the participant’s identity is destroyed after the analysis, and that participation may be stopped at any time without giving a reason.

Minutes	Step
00–04	Consent, screen recording started, three background questions
04–07	Two-minute demonstration on a dummy building used in neither task: where to type and how a deployment is confirmed, never how the task is solved
07–19	S1, stopped at 12 minutes, then the difficulty question
19–25	S2, then the difficulty question
25–30	SUS, then the three closing questions asked aloud

Table 8.2: Session structure.

Background questions Field of study or profession; experience with IoT systems (none / basic / hands-on / professional); prior use of ThingsBoard or an equivalent platform (never / once or twice / regularly).

Difficulty, after each task A five-point post-task rating of difficulty, of the kind compared by Sauro and Dumas [58] and since known as the Single Ease Question: *Overall, how difficult or easy was this task?* 1 = very difficult, 5 = very easy. That study found this format among the more sensitive at small sample sizes, which is the situation here, and it is the only instrument that lets the two tasks be compared with each other.

System Usability Scale, at the end The ten items of the SUS [59] on a five-point scale, 1 = strongly disagree, 5 = strongly agree. Items 2, 4, 6, 8 and 10 are reverse-scored.

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

Two further items on the same scale, but not part of the SUS score. They cover R5 and R2, which the SUS does not touch:

11. Before the system wrote anything to the platform, it was clear to me what it was about to do.
12. When the system misunderstood something, I was able to correct it.

8.1.6 Analysis and limitations

Descriptive statistics only. Ten participants is a defensible size for surfacing usability problems, which is what this study is for [60], but not one at which effects are estimated, and with a single condition no hypothesis test is warranted.

The SUS is scored in the usual way (odd items contribute $score - 1$, even items $5 - score$, and the sum is multiplied by 2.5) and read against the benchmark mean of 68, the fiftieth percentile of a curved grading scale built from several hundred usability studies [61]. It is reported with a 95% confidence interval whose width is stated plainly: with ten participants it was expected to be in the order of ± 10 points. Difficulty is reported as median and interquartile range per task.

Limitations The design of the study bounds what it can show. There is **no comparison condition**, so the study describes the assistant but cannot establish that it outperforms manual configuration. The ten participants are drawn from an academic rather than a professional population, which limits how far the results generalise to real operational settings. The experimenter is also the author of the artifact, a coincidence that can inflate subjective scores; a third-party facilitator would be preferable where one is available. A **novelty effect** is likely as well, since a conversational assistant is new and enjoyable in a way that an administrative interface is not.

Two further limits concern the artifact and the tasks. The reasoning layer is **non-deterministic**: fixing the model and the commit reduces the variation between runs but does not eliminate it, and divergent outcomes for equivalent requests are therefore reported as data rather than discarded. Finally, the **tasks** are **short**. Four rooms are not a university building, and conversational interfaces fare worst on short transactional tasks, so the tasks may not reflect how the assistant fares when configuring a full building.

8.2 Conduct of the study

The sessions took place between 11 and 16 September 2026. Seven people took part, fewer than the ten planned in Section 8.1. Every summary that depends on the size of the sample, the confidence interval of the SUS above all, is correspondingly wider than anticipated. The artifact was run at commit `c3fdb9a` with the model `anthropic/claude-haiku-4.5`. The time spent on each task was recorded in minutes in the questionnaire, alongside the ratings.

Two parts of the protocol were carried out differently. Correctness was not extracted automatically from the stored model, since the model is by construction what the platform receives: each S1 installation was instead compared with the brief, entity by entity and relation by relation, and the planned count of untrue statements made by the system was not performed. The sampling parameters of the model were left at the defaults of the provider.

8.2.1 Participants

All seven participants were university students. Four named a field in computer science or engineering: computer science (two), computer engineering and electrical engineering; the other three described themselves only as students. The Table 8.3 summarises their background.

Characteristic	Participants
<i>Experience with IoT systems</i>	
None	0
Basic	5
Hands-on	2
Professional	0
<i>Prior use of ThingsBoard or an equivalent platform</i>	
Never	1
Once or twice	4
Regularly	2
<i>Field of study</i>	
Computer science	2
Computer engineering	1
Electrical engineering	1
Not specified	3

Table 8.3: Background of the participants ($n = 7$).

8.2.2 Time on task

Participants spent a median of 10.5 minutes on S1, with a range of 8 to 12, and a median of 5 minutes on S2, with a range of 4 to 6 (Table 8.4). One participant, P04, reached the time limit on both tasks.

Task	Limit	n	Median	Range	At the limit
S1	12	7	10.5	8–12	1
S2	6	7	5	4–6	1

Table 8.4: Time on task, in minutes. *At the limit* counts the participants stopped when the time allowed ran out.

8.3 Usability

8.3.1 System Usability Scale

The mean SUS score is 60.7 (SD 22.0), below the benchmark of 68, with a 95% confidence interval of 40.4–81.0 (Table 8.5). The interval is twice as wide as the ± 10 points anticipated for ten participants, and it contains the benchmark: the data are compatible both with usability below average and with usability clearly above it. The median, 72.5, lies above the benchmark, and the gap between the two measures of centre is itself the main finding (Figure 8.1). Five scores lie between 55 and 82.5, four of them above 68, while two are much lower, 35 for P03 and 27.5 for P04, and pull the mean down. These two participants reported basic experience with IoT systems, gave the two lowest difficulty ratings for S1, and one of them reached the time limit on both tasks. With seven participants this is an observation rather than an explanation: basic experience is reported also by the two participants with the highest scores.

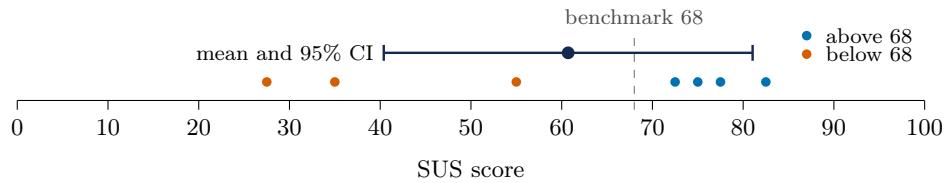


Figure 8.1: SUS score of each participant, with the mean and its 95% confidence interval. The dashed line marks the benchmark of 68.

Measure	Scale	Result
System Usability Scale	0–100	Mean 60.7 (SD 22.0), 95% CI 40.4–81.0; median 72.5, range 27.5–82.5; 4 of 7 above 68
Difficulty, task S1	1–5	Median 4, IQR 2.5–4
Difficulty, task S2	1–5	Median 3, IQR 2.5–4
Item 11: clear what would be written	1–5	Median 4, IQR 3–4
Item 12: able to correct misunderstandings	1–5	Median 4, IQR 3–4

Table 8.5: Usability measures ($n = 7$). Higher is better on every scale.

The means of the individual items show where the score comes from (Table 8.6). The strongest agreement concerns learnability: that most people would learn to use the system very quickly (item 7, mean 4.4). The weakest of the positively worded items are the wish to use the system frequently (item 1, 3.0) and confidence in using it (item 9, 3.1), and among the negatively worded items the one most agreed with is inconsistency (item 6, 3.1). With seven participants, differences of a few tenths between items indicate where to look rather than establishing a ranking.

Item	Statement	Mean (1–5)
1	I think that I would like to use this system frequently	3
2 [†]	I found the system unnecessarily complex	2.4
3	I thought the system was easy to use	3.4
4 [†]	I think that I would need the support of a technical person to be able to use this system	2.6
5	I found the various functions in this system were well integrated	3.3
6 [†]	I thought there was too much inconsistency in this system	3.1
7	I would imagine that most people would learn to use this system very quickly	4.4
8 [†]	I found the system very cumbersome to use	2.6
9	I felt very confident using the system	3.1
10 [†]	I needed to learn a lot of things before I could get going with this system	2.3

Table 8.6: Mean response to each SUS item ($n = 7$). For the negatively worded items, marked [†], a lower mean is better.

8.3.2 Task difficulty and additional items

On the five-point difficulty question, where 5 means very easy, S1 obtained a median of 4 (IQR 2.5–4) and S2 a median of 3 (IQR 2.5–4) (Figure 8.2). The first task was judged fairly easy and the second neither easy nor difficult, a difference consistent with the design of S2, whose description omits information on purpose and requires at least one clarification. The ratings of S1 are the more dispersed: four participants rated it 4, the other three 1, 2 and 3.

Items 11 and 12 both obtained a median of 4 (IQR 3–4). Four participants out of seven agreed that before any write it was clear what the system was about to do (R5), and four that they could correct the system when it misunderstood them (R2). Agreement is not strong on either item, however: only one participant answered 5 to each.

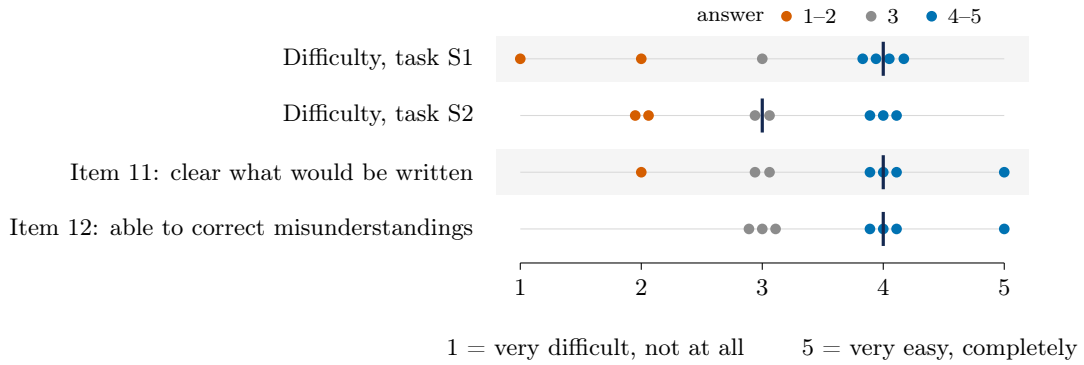


Figure 8.2: Difficulty of each task and responses to items 11 and 12, one dot per participant; the vertical bar marks the median. Higher is better on every row.

8.4 Overview by participant

Table 8.7 brings the two halves of the research question together, one row per participant, so that the outcome of the tasks can be read alongside the judgement given of them. It is meant to be read, not tested: with seven participants no association is estimated.

Participant	IoT experience	S1			S2		SUS
		Entities	Min	Difficulty	Min	Difficulty	
P01	Basic	19/19	10	4	5	3	55
P02	Hands-on	19/19	8	4	4	4	75
P03	Basic	16/19	11	2	5.5	2	35
P04	Basic	12/19	12	1	6	2	27.5
P05	Basic	19/19	10.5	4	4	4	77.5
P07	Hands-on	19/19	9	4	5	3	72.5
P08	Basic	19/19	11	3	5	4	82.5

Table 8.7: Results by participant. All participants completed both tasks. Difficulty on a scale from 1, very difficult, to 5, very easy; time in minutes, with limits of 12 for S1 and 6 for S2.

The two lowest SUS scores, 35 for P03 and 27.5 for P04, belong to the two participants who rated both tasks as the most difficult and who took the two longest times on S2; P04 also took the longest on S1 and reached the limit on both tasks. Both nonetheless completed the tasks. P03 and P04 had some errors on the connection between entities but still all of them were present in the system.

8.5 Summary

On correctness, all seven participants completed both tasks: every S1 installation was written onto the platform, and every S2 configuration was admitted for deployment. Five participants obtained the S1 installation exactly as expected, and the other two with limited discrepancies.

On usability, the SUS has a mean of 60.7 with a 95% confidence interval of 40.4–81.0, and a median of 72.5; four of seven participants scored above the benchmark of 68, and two much lower scores account for the gap between mean and median. On a scale from 1 to 5 the first task was judged fairly easy, with a median of 4, and the second neither easy nor difficult, with a median of 3. Items 11 and 12, on the transparency of the planned writes and the possibility of correcting the system, both had a median of 4.

9. Discussion

The preceding chapters reported what was designed, what was built and what happened when it was used. This chapter asks what those results mean: whether the gaps that motivated the work have been closed, to what extent the research questions can be considered answered, and where the boundaries of the contribution lie.

9.1 Revisiting the identified gaps

Chapter 3 closed with five gaps. The first four are answered in four different ways, and the fifth follows from them taken together.

The **semantic communication barrier** is crossed, but in one direction only. A building is described in prose, in the language the architect prefers, and equipment is requested in the same register; what makes this safe is not the language model but the restriction around it. The extraction schema has no device field, so equipment cannot enter through a description of the building; room types and access points survive only when the architect’s own words name them; device identifiers, metadata keys and platform names are closed vocabularies generated from the catalogue. Natural language decides what was asked, not what is written to the platform.

The **absence of unified hierarchical orchestration** is addressed by an architecture of exactly the missing shape: a supervisor that decides the stages of a turn, five specialised agents that execute them over one shared state, and a dedicated node that aggregates the turn into a single reply. Two details proved to matter more than the hierarchy itself. Deciding the whole queue of stages before any agent runs prevents the supervisor from reading an agent’s own output as a new request, which had produced turns reporting work that never happened; and confining the aggregating call to two short fields, with the deterministic renderings placed around its answer rather than passed through it, prevents the last call of the turn from rewording established facts. In a supervisor architecture the fragile points are the first and the last call, not the delegation between them.

Inconsistent infrastructure automation is resolved for the provisioning of structure. From the confirmed model the artifact derives an ordered plan, executes it itself and reports the outcome of each operation, and it does so on two platforms whose entity vocabularies and access protocols differ, with that difference confined to the adapter. It is not resolved for everything a platform holds: rules, dashboards, alarms and telemetry are outside the adapter contract, and so is the lifecycle after the first deployment.

The **state and reasoning gap** splits. Its state half is closed: the shared state is checkpointed after every node, a session is a thread identifier, operation identifiers derived from the model let a failed deployment resume from the operation that failed, and every delegation is appended to a readable trace. Its reasoning half is answered in

the terms in which C6 was stated: the language model interprets the request, and code decides what follows. Every failure worth reporting in Chapter 7 was instead a failure of unconstrained reasoning. The contribution is therefore not more autonomy but a division of labour: the language model is asked what the words say, and code decides what follows.

The **absence of any combination of intent, hierarchy and deployment** is the gap the artifact addresses most directly, because it is the sum of the answers above. Hi-MAD combines hierarchy and deployment without natural language, LUMEN combines natural language and hierarchy without deployment, and the orchestrator of Jahanbakhsh et al. combines natural language and deployment without a hierarchy of agents. The artifact holds the three together in a single loop: a description in prose is turned into a shared model by a hierarchy of agents, and that model is provisioned on the platform. Among the reviewed works this combination is new; its boundary is the one stated above, since what is deployed is the structure of an installation rather than its complete configuration.

9.2 Answers to the research questions

RQ1 – architectural design. The work identifies an architecture that carries an abstract, natural-language requirement through to a provisioned platform, and shows that it holds together in practice. It rests on four separations: a supervisor that decides the stages of a turn and specialised agents that execute them; a typed conceptual model shared by all of them and checkpointed rather than recomputed; domain knowledge held in declarative files rather than in code or prompts; and an adapter boundary that keeps the target platform out of everything above it. The last of these is the one the study tests hardest. The same model was realised on two platforms with different entity vocabularies and different access protocols, and no component above the adapter had to change, which is direct evidence that the deployment strategy the architecture produces is indeed platform-agnostic. Establishing that this arrangement is superior to other possible ones would require implementing and measuring alternatives, and that comparison is a natural continuation of the work rather than a question this artifact was built to settle.

RQ2 – implementation strategy. The work brings a concrete answer in the form of five rules, each of which earned its place by fixing an observed failure. Every decision is a declared schema rather than prose to be parsed. Every vocabulary the model chooses from is closed and generated from the knowledge base. The model may assert only what the architect said or what the stored model already holds. The delegation for a turn is decided once, before any agent has spoken. The irreversible step sits behind an interruption, and the plan is derived again on resumption, so that a confirmation authorises a plan rather than a session. Together these rules make a multi-step deployment workflow reliable in a specific and checkable sense: the properties that matter are decided by code and by declarative rules, so they can be inspected and reproduced without depending on the behaviour of the model. This is the part of the work most likely to transfer to agentic systems built for other domains.

RQ3 – empirical validation. The loop works end to end in the hands of people meeting it for the first time. All seven participants, none of whom had seen the system

before, described a building in their own words and reached a provisioned installation on the platform within the time allowed. The perceived experience was mixed: the mean SUS score is 60.7, below the benchmark of 68, with a confidence interval (40.4–81.0) that contains it; the median is 72.5 and four of the seven participants score above 68, the mean being pulled down by two much lower scores. The two items written for this study both have a median of 4 out of 5, indicating that participants generally understood what the system was about to write and were able to correct it when it misunderstood them. Five of the seven installations matched the intended configuration entirely. In the remaining two, every entity was created but some of the relations between them did not match what the participant intended, which confirms the value of the review and confirmation step that the design places before any write, and points to the extraction stage as the most useful target for further work. The usability result is best read as formative, which is also how the study was designed: with seven participants the confidence interval around the mean spans the benchmark, so the evidence supports the conclusion that the loop is usable enough for unaided first use, while a summative comparison against an established threshold would call for a larger sample of practising architects.

9.3 Limitations

9.3.1 Scalability

The artifact was tested on a building of the size of the reference scenario. Several of its mechanisms are designed for that size, and would need rethinking for a larger scenario, like a campus.

Every change costs as much as the whole model. The installation is stored as one object, saved after every node and checked again by every node that reads it. Renaming a single room in a building of several hundred spaces therefore rewrites and revalidates the entire model. The state is already typed and saved in versions, so the missing piece is the ability to write back one part of the model instead of all of it.

The prompts grow with the building. The extraction call receives the current model, and the node that writes the reply receives the rooms of each floor by name, up to fifteen per floor. Past a few hundred spaces, the description fills the space available to the extraction call, and the list of rooms given to the reply is no longer complete. That is precisely the situation in which the reply was seen to invent rooms, so the cap keeps the prompt small but does not keep the reply grounded. Retrieving the relevant part of the model, instead of listing all of it, is what would.

Deployment runs one operation at a time. The plan is executed serially through a single session and stops at the first refusal. A plan contains about two operations for each space and three for each device, so a large building means thousands of operations in a row. Only a parent and its children really have to be created in order, so separate parts of a building could be deployed in parallel. The current implementation does not do this.

The catalogue is copied into every schema. Device identifiers, use cases, metadata keys and platform names become `Literal` types generated from the catalogue. With seven device types and three profiles this is what keeps the model's choices grounded. With several hundred entries the same list would have to be sent with every request, and a search over the catalogue would have to replace it. Three fixed ceilings complete

the picture: three stages per turn, twenty devices of one type per room, and one user per checkpoint file.

9.3.2 Coverage and dependence on the model

The artifact creates a configuration, and only that. Removing an entity from the model does not remove it from the platform, and a platform edited by hand afterwards is not brought back into line. Both are operations that the adapter contract can accommodate, but neither is implemented. The knowledge base has a similar boundary: new equipment and new profiles are a YAML edit, while a new kind of rule could require changing the validation engine, and a room that does not fit the closed vocabulary is recorded as unknown.

All the results come from a single language model, used through a single gateway. Changing model is a matter of configuration rather than code, but how well a building is extracted, and how often grounding has to discard an invented room type, depend on that model as much as on the architecture. Neither was measured across models. The constraints limit where a weaker model can do harm, but they do not make the system deterministic: three calls remain whose mistakes the rules cannot catch. They are the reading of what the architect's words name, the reading of the architect's confirmation, and the reply itself. The confirmation is the most sensitive of the three, since a refusal misread as approval is the only one of these errors that leads to a write on the platform.

9.3.3 Evaluation and operation

The study was formative by design, and this sets its limits. Seven participants took part, recruited by convenience and students rather than practising architects. Each used the system for thirty minutes, on two scripted tasks given in the same order, with the author acting as facilitator and later assessing the results. Both tasks had a time limit, which one participant reached in each, so the reported times say what fitted into the time allowed. Correctness was judged by comparing each installation with the task description and counting the entities that matched the intent, without weighing how serious a mismatch would be in practice. Nothing was measured about how long the same configuration takes by hand, so the claim that the loop removes repetitive work rests on the shape of the activity rather than on a comparison. No participant used the system twice, so learning, longer configurations and the durability of the confirmation habit remain untested. These are limits of this study rather than of the artifact, and each one describes the study that should follow it.

The prototype is a research instrument in the same sense. Credentials are kept in environment variables, the OpenRemote adapter signs in as a service user with broad rights, and the trace is written for the architect who ran the session rather than as an audit log, stored together with every earlier configuration in one unencrypted file. Reducing those privileges and separating the audit record from the session state would be necessary before any use outside a controlled setting.

9.4 Implications

Two lessons reach beyond this artifact.

The first is about how a language model can be used in a system that acts. Through-

out this work the model is asked only what the architect's words say, not what should follow from them, and the consequences are worked out in code. A room type survives only if it was named, the deployment plan is derived from the stored model rather than written by the model, and the final reply is assembled around tables the code produced. This costs expressiveness and gains checkability, and the failures collected here suggest the exchange is worth making wherever a rule can decide instead. Nothing about it is specific to IoT, so it should hold for any agentic system whose actions cannot be undone.

The second is about the boundary between the system and the platform. Adding a second platform required no change above the adapter, and the harder of the two has neither a device entity nor a relation entity: it was accommodated by reinterpreting an operation, not by weakening the plan. A contract of four operations and a reachability check was therefore enough for two platforms that work quite differently, which suggests that provisioning is uniform enough between IoT platforms, at least in its structure, to be worth abstracting. Whether it stays uniform once rules and dashboards are included is the next thing the design should be tested against.

10. Conclusion

This thesis started from a practical problem. Configuring an IoT installation on a platform is work that an architect does one entity at a time, in the vocabulary of the platform rather than that of the building, and with nothing that checks the result before it is written. The literature reviewed in Chapter 3 showed that this is not only an operational inconvenience. Existing systems start from structured metrics rather than from a description in the architect’s own words, rarely divide the work among specialised agents, often limit automation to the edge layer, and frequently keep no state across the steps of a multi-stage operation.

The work answered this problem by building an artifact and by testing it with users. A single conceptual model of the installation, shared by every agent, carries a building from a description in prose to a provisioned platform. A supervisor decides which stages each request needs and delegates them to specialised agents. A declarative catalogue limits what can be selected, a validation engine decides what may proceed, and an explicit confirmation stands before the only step that cannot be undone. An adapter keeps the target platform separate from everything above it. The prototype realises the same model on ThingsBoard, through its MCP server, and on OpenRemote, through its REST API, and it recovers from an interrupted session or a partial failure by resuming from the operation that failed.

10.1 Contributions

The work makes four contributions.

- **An architecture** for turning a natural-language description into a provisioned IoT installation. A supervisor delegates the stages of the configuration to specialised agents that share one checkpointed conceptual model, and the target platform is kept behind an adapter contract of four operations and a reachability check. The architecture was realised on two platforms whose entity models and access protocols differ, without any change above the adapter.
- **A set of rules for using a language model inside a system that acts.** Every decision is a declared schema, every vocabulary is closed and generated from the knowledge base, every assertion must be supported by what the architect said or by the stored model, the delegation of a turn is decided once, and the final reply is built around fact certified by the code and not invented. Each rule is reported together with the failure that motivated it, and none of them depends on IoT, so they are the part of the work most likely to be reused in other agentic systems.
- **A working, open prototype** that runs on a laptop and is released with its

source code, so that the architecture, the rules and the behaviour reported in this thesis can be inspected and reproduced.

- **A first empirical account** of how people use such a system when they meet it for the first time. All seven participants described a building in their own words and reached a provisioned installation, one of them only at the time limit. The mean SUS score was 60.7 (median 72.5), with four of the seven above the benchmark of 68, and the two installations whose relations did not match the brief show where the next improvements should be made.

10.2 Future work

The limitations discussed in Section 9.3 point to four directions.

Making the system scale with the change rather than with the building. At present every modification rewrites the whole model, the extraction call has to return the entire building each time, and the plan is executed one operation at a time. Writing back only the part of the model that changed, giving the language model only the part of the building or of the catalogue that a request concerns, and deploying independent parts of the building in parallel would remove these bottlenecks without changing the architecture.

Completing the lifecycle of an installation. The artifact creates a configuration but does not update, remove or reconcile it. Extending the adapter contract with these operations, and adding a step that compares the model with what the platform actually holds, would make the system usable on buildings that keep changing, which is the normal case. Extending the same contract to rules and dashboards would show how far the uniformity observed between the two platforms really goes.

Measuring what has so far been observed. The quality of extraction and of routing, and the frequency with which grounding has to discard invented information, were watched throughout development but never measured, and all results come from a single model. A set of building descriptions with known expected models, run against several models, would separate the merits of the architecture from those of the model in use. The same benchmark would also be the right place to strengthen the reading of the architect's confirmation, which is the one model call whose mistake can lead to a write.

Evaluating the system against current practice. The central claim of the artifact is that it removes repetitive work, and the natural next study compares it directly with that work. Practising IoT architects could carry out the same configuration both through the platform consoles and through the artifact, with correctness judged by someone other than the author. A longitudinal version of the study would also show whether users keep reading the confirmation carefully once it has become routine.

This work does not present a system that configures an IoT installation on its own. It shows that two goals often seen as being in tension can be achieved together. An architect can describe a building in their own words, and every entity that reaches the platform can still be one that the catalogue, the validation rules and an explicit confirmation have allowed through. The evidence collected here suggests that this combination is what makes the system usable, and also what keeps its mistakes small enough to be caught.

Acknowledgements

I would like to thank my supervisor, Prof. Massimo Callisto De Donato, for the guidance throughout the work, his enviable availability and for proposing this topic to me.

My thanks go to my co-supervisor, Prof. Stefán Ólafsson. His course on Natural Language Processing was one of the most engaging of my studies: it is where language stopped being only something we speak and became something a machine can be asked to understand, and much of the curiosity behind this thesis started there.

I am deeply grateful to the University of Camerino and to Reykjavik University for the opportunity to take part in the Double Degree programme. Studying in two countries, within two academic traditions and among people I would never have met otherwise, has shaped these years far more than any single course could have done. I arrived in Iceland with a study plan and came back with a different way of looking at my own work, at the whole world around me and I consider that the most valuable thing I take with me.

Alla mia famiglia, in Italiano, perché certe cose si dicono nella lingua di casa.

A mio padre e a mia madre, che mi hanno permesso di arrivare fin qui supportandomi e credendo in me ognuno a modo loro, ma sicuramente con tutto il cuore che potevano.

A mia sorella, che c'è sempre stata, per ascoltare, per capire, per comprendere senza giudicare...

And for everyone else: those who know, know. *Chi sa, sa.*



Declaration of Originality and Use of AI

I declare that I have used generative artificial intelligence tools within the limits allowed by the University Guidelines (Rectoral Decree no. 60/2026) and in agreement with my Supervisors. In particular, I used Claude (Anthropic), model Claude Sonnet 5, during the period [05/2026 – 09/2026] as support for: brainstorming on topic, title and structure; orientation in the literature search, with every source verified in official scientific databases; language revision and rephrasing suggestions for texts written by me; translations, summaries and outlines of texts I had already read; suggestions for code, charts and figures; preparation of the thesis presentation. All content has been verified and revised by me, and I take full responsibility for it.

Bibliography

- [1] Omer Ali et al. ‘A Comprehensive Review of Internet of Things: Technology Stack, Middlewares, and Fog/Edge Computing Interface’. en. In: *Sensors* 22.3 (Jan. 2022), p. 995. ISSN: 1424-8220. DOI: [10.3390/s22030995](https://doi.org/10.3390/s22030995). URL: <https://www.mdpi.com/1424-8220/22/3/995> (visited on 01/06/2026).
- [2] Jie Lin et al. ‘A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications’. In: *IEEE Internet of Things Journal* 4.5 (Oct. 2017), pp. 1125–1142. ISSN: 2327-4662, 2372-2541. DOI: [10.1109/JIOT.2017.2683200](https://doi.org/10.1109/JIOT.2017.2683200). URL: <https://ieeexplore.ieee.org/document/7879243/> (visited on 01/06/2026).
- [3] Qiaomu Li and Ying Xie. ‘From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems’. en. In: ().
- [4] Ashis Kumar Pati. ‘Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications’. In: *IEEE Access* 13 (2025), pp. 151824–151837. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2025.3585609](https://doi.org/10.1109/ACCESS.2025.3585609). URL: <https://ieeexplore.ieee.org/document/11071266/> (visited on 30/04/2026).
- [5] Ajay Bandi et al. ‘The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges’. en. In: *Future Internet* 17.9 (Sept. 2025), p. 404. ISSN: 1999-5903. DOI: [10.3390/fi17090404](https://doi.org/10.3390/fi17090404). URL: <https://www.mdpi.com/1999-5903/17/9/404> (visited on 03/05/2026).
- [6] Ranjan Sapkota, Konstantinos I. Roumeliotis and Manoj Karkee. ‘AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and challenges’. en. In: *Information Fusion* 126 (Feb. 2026), p. 103599. ISSN: 15662535. DOI: [10.1016/j.inffus.2025.103599](https://doi.org/10.1016/j.inffus.2025.103599). URL: <https://linkinghub.elsevier.com/retrieve/pii/S1566253525006712> (visited on 04/05/2026).
- [7] Simone Gallo, Fabio Paternò and Alessio Malizia. ‘A conversational agent for creating automations exploiting large language models’. In: *Personal and Ubiquitous Computing* 28.6 (2024), pp. 931–946. DOI: [10.1007/s00779-024-01825-5](https://doi.org/10.1007/s00779-024-01825-5).
- [8] Adela Nedisan Videsjorden et al. ‘LUMEN: Enhancing IoT System Observability with Multi-Agent Large Language Models and Knowledge Graphs’. In: *ACM Transactions on Internet of Things* 7.2 (2026), pp. 1–40. DOI: [10.1145/3772077](https://doi.org/10.1145/3772077).
- [9] Stefan Kopp et al. ‘Towards a common framework for multimodal generation: The behavior markup language’. In: *International workshop on intelligent virtual agents*. Springer. 2006, pp. 205–217.

- [10] Yuan He, Junchen Guo and Xiaolong Zheng. ‘From Surveillance to Digital Twin: Challenges and Recent Advances of Signal Processing for Industrial Internet of Things’. In: *IEEE Signal Processing Magazine* 35 (5 Sept. 2018), pp. 120–129. ISSN: 1558-0792. DOI: [10.1109/MSP.2018.2842228](https://doi.org/10.1109/MSP.2018.2842228).
- [11] Jiasi Chen and Xukan Ran. ‘Deep Learning with Edge Computing: A Review’. In: *Proceedings of the IEEE* 107 (8 Aug. 2019), pp. 1655–1674. ISSN: 1558-2256. DOI: [10.1109/JPROC.2019.2921977](https://doi.org/10.1109/JPROC.2019.2921977).
- [12] Ala Al-Fuqaha et al. ‘Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications’. In: *IEEE Communications Surveys and Tutorials* 17 (4 Oct. 2015), pp. 2347–2376. ISSN: 1553-877X. DOI: [10.1109/COMST.2015.2444095](https://doi.org/10.1109/COMST.2015.2444095).
- [13] ThingsBoard, Inc. *ThingsBoard Documentation*. URL: <https://thingsboard.io/docs/> (visited on 14/09/2026).
- [14] OpenRemote. *OpenRemote Documentation*. URL: <https://docs.openremote.io/> (visited on 14/09/2026).
- [15] Mads Holten Rasmussen et al. ‘BOT: The building topology ontology of the W3C linked building data group’. In: *Semantic Web* 12.1 (Nov. 2020). Ed. by Krzysztof Janowicz, pp. 143–161. ISSN: 22104968, 15700844. DOI: [10.3233/SW-200385](https://doi.org/10.3233/SW-200385). URL: <https://journals.sagepub.com/doi/full/10.3233/SW-200385> (visited on 08/08/2026).
- [16] Laura Daniele, Frank den Hartog and Jasper Roes. ‘Created in Close Interaction with the Industry: The Smart Appliances REference (SAREF) Ontology’. In: *Formal Ontologies Meet Industry*. Vol. 225. Lecture Notes in Business Information Processing. Springer, 2015, pp. 100–112. DOI: [10.1007/978-3-319-21545-7_9](https://doi.org/10.1007/978-3-319-21545-7_9).
- [17] Bharathan Balaji et al. ‘Brick : Metadata schema for portable smart building applications’. en. In: *Applied Energy* 226 (Sept. 2018), pp. 1273–1292. ISSN: 03062619. DOI: [10.1016/j.apenergy.2018.02.091](https://doi.org/10.1016/j.apenergy.2018.02.091). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0306261918302162> (visited on 08/08/2026).
- [18] Dirk Heylen et al. *AAMAS 2009 Workshop Towards a Standard Markup Language for Embodied Dialogue Acts PROCEEDINGS*. Tech. rep. 2009.
- [19] Binfeng Xu et al. *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*. 2023. arXiv: [2305.18323](https://arxiv.org/abs/2305.18323).
- [20] Yongliang Shen et al. ‘HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face’. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. arXiv: [2303.17580](https://arxiv.org/abs/2303.17580).
- [21] Anthropic. *Building Effective Agents*. 19th Dec. 2024. URL: <https://www.anthropic.com/engineering/building-effective-agents> (visited on 15/09/2026).
- [22] Junlin Wang et al. *Mixture-of-Agents Enhances Large Language Model Capabilities*. 2024. arXiv: [2406.04692](https://arxiv.org/abs/2406.04692).
- [23] LangChain, Inc. *LangGraph Documentation*. URL: [https://github.com/langchain-ai/langgraph/](https://github.com/langchain-ai/langgraph) (visited on 14/09/2026).
- [24] Anand S. Rao and Michael P. Georgeff. ‘BDI Agents: From Theory to Practice’. In: *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*. 1995, pp. 312–319.

- [25] Brandon T. Willard and Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: [2307.09702](#).
- [26] Patrick Lewis et al. ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. arXiv: [2005.11401](#).
- [27] Traian Rebedea et al. ‘NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails’. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2023. arXiv: [2310.10501](#).
- [28] Abul Ehtesham et al. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. arXiv:2505.02279 [cs]. May 2025. DOI: [10.48550/arXiv.2505.02279](#). URL: <http://arxiv.org/abs/2505.02279> (visited on 06/05/2026).
- [29] Aristeia Athanasopoulou, Nikos Fotiou and Avraam Chatzopoulos. ‘Interacting with IoT Data Spaces Using LLMs and the Model Context Protocol’. en. In: *Sensors* 26.4 (Feb. 2026), p. 1193. ISSN: 1424-8220. DOI: [10.3390/s26041193](#). URL: <https://www.mdpi.com/1424-8220/26/4/1193> (visited on 06/05/2026).
- [30] Model Context Protocol. *Model Context Protocol Specification, version 2025-06-18*. URL: <https://modelcontextprotocol.io/specification/2025-06-18> (visited on 15/09/2026).
- [31] BeeAI. *ACP Joins Forces with A2A Under the Linux Foundation*. 25th Aug. 2025. URL: <https://github.com/orgs/i-am-bee/discussions/5> (visited on 15/09/2026).
- [32] LangChain, Inc. *LangChain MCP Adapters*. URL: <https://github.com/langchain-ai/langchain-mcp-adapters> (visited on 15/09/2026).
- [33] LangChain, Inc. *LangChain*. URL: <https://github.com/langchain-ai/langchain> (visited on 15/09/2026).
- [34] LlamaIndex, Inc. *LlamaIndex*. URL: https://github.com/run-llama/llama_index (visited on 15/09/2026).
- [35] Microsoft. *Microsoft Agent Framework*. URL: <https://github.com/microsoft/agent-framework> (visited on 15/09/2026).
- [36] CrewAI, Inc. *CrewAI*. URL: <https://github.com/crewAIInc/crewAI> (visited on 15/09/2026).
- [37] OpenAI. *OpenAI Agents SDK*. URL: <https://github.com/openai/openai-agents-python> (visited on 15/09/2026).
- [38] Flavio Corradini et al. ‘X-IoT: a model-driven approach to support IoT application portability across IoT platforms’. en. In: *Computing* 105.9 (Sept. 2023), pp. 1981–2005. ISSN: 0010-485X, 1436-5057. DOI: [10.1007/s00607-023-01155-z](#). URL: <https://link.springer.com/10.1007/s00607-023-01155-z> (visited on 02/06/2026).

- [39] David Carrascal et al. ‘Softwarized Edge Intelligence for Advanced IIoT Ecosystems: A Data-Driven Architecture Across the Cloud/Edge Continuum’. en. In: *Applied Sciences* 15.19 (Oct. 2025), p. 10829. ISSN: 2076-3417. DOI: [10.3390/app151910829](https://doi.org/10.3390/app151910829). URL: <https://www.mdpi.com/2076-3417/15/19/10829> (visited on 25/06/2026).
- [40] Fatima-Zohra Bemrah, Azeddine Chikh and Samir Ouchani. ‘Hi-MAD: A Hierarchical Multi-Agent DRL framework for resilience and service optimization in IoT systems’. en. In: *Information and Software Technology* 196 (Aug. 2026), p. 108151. ISSN: 09505849. DOI: [10.1016/j.infsof.2026.108151](https://doi.org/10.1016/j.infsof.2026.108151). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0950584926001400> (visited on 12/06/2026).
- [41] Chun-Chih Lin et al. ‘Achieving Robust Resource Orchestration for Highly Dense Heterogeneous IoT Systems’. In: *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. London, United Kingdom: IEEE, May 2025, pp. 1–10. ISBN: 979-8-3315-4305-1. DOI: [10.1109/INFOCOM55648.2025.11044551](https://doi.org/10.1109/INFOCOM55648.2025.11044551). URL: <https://ieeexplore.ieee.org/document/11044551/> (visited on 15/06/2026).
- [42] Jie Deng, Yongxu Zhu and Fu Xiao. ‘Leveraging large language model agents for cost-effective sensor data handling and urban traffic navigation’. en. In: *Engineering Applications of Artificial Intelligence* 173 (June 2026), p. 114363. ISSN: 09521976. DOI: [10.1016/j.engappai.2026.114363](https://doi.org/10.1016/j.engappai.2026.114363). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0952197626006445> (visited on 08/06/2026).
- [43] Shuaishuai Guo et al. ‘Task-Oriented Communications for Agentic IoT: An LLM-Driven QoS/Security Policy Generation via Dynamic Model Context Protocol’. In: *2025 Seventeenth International Conference on Wireless Communications and Signal Processing (WCSP)*. Chongqing, China: IEEE, Oct. 2025, pp. 1–5. ISBN: 979-8-3315-8303-3. DOI: [10.1109/WCSP68525.2025.1010287](https://doi.org/10.1109/WCSP68525.2025.1010287). URL: <https://ieeexplore.ieee.org/document/11352215/> (visited on 26/06/2026).
- [44] Gaetano Cimino, Vincenzo Deufemia and Mattia Limone. ‘IoT Rule Generation With Cross-View Contrastive Learning and Perplexity-Based Ranking’. In: *IEEE Internet of Things Journal* 12.17 (2025), pp. 35828–35847. DOI: [10.1109/JIOT.2025.3579920](https://doi.org/10.1109/JIOT.2025.3579920).
- [45] Negin Jahanbakhsh et al. ‘Leveraging Retrieval-Augmented Generation for Automated Smart Home Orchestration’. In: *Future Internet* 17.5 (2025), p. 198. DOI: [10.3390/fi17050198](https://doi.org/10.3390/fi17050198).
- [46] Alan R. Hevner et al. ‘Design Science in Information Systems Research’. In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: [10.2307/25148625](https://doi.org/10.2307/25148625).
- [47] Alan R. Hevner. ‘A Three Cycle View of Design Science Research’. In: *Scandinavian Journal of Information Systems* 19.2 (2007), pp. 87–92.
- [48] Salvatore T. March and Gerald F. Smith. ‘Design and natural science research on information technology’. In: *Decision Support Systems* 15.4 (1995), pp. 251–266. DOI: [10.1016/0167-9236\(94\)00041-2](https://doi.org/10.1016/0167-9236(94)00041-2).
- [49] Vijay K. Vaishnavi and William Kuechler. *Design Science Research Methods and Patterns: Innovating Information and Communication Technology*. 2nd ed. Boca Raton, FL: CRC Press, 2015.

-
- [50] Ken Peffers et al. ‘A Design Science Research Methodology for Information Systems Research’. In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
 - [51] John Venable, Jan Pries-Heje and Richard Baskerville. ‘FEDS: a Framework for Evaluation in Design Science Research’. In: *European Journal of Information Systems* 25.1 (2016), pp. 77–89. DOI: [10.1057/ejis.2014.36](https://doi.org/10.1057/ejis.2014.36).
 - [52] Laura Daniele, Frank den Hartog and Jasper Roes. ‘Created in close interaction with the industry: the smart appliances reference (SAREF) ontology’. In: *International Workshop Formal Ontologies Meet Industries*. Springer. 2015, pp. 100–112.
 - [53] Aidan Hogan et al. ‘Knowledge Graphs’. In: *ACM Computing Surveys* 54.4 (2021), 71:1–71:37. DOI: [10.1145/3447772](https://doi.org/10.1145/3447772).
 - [54] Lars-Peter Meyer et al. ‘Assessing SPARQL Capabilities of Large Language Models’. In: *NLP4KGc Workshop at SEMANTiCS 2024*. Vol. 3874. CEUR Workshop Proceedings. 2024, pp. 35–53.
 - [55] Marco Pritoni et al. ‘Metadata Schemas and Ontologies for Building Energy Applications: A Critical Review and Use Case Analysis’. In: *Energies* 14.7 (2021), p. 2024. DOI: [10.3390/en14072024](https://doi.org/10.3390/en14072024).
 - [56] Pavan Belagatti. *LangChain vs LangGraph: How to Choose the Right AI Framework!* DEV Community. 4th Dec. 2025. URL: <https://dev.to/pavanbelagatti/langchain-vs-langgraph-how-to-choose-the-right-ai-framework-497h> (visited on 26/09/2026).
 - [57] OpenRouter. *How OpenRouter Model Routing Works*. OpenRouter Blog. 12th June 2026. URL: <https://openrouter.ai/blog/insights/model-routing/> (visited on 26/09/2026).
 - [58] Jeff Sauro and Joseph S. Dumas. ‘Comparison of three one-question, post-task usability questionnaires’. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. ACM, 2009, pp. 1599–1608. DOI: [10.1145/1518701.1518946](https://doi.org/10.1145/1518701.1518946).
 - [59] John Brooke. ‘SUS: a ‘quick and dirty’ usability scale’. In: *Usability Evaluation in Industry*. Ed. by Patrick W. Jordan et al. London: Taylor & Francis, 1996, pp. 189–194.
 - [60] Jakob Nielsen and Thomas K. Landauer. ‘A mathematical model of the finding of usability problems’. In: *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems*. ACM, 1993, pp. 206–213. DOI: [10.1145/169059.169166](https://doi.org/10.1145/169059.169166).
 - [61] Jeff Sauro and James R. Lewis. *Quantifying the User Experience: Practical Statistics for User Research*. 2nd ed. Cambridge, MA: Morgan Kaufmann, 2016.